



المعهد العالي للإعلامية وتكنولوجيايات الاتصال بسوسة

Rapport de stage de fin d'études

Présenté en vue de l'obtention du diplôme de Licence en Ingénierie des
Systèmes Informatiques (Computer Engineering)

Spécialité : Systèmes Embarqués et Internet des Objets

**Commande Numérique d'un Système Thermique par
Réseaux de Neurones Embarqués avec Architecture
IoT**

Réalisé par :

Hiba Ben Mansour

Encadrant académique : M. Sami Gazzah

Encadrant professionnel : M. Faouzi Bouani

Société d'accueil



Année universitaire : 2025/2026

INSTITUT SUPÉRIEUR D'INFORMATIQUE
ET DES TECHNOLOGIES DE COMMUNICATION DE SOUSSE



المعهد العالي للإعلامية وتكنولوجيايات الاتصال بسوسة

Rapport de stage de fin d'études

Présenté en vue de l'obtention du diplôme de Licence en Ingénierie des
Systèmes Informatiques (Computer Engineering)

Spécialité : Systèmes Embarqués et Internet des Objets

**Commande Numérique d'un Système Thermique par
Réseaux de Neurones Embarqués avec Architecture IoT**

Réalisé par :

Hiba Ben Mansour

Encadrant académique : M. Sami Gazzah

Signature :

Date : 30/05/2026

Encadrant professionnel : M. Faouzi Bouani

Signature :

Date : 30/05/2026

Société d'accueil



Année universitaire : 2025/2026

Dédicaces



*À ceux qui ont accompagné ce parcours
par leur amour, leur confiance et leur soutien sincère.*

À mes chers parents,

À vous qui avez toujours cru en moi et qui m'avez soutenue à chaque étape de mon parcours. Votre amour, vos sacrifices, vos prières et vos encouragements ont été une source précieuse de force et de motivation. Je vous dédie ce travail avec tout mon amour, mon respect et ma profonde gratitude.

À mes frères et sœurs,

Pour votre affection, votre présence et vos encouragements constants. Votre soutien m'a aidée à avancer avec confiance, à surmonter les moments difficiles et à poursuivre ce chemin avec détermination.

À toute ma famille,

Pour votre bienveillance, vos conseils et l'attention que vous m'avez toujours portée. Votre présence et vos pensées positives ont accompagné ce parcours et lui ont donné une valeur particulière.

À toutes les personnes qui m'ont soutenue,

À celles et ceux qui m'ont aidée, encouragée ou accompagnée, de près ou de loin, dans la réalisation de ce travail. Chaque geste d'aide, chaque parole bienveillante et chaque marque de confiance ont compté. Je vous adresse mes sincères remerciements et ma profonde reconnaissance.

Remerciements



Je tiens à remercier sincèrement toutes les personnes qui ont contribué au bon déroulement de mon stage, ainsi que celles qui m'ont soutenue durant la réalisation de ce projet et la rédaction de ce rapport.

En premier lieu, je souhaite exprimer ma profonde gratitude à mon encadrant académique, **M. Sami Gazzah**. Votre disponibilité, vos conseils avisés, vos remarques constructives et votre accompagnement attentif m'ont été d'une grande aide tout au long de ce travail. Votre soutien a largement contribué à l'amélioration de ce projet et à son aboutissement dans les meilleures conditions.

Je remercie également mon encadrant professionnel, **M. Faouzi Bouani**, pour son encadrement, son soutien et ses orientations pertinentes. Vos conseils, votre rigueur et l'intérêt que vous avez porté à l'avancement de ce projet ont joué un rôle important dans le développement de ce travail et dans l'enrichissement de mon expérience.

J'adresse aussi mes sincères remerciements à l'ensemble de mes enseignants de l'**Institut Supérieur d'Informatique et des Technologies de Communication de Sousse**, pour la qualité de leur formation, leurs efforts et les connaissances transmises tout au long de mon parcours universitaire. Leur enseignement a constitué une base essentielle dans la réalisation de ce projet.

Enfin, je tiens à remercier toutes les personnes qui, de près ou de loin, m'ont encouragée, soutenue ou aidée dans l'accomplissement de ce travail. Qu'elles trouvent ici l'expression de ma sincère reconnaissance.

Table des matières

Liste des figures	vii
Liste des tableaux	x
Liste des abréviations	xii
Introduction générale	1
1 Cadre Général du Projet.....	2
1.1 Introduction.....	3
1.2 Organisme d'accueil	3
1.3 Contexte du projet	3
1.4 Problématique.....	4
1.5 Étude de l'existant	4
1.6 Critiques de l'existant	5
1.7 Solution proposée	5
1.7.1 Objectifs	5
1.8 Méthodologie de travail	6
1.9 Planification du projet	7
1.10 Conclusion	8
2 Analyse et Spécification des Besoins.....	9
2.1 Introduction.....	10
2.2 Description générale du système	10
2.3 Architecture fonctionnelle globale	10
2.4 Besoins fonctionnels	11
2.4.1 Contrôle thermique	11
2.4.2 Traitement intelligent	12
2.4.3 Gestion des modes de fonctionnement	12
2.4.4 Commande à distance.....	12
2.4.5 Supervision et communication IoT.....	13
2.4.6 Gestion des données	13
2.4.7 Sécurité de fonctionnement.....	13
2.5 Besoins non fonctionnels	14
2.6 Identification des acteurs.....	15
2.7 Contraintes du système	16
2.8 Conclusion	16

3	Modélisation et Conception du Système	18
3.1	Introduction	20
3.2	Diagrammes et modélisation UML	20
3.2.1	Diagramme de cas d'utilisation global	20
3.2.2	Raffinement des cas d'utilisation	22
3.2.3	Diagrammes de séquence	28
3.2.4	Diagramme d'activité de la boucle de contrôle embarquée	32
3.2.5	Diagramme de déploiement de l'architecture IoT	34
3.3	Modélisation du système thermique	35
3.3.1	Description du système physique	35
3.3.2	Schéma du système thermique	35
3.3.3	Modélisation mathématique	36
3.3.4	Identification des paramètres	38
3.3.5	Simulation du système	39
3.3.6	Principe de validation du modèle	39
3.4	Génération du dataset	39
3.4.1	Utilisation de GEKKO et du contrôleur MPC	39
3.4.2	Stratégie de génération	40
3.4.3	Structure des données	40
3.5	Clarification méthodologique sur les deux rôles du MPC	41
3.5.1	MPC de référence utilisé pour la génération du dataset	41
3.5.2	MPC simplifié utilisé pour l'évaluation expérimentale	42
3.5.3	Justification du choix du NN-MPC	42
3.6	Conception du modèle de réseau de neurones	43
3.6.1	Objectif du modèle	43
3.6.2	Choix du modèle	44
3.6.3	Variables d'entrée et sortie	44
3.6.4	Architecture du réseau	45
3.6.5	Justification de l'architecture	45
3.6.6	Avantages de l'approche NN-MPC	46
3.7	Entraînement du modèle	46
3.7.1	Prétraitement des données	46
3.7.2	Paramètres d'apprentissage	46
3.7.3	Suivi de l'apprentissage	47
3.7.4	Critères d'évaluation du modèle	47
3.7.5	Principe de comparaison entre la commande prédite et la commande MPC	47
3.8	Conversion du modèle	48
3.8.1	Conversion en TensorFlow Lite	48

3.9	Conclusion	48
4	Implémentation et Validation du Système	50
4.1	Introduction	52
4.2	Architecture matérielle du système	52
4.2.1	Unité de traitement embarquée : Raspberry Pi	52
4.2.2	Capteur de température et d'humidité DHT22	53
4.2.3	Relais statique SSR	55
4.2.4	Lampe incandescente	56
4.2.5	Enceinte thermique	57
4.2.6	Montage expérimental	57
4.3	Architecture globale et chaîne fonctionnelle	58
4.3.1	Chaîne fonctionnelle du système	59
4.3.2	Synthèse des choix techniques	59
4.4	Implémentation logicielle embarquée	59
4.4.1	Environnement de développement	60
4.4.2	Déploiement du modèle NN-MPC sur Raspberry Pi	60
4.4.3	Boucle de contrôle embarquée	60
4.4.4	Génération du signal de commande	61
4.5	Communication MQTT et structure des messages	62
4.5.1	Topics MQTT utilisés	62
4.5.2	Structure du message de mesure	63
4.5.3	Structure du message de commande	63
4.6	Supervision et pilotage avec Node-RED	64
4.6.1	Logique interne du flow Node-RED	64
4.6.2	Séparation entre mesures et commandes	67
4.6.3	Développement du dashboard Node-RED	67
4.6.4	Gestion du service système	68
4.6.5	Mode programmé et ordonnanceur temporel	68
4.7	Stockage et analyse historique des données	69
4.7.1	Analyse historique avec Grafana	70
4.8	Robustesse et gestion des défauts	71
4.9	Précautions de sécurité	72
4.10	Validation fonctionnelle du système	72
4.10.1	Scénario de test	72
4.10.2	Vérification de la communication MQTT	73
4.10.3	Vérification du dashboard Node-RED	73
4.10.4	Vérification du mode programmé	73

4.10.5	Vérification de l'intégration Grafana	73
4.11	Conclusion	73
5	Résultats et Discussion	75
5.1	Introduction.....	76
5.2	Génération du dataset d'apprentissage	76
5.3	Apprentissage du modèle NN-MPC	78
5.3.1	Architecture du réseau de neurones	78
5.3.2	Paramètres d'apprentissage	78
5.3.3	Courbes d'apprentissage	79
5.3.4	Prédiction de la commande MPC.....	80
5.3.5	Taille du modèle et performance d'inférence.....	81
5.4	Validation du modèle thermique	81
5.5	Comparaison expérimentale des contrôleurs	83
5.5.1	Comparaison des réponses en température	83
5.5.2	Comparaison des commandes appliquées	84
5.5.3	Synthèse quantitative des performances	84
5.6	Analyse de robustesse du contrôleur NN-MPC	85
5.6.1	Suivi de consignes multiples	85
5.6.2	Rejet de perturbations	86
5.7	Analyse énergétique	87
5.8	Résultats fonctionnels de la supervision IoT	89
5.8.1	Dashboard de supervision temps réel	89
5.8.2	Validation des modes de fonctionnement	90
5.8.3	Gestion du service embarqué	92
5.8.4	Analyse historique intégrée avec Grafana.....	93
5.8.5	Diagnostic automatique et résumé expérimental	93
5.8.6	Synthèse de la validation fonctionnelle	94
5.9	Discussion générale.....	94
5.10	Conclusion	95
	Conclusion générale	97
	Bibliographie et références	99

Liste des figures

1.1	Modèle du Cycle en V	7
1.2	Planification de projet	8
2.1	Architecture fonctionnelle globale du système de régulation thermique intelligent.	11
3.1	Diagramme de cas d'utilisation global du système de régulation thermique intelligent	21
3.2	Raffinement du cas d'utilisation relatif à la supervision du système en temps réel.	23
3.3	Raffinement du cas d'utilisation relatif à la configuration de la régulation thermique.	26
3.4	Raffinement du cas d'utilisation relatif à la consultation de l'historique et à l'analyse des performances.	28
3.5	Diagramme de séquence du pilotage du système depuis le dashboard Node-RED.	30
3.6	Diagramme de séquence de la régulation embarquée par le contrôleur NN-MPC.	32
3.7	Diagramme d'activité de la boucle de contrôle embarquée exécutée sur la Raspberry Pi.	33
3.8	Diagramme de déploiement de l'architecture matérielle et logicielle du système de régulation thermique connecté.	34
3.9	Schéma simplifié du système thermique en boucle fermée	35
3.10	Représentation du bilan thermique du système mettant en évidence l'apport de chaleur, les pertes vers l'environnement et l'énergie stockée.....	36
4.1	Carte Raspberry Pi 4 utilisée comme unité de traitement embarquée.	53
4.2	Capteur DHT22 utilisé pour la mesure de la température et de l'humidité.	54
4.3	Relais statique utilisé pour la commande de la lampe chauffante.....	56
4.4	Lampe incandescente utilisée comme élément chauffant.	57
4.5	Montage expérimental réel du système de régulation thermique.....	58

4.6	Architecture globale du système de supervision et de contrôle thermique. .	58
4.7	Conversion de la commande en pourcentage en signal Slow PWM.	61
4.8	Architecture MQTT reliant la Raspberry Pi, le broker Mosquitto et les outils de supervision.....	62
4.9	Flow Node-RED développé pour la supervision, le pilotage et la gestion du service.	65
4.10	Dashboard Node-RED principal développé pour la supervision temps réel et le pilotage du système thermique.	67
4.11	Module de gestion du service <code>thermal-control.service</code> intégré dans le dashboard Node-RED.	68
4.12	Interface du mode programmé permettant de définir l'heure de début, la durée et la consigne.	69
4.13	Dashboard Grafana intégré dans Node-RED via iframe pour l'analyse historique des données expérimentales.	70
4.14	Exemple d'état critique détecté par le module d'alertes et d'état du dashboard.....	71
5.1	Dataset généré : trajectoire de consigne, réponse thermique simulée et commande MPC optimale sur 10 000 pas de temps.	77
5.2	Évolution des pertes d'apprentissage et de validation du modèle NN-MPC.	79
5.3	Comparaison entre la commande MPC de référence et la commande prédite par le modèle NN-MPC.	80
5.4	Validation du modèle thermique : comparaison entre la réponse mesurée et la réponse simulée pour $u_0 = 60\%$	82
5.5	Comparaison des réponses en température des contrôleurs PID, MPC et NN-MPC pour une consigne de 30°C	83
5.6	Comparaison des commandes PWM appliquées par les contrôleurs PID, MPC et NN-MPC.	84
5.7	Réponse du contrôleur NN-MPC pour cinq consignes successives : 25°C , 28°C , 30°C , 33°C et 35°C	85
5.8	Réponse du système face à une perturbation externe pour une consigne de 30°C	86
5.9	Comparaison de l'indice énergétique utile jusqu'au temps de stabilisation.	89
5.10	Dashboard Node-RED principal développé pour la supervision temps réel du système thermique NN-MPC.	90

5.11	Mode AUTO : modification de la consigne avec commande PWM calculée automatiquement par le contrôleur NN-MPC.	91
5.12	Mode MANUAL : imposition directe de la commande PWM pour les essais et les diagnostics.	91
5.13	Mode PROGRAMME : planification temporelle des consignes appliquées automatiquement par Node-RED.	92
5.14	Module de gestion du service <code>thermal-control.service</code> intégré dans le dashboard.	92
5.15	Page d'analyse Grafana intégrée dans Node-RED avec filtres temporels et visualisation des données expérimentales.	93
5.16	Modules de diagnostic automatique et de résumé expérimental intégrés dans la supervision IoT.	94

Liste des tableaux

2.1	Synthèse des besoins fonctionnels du système.....	14
2.2	Besoins non fonctionnels du système.....	15
2.3	Acteurs du système	16
3.1	Description du raffinement relatif à la supervision temps réel.....	23
3.2	Description du raffinement relatif à la configuration de la régulation thermique	25
3.3	Description du raffinement relatif à l'analyse historique et aux performances	27
3.4	Caractéristiques du dataset généré.....	41
3.5	Différence entre les deux utilisations du MPC dans ce projet	43
4.1	Caractéristiques principales de la Raspberry Pi 4.....	53
4.2	Configuration des broches GPIO utilisées.....	53
4.3	Caractéristiques principales du capteur DHT22.....	54
4.4	Connexion du capteur DHT22	55
4.5	Caractéristiques principales du relais SSR	56
4.6	Connexion du relais SSR.....	56
4.7	Caractéristiques principales de la lampe incandescente	57
4.8	Synthèse des choix techniques du système	59
4.9	Topics MQTT utilisés dans le système	62
4.10	Structure du message de mesure publié par la Raspberry Pi.....	63
4.11	Structure du message de commande envoyé par Node-RED	64
4.12	Organisation logique du flow Node-RED.....	66
4.13	Commandes système intégrées dans le dashboard Node-RED.....	68
4.14	Grandeurs enregistrées pendant les essais	70
4.15	Mécanismes de robustesse intégrés	72
5.1	Caractéristiques du dataset d'apprentissage.....	77
5.2	Architecture du réseau de neurones NN-MPC	78
5.3	Hyperparamètres d'apprentissage du modèle NN-MPC	79

5.4	Performances de prédiction du modèle NN-MPC	80
5.5	Taille du modèle et performances d'inférence	81
5.6	Indicateurs quantitatifs de validation du modèle thermique.....	82
5.7	Comparaison expérimentale des performances pour une consigne de 30 °C.	84
5.8	Performances du NN-MPC pour plusieurs consignes	86
5.9	Performances de rejet de perturbation pour une consigne de 30 °C	87

Liste des abréviations

Abréviation	Signification
CSV	Comma-Separated Values, format d'enregistrement des données.
DHT22	Capteur numérique de température et d'humidité.
GEKKO	Bibliothèque Python utilisée pour la simulation et l'optimisation.
GPIO	General Purpose Input/Output, broches d'entrée/sortie de la Raspberry Pi.
HMI	Human-Machine Interface, interface homme-machine.
IoT	Internet of Things, Internet des objets.
JSON	JavaScript Object Notation, format léger d'échange de données.
KPI	Key Performance Indicator, indicateur clé de performance.
MAE	Mean Absolute Error, erreur absolue moyenne.
MLP	Multi-Layer Perceptron, perceptron multicouche.
MPC	Model Predictive Control, commande prédictive basée sur modèle.
MQTT	Message Queuing Telemetry Transport, protocole de communication IoT.
MSE	Mean Squared Error, erreur quadratique moyenne.
NN	Neural Network, réseau de neurones.
NN-MPC	Neural Network Model Predictive Control, commande prédictive approximée par réseau de neurones.

Abréviation	Signification
PID	Proportional Integral Derivative, contrôleur proportionnel intégral dérivé.
PWM	Pulse Width Modulation, modulation de largeur d'impulsion.
R²	Coefficient de détermination.
ReLU	Rectified Linear Unit, fonction d'activation.
SSR	Solid State Relay, relais statique.
TFLite	TensorFlow Lite, version légère de TensorFlow pour systèmes embarqués.
UML	Unified Modeling Language, langage de modélisation unifié.

Résumé

Ce projet de fin d'études présente la conception et la validation d'un système de régulation thermique intelligent basé sur un contrôleur NN-MPC embarqué et une architecture IoT. Le système est réalisé sur une maquette thermique chauffée par une lampe incandescente, mesurée par un capteur DHT22 et pilotée par une Raspberry Pi via un relais statique SSR.

L'approche proposée utilise un contrôleur MPC de référence pour générer un dataset d'apprentissage, puis un réseau de neurones est entraîné afin d'approximer la loi de commande MPC. Le modèle obtenu est converti en TensorFlow Lite afin d'assurer une exécution rapide et légère sur Raspberry Pi.

Une supervision IoT basée sur MQTT, Node-RED, InfluxDB et Grafana permet le suivi temps réel, le pilotage des modes de fonctionnement, le stockage des données et l'analyse historique. Les résultats montrent que le contrôleur NN-MPC offre un bon compromis entre rapidité, précision, faible dépassement, robustesse et coût de calcul réduit.

Mots-clés : Régulation thermique, NN-MPC, MPC, réseaux de neurones, Raspberry Pi, TensorFlow Lite, MQTT, Node-RED, InfluxDB, Grafana, IoT.

Abstract

This final-year project presents the design and validation of an intelligent thermal regulation system based on an embedded NN-MPC controller and an IoT architecture. The system is implemented on a thermal prototype heated by an incandescent lamp, measured by a DHT22 sensor and controlled by a Raspberry Pi through a solid-state relay.

The proposed approach uses a reference MPC controller to generate a training dataset, then a neural network is trained to approximate the MPC control law. The resulting model is converted to TensorFlow Lite to ensure fast and lightweight execution on the Raspberry Pi.

An IoT supervision layer based on MQTT, Node-RED, InfluxDB and Grafana enables real-time monitoring, operating mode control, data storage and historical analysis. The results show that the NN-MPC controller provides a good compromise between response speed, accuracy, low overshoot, robustness and reduced computational cost.

Keywords : Thermal regulation, NN-MPC, MPC, neural networks, Raspberry Pi, TensorFlow Lite, MQTT, Node-RED, InfluxDB, Grafana, IoT.

Introduction générale

La régulation thermique occupe une place importante dans de nombreux domaines industriels, énergétiques et domestiques. Les systèmes thermiques se caractérisent généralement par une dynamique lente, une forte inertie et une sensibilité aux perturbations, ce qui rend leur commande exigeante lorsqu'un suivi précis de la température est recherché.

Dans ce contexte, l'évolution des techniques de commande et de l'intelligence artificielle ouvre la voie à des solutions plus performantes, capables d'associer qualité de régulation, rapidité d'exécution et intégration sur des plateformes embarquées. L'enjeu ne consiste plus uniquement à contrôler un procédé thermique, mais également à concevoir une solution complète, exploitable en temps réel et adaptée à un environnement connecté.

Ce projet de fin d'études s'inscrit dans cette perspective. Il porte sur la conception et la validation d'un système de régulation thermique intelligent basé sur un contrôleur NN-MPC embarqué, associé à une architecture IoT de supervision. Le travail réalisé couvre l'ensemble de la chaîne de développement, depuis la modélisation du procédé jusqu'à l'implémentation pratique, l'analyse des performances et la visualisation des données.

Le rapport est organisé en cinq chapitres. Le premier chapitre présente le cadre général du projet, la problématique étudiée et la solution proposée. Le deuxième chapitre est consacré à l'analyse et à la spécification des besoins. Le troisième chapitre décrit la modélisation du système et la conception du contrôleur NN-MPC. Le quatrième chapitre expose l'implémentation matérielle et logicielle ainsi que l'architecture de supervision IoT. Enfin, le cinquième chapitre présente les résultats expérimentaux et la discussion des performances obtenues.

Cadre Général du Projet

Sommaire

1.1	Introduction.....	3
1.2	Organisme d'accueil.....	3
1.3	Contexte du projet.....	3
1.4	Problématique.....	4
1.5	Étude de l'existant.....	4
1.6	Critiques de l'existant.....	5
1.7	Solution proposée.....	5
1.7.1	Objectifs.....	5
1.8	Méthodologie de travail.....	6
1.9	Planification du projet.....	7
1.10	Conclusion.....	8

1.1 Introduction

Ce premier chapitre a pour objectif de définir le cadre général de notre projet de fin d'études. Il dresse un état des lieux de la régulation thermique dans le monde industriel, met en évidence les limites des approches classiques face aux exigences modernes, et justifie l'intérêt des réseaux de neurones embarqués. Nous y présenterons également l'organisme d'accueil, la problématique abordée, la solution proposée avec ses objectifs détaillés, ainsi que la méthodologie et la planification adoptées pour mener à bien ce travail.

1.2 Organisme d'accueil

Dans le cadre de l'obtention de mon diplôme de Licence en Ingénierie des Systèmes Informatiques (Computer Engineering), spécialité Systèmes Embarqués et Internet des Objets, à l'Institut Supérieur d'Informatique et des Technologies de Communication (ISIT-Com) de Hammam Sousse, j'ai effectué mon stage de fin d'études au sein du Laboratoire d'Analyse, Conception et Commande des Systèmes (LACS) de l'ENIT. Durant ce stage, j'ai développé un projet intitulé « **Commande Numérique d'un système Thermique par Réseau de Neurones Embarqués** », une solution innovante visant à optimiser la régulation thermique via l'intelligence artificielle.

L'objectif principal de ce projet est de concevoir un régulateur neuro-prédictif (NN-MPC) capable d'offrir la précision d'une commande experte tout en étant déployé sur une plateforme aux ressources limitées comme la Raspberry Pi. Ce système garantit une régulation en temps réel performante, tout en intégrant une couche de supervision IoT pour le monitoring à distance des données. Cette approche permet ainsi de concilier la réactivité des algorithmes classiques avec l'intelligence des modèles prédictifs, assurant une gestion thermique à la fois stable, précise et connectée.

1.3 Contexte du projet

De nos jours, le contrôle thermique est devenu une fonction critique au cœur des systèmes technologiques modernes. Ses champs d'application sont très vastes et touchent des secteurs clés, tels que :

- **La domotique** : Systèmes de climatisation et de chauffage intelligents pour le confort des bâtiments (Smart Home).
- **L'industrie** : Régulation stricte des processus de fabrication chimiques ou mécaniques.
- **Les infrastructures IT** : Gestion thermique et refroidissement des serveurs dans les Datacenters.

Dans tous ces secteurs, un constat s'impose : l'efficacité énergétique, la stabilité et

la précision des systèmes reposent intégralement sur la qualité et l'intelligence de leur algorithme de régulation. La littérature récente confirme l'intérêt croissant des approches prédictives et des méthodes fondées sur l'apprentissage pour améliorer la commande des systèmes thermiques à forte inertie, notamment dans les applications énergétiques et de régulation de température [9, 10, 11]. C'est précisément dans cette optique d'optimisation algorithmique que s'inscrit le présent projet.

1.4 Problématique

Le contrôle des systèmes thermiques est complexe en raison de leur forte inertie et de leur sensibilité aux perturbations externes. Pour obtenir une régulation parfaite, il est nécessaire d'utiliser des commandes intelligentes capables d'anticiper l'évolution de la température.

Cependant, le déploiement de cette intelligence sur des cartes embarquées (Edge Computing) pose un problème technique majeur : les algorithmes performants exigent une grande puissance de calcul, ce qui est souvent incompatible avec les ressources limitées d'une carte standard (comme une Raspberry Pi).

Le défi principal de ce projet se résume donc à la question fondamentale suivante :

« Comment intégrer la précision et l'intelligence d'une commande experte sur une carte embarquée aux ressources limitées, pour une régulation thermique en temps réel ? »

1.5 Étude de l'existant

Dans l'industrie actuelle, le contrôle thermique s'articule généralement autour de deux grandes familles de solutions :

- **Le contrôle classique (Régulateurs PID)** : C'est la méthode la plus répandue en raison de sa simplicité mathématique et de sa facilité d'implémentation directe sur des automates ou des microcontrôleurs basiques.
- **Le contrôle avancé (Commande Prédictive – MPC)** : Cette approche utilise un modèle mathématique du système pour anticiper le comportement futur du procédé et calculer une commande optimale en respectant des contraintes physiques. Le MPC est une stratégie de commande avancée largement étudiée dans la littérature et utilisée dans plusieurs applications industrielles grâce à sa capacité à gérer les contraintes et à améliorer les performances de régulation [1, 2].

1.6 Critiques de l'existant

Ces deux approches, bien qu'éprouvées, présentent des limites opposées face aux exigences de l'embarqué moderne :

- **Limites du PID** : Il réagit uniquement à l'erreur présente et passée. Face à l'inertie thermique, son manque d'anticipation entraîne souvent des dépassements de température (overshoots) importants. De plus, il offre une robustesse insuffisante face aux perturbations externes.
- **Limites du MPC** : Bien qu'il offre des performances optimales sans dépassement, le contrôleur MPC nécessite la résolution d'équations d'optimisation complexes à chaque pas de temps. Cette lourdeur de calcul exige un temps de traitement élevé, ce qui rend son déploiement en temps réel sur des cartes embarquées standard (comme une Raspberry Pi) presque impossible sans provoquer de latence critique.

1.7 Solution proposée

Afin de s'affranchir des limitations précitées, nous proposons la conception d'un régulateur neuro-prédicatif embarqué (NN-MPC), soutenu par une infrastructure IoT. Notre approche innovante s'articule autour de deux axes complémentaires :

⇒ **L'intelligence embarquée (Edge AI) :**

La démarche consiste à simuler un comportement de contrôle expert (MPC) dans l'environnement GEKKO pour générer des données d'apprentissage [22]. Ces données permettent d'entraîner un réseau de neurones à approximer cette stratégie optimale. Cette logique s'inscrit dans les travaux consacrés à l'approximation neuronale de lois de commande prédictive, où le réseau apprend à reproduire le comportement d'un contrôleur MPC afin de réduire le coût de calcul en ligne [4, 7]. Une fois converti et allégé via TensorFlow Lite, ce modèle est déployé localement sur une carte Raspberry Pi [21, 14]. Il pilote ainsi l'actionneur thermique en temps réel, alliant *la réactivité d'un PID à la capacité d'anticipation d'un MPC*.

⇒ **La supervision connectée (IoT) :**

En parallèle de la boucle de contrôle locale, une architecture de communication basée sur le protocole MQTT et la base de données InfluxDB est intégrée [23, 26]. Elle assure la remontée fluide des mesures vers un Dashboard (Grafana/Node-RED), permettant un monitoring et une gestion à distance du système [27, 25].

1.7.1 Objectifs

Les objectifs principaux fixés par le cahier des charges s'articulent autour de quatre axes majeurs :

- **Conception algorithmique** : Développer un régulateur neuro-prédictif (NN-MPC) garantissant un maintien de la température avec une erreur statique limitée à $\pm 0.1^\circ\text{C}$ et un dépassement thermique minimal, inférieur à 1%.
- **Déploiement embarqué (Edge AI)** : Optimiser le modèle d'intelligence artificielle pour un fonctionnement local sur carte Raspberry Pi, avec une contrainte de latence d'inférence strictement inférieure à 100 ms.
- **Supervision IoT** : Intégrer une architecture de communication asynchrone permettant la surveillance à distance, sans perturber l'exécution en temps réel de la boucle de contrôle critique.
- **Validation expérimentale** : Évaluer les performances matérielles du système sur un banc d'essai physique et mener une analyse comparative face aux approches PID et MPC classique.

1.8 Méthodologie de travail

Pour mener à bien ce projet, nous avons adopté la méthodologie du **Cycle en V** [12]. Ce modèle garantit une démarche rigoureuse en alignant chaque phase de conception avec sa phase de validation correspondante. Notre travail s'articule autour de trois grandes étapes :

1. La phase de Conception (Branche descendante) :

- **Spécification** : Définition des exigences fonctionnelles du système de régulation thermique.
- **Architecture** : Choix de l'infrastructure matérielle (Raspberry Pi, capteurs) et logicielle (IoT).
- **Modélisation** : Développement mathématique du contrôleur expert (MPC) de référence.

2. La phase de Réalisation (Le creux du V) :

- **Montage et Collecte** : Assemblage du banc d'essai et génération du dataset via le MPC.
- **Apprentissage IA** : Conception et entraînement du réseau de neurones (NN-MPC) sous TensorFlow.

3. La phase de Validation (Branche ascendante) :

- **Intégration** : Conversion et déploiement du modèle léger (TFLite) sur la carte embarquée.
- **Tests système** : Vérification de la boucle de contrôle en temps réel et de la supervision IoT.
- **Recette finale** : Évaluation expérimentale globale et comparaison des performances avec le PID.

La figure 1.1 illustre cette progression, de l'expression du besoin jusqu'à la validation

finale du système.

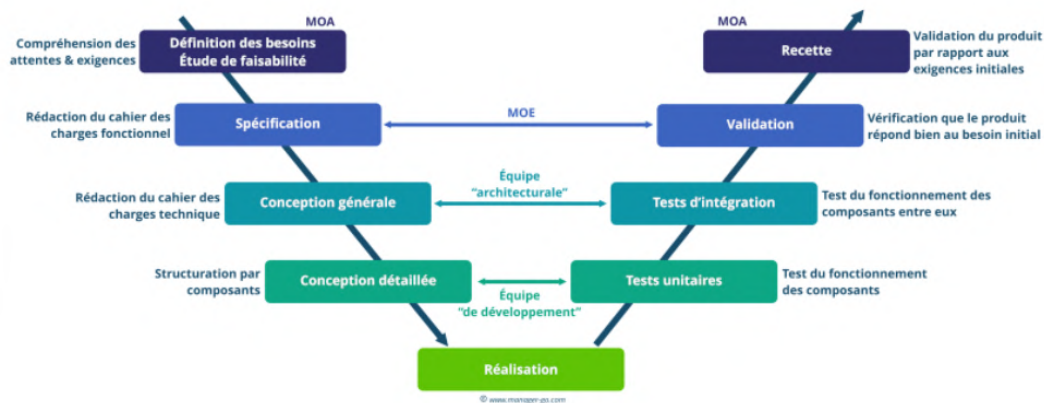


FIGURE 1.1 – Modèle du Cycle en V

1.9 Planification du projet

La planification des tâches est une étape essentielle pour garantir le bon déroulement du projet. Elle consiste à définir les actions à réaliser et à leur fixer des délais précis. Cette démarche offre une vision globale claire et permet d'anticiper les risques éventuels.

Voici les tâches que j'ai suivies dans mon projet :

- **Tâche 1** : Analyse des besoins, étude de l'existant et choix des composants matériels.
- **Tâche 2** : Construction du banc d'essai, modélisation experte et collecte des données.
- **Tâche 3** : Conception, entraînement et optimisation du réseau de neurones.
- **Tâche 4** : Implémentation embarquée sur Raspberry Pi et développement de la couche IoT.
- **Tâche 5** : Rédaction continue du rapport, élaboration des tests et validation expérimentale finale.

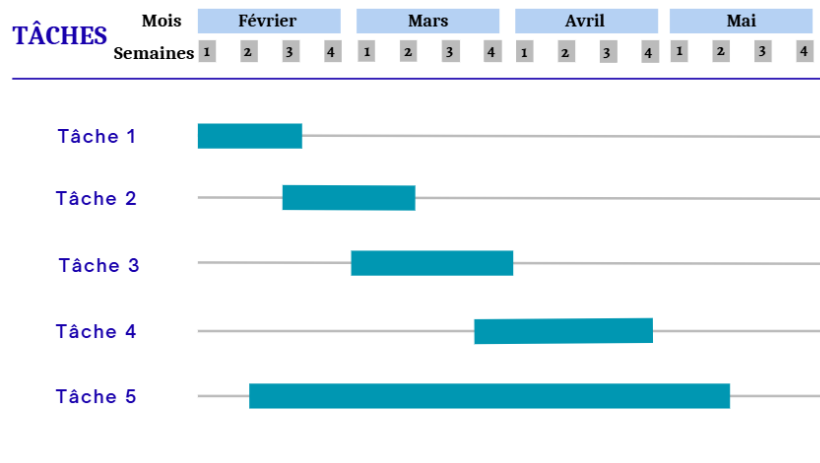


FIGURE 1.2 – Planification de projet

1.10 Conclusion

Ce premier chapitre a permis de poser le cadre général de notre projet. Nous avons mis en évidence les limites des régulateurs thermiques classiques, ce qui justifie notre choix d'une commande intelligente (NN-MPC) couplée à l'IoT.

Maintenant que la solution proposée, les objectifs et le planning sont définis, nous pouvons passer à l'étape suivante. Le prochain chapitre sera donc consacré à l'analyse détaillée des besoins et aux spécifications techniques de notre système.

Analyse et Spécification des Besoins

Sommaire

2.1	Introduction.....	10
2.2	Description générale du système	10
2.3	Architecture fonctionnelle globale	10
2.4	Besoins fonctionnels	11
2.4.1	Contrôle thermique	11
2.4.2	Traitement intelligent	12
2.4.3	Gestion des modes de fonctionnement	12
2.4.4	Commande à distance.....	12
2.4.5	Supervision et communication IoT.....	13
2.4.6	Gestion des données	13
2.4.7	Sécurité de fonctionnement.....	13
2.5	Besoins non fonctionnels	14
2.6	Identification des acteurs.....	15
2.7	Contraintes du système	16
2.8	Conclusion	16

2.1 Introduction

Ce chapitre est consacré à l'analyse et à la spécification des besoins du système de régulation thermique intelligent.

L'objectif est de traduire la problématique présentée dans le chapitre précédent en exigences fonctionnelles et non fonctionnelles claires. Cette étape permet de définir les fonctionnalités attendues, les contraintes de performance, les acteurs impliqués ainsi que les limites du système à concevoir.

Dans ce cadre, nous présentons d'abord une description générale du système, puis son architecture fonctionnelle globale. Ensuite, nous détaillons les besoins fonctionnels et non fonctionnels, avant d'identifier les acteurs principaux qui interagissent avec la solution proposée.

2.2 Description générale du système

Le système proposé est une solution de régulation thermique intelligente basée sur une architecture embarquée et connectée. Il a pour objectif de maintenir la température d'une enceinte thermique autour d'une consigne définie par l'utilisateur.

Le système repose sur une boucle de régulation fermée. La température et l'humidité sont mesurées à l'intérieur de l'enceinte thermique, puis transmises à une unité de traitement embarquée. Cette unité exécute un modèle de contrôle intelligent de type NN-MPC afin de calculer la commande thermique appropriée.

La commande calculée est ensuite convertie en signal de type Slow PWM et appliquée à l'actionneur thermique. En parallèle, une couche IoT assure la supervision à distance, l'envoi des consignes, le stockage des données et la visualisation des mesures.

Le système doit donc assurer deux fonctions principales :

- une fonction de **régulation locale**, responsable de l'acquisition, du calcul de commande et de l'actionnement thermique ;
- une fonction de **supervision IoT**, responsable du suivi à distance, de la configuration, du stockage et de l'analyse des données.

Cette séparation permet de garantir que la régulation thermique reste fonctionnelle localement, tout en offrant une interface de supervision et d'analyse à distance.

2.3 Architecture fonctionnelle globale

L'architecture fonctionnelle du système est organisée autour de deux niveaux complémentaires.

Le premier niveau correspond à la **boucle de contrôle locale**. Il assure la mesure de la température, l'exécution du contrôleur NN-MPC, la génération de la commande Slow

PWM et l'actionnement thermique.

Le second niveau correspond à la **couche IoT**. Il permet de transmettre les données du système, d'envoyer les consignes, de stocker les mesures et de visualiser les historiques à travers une interface de supervision[23, 25, 26, 27].

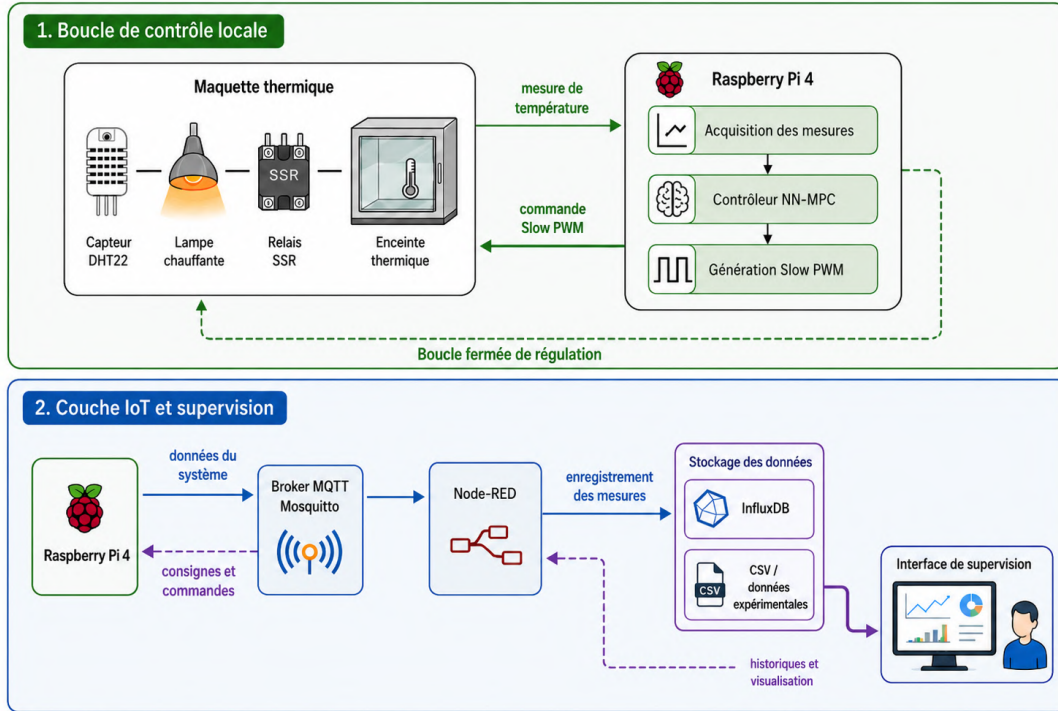


FIGURE 2.1 – Architecture fonctionnelle globale du système de régulation thermique intelligent.

La Fig. 2.1 présente une vue fonctionnelle globale du système. Elle met en évidence les deux niveaux principaux de la solution : la boucle de régulation embarquée et la couche de supervision IoT. Les détails matériels, logiciels et les choix d'implémentation seront présentés dans les chapitres suivants.

2.4 Besoins fonctionnels

Les besoins fonctionnels décrivent les services que le système doit fournir. Ils sont définis à partir des objectifs du projet et de l'architecture globale retenue.

2.4.1 Contrôle thermique

Le système doit assurer la régulation de la température à l'intérieur de l'enceinte thermique. Pour cela, il doit être capable de mesurer la température, de la comparer à une consigne et de calculer une commande adaptée.

Les fonctions attendues sont :

- mesurer la température et l’humidité en temps réel ;
- comparer la température mesurée à la consigne ;
- calculer une commande thermique adaptée ;
- générer un signal de commande de type Slow PWM ;
- piloter l’actionneur thermique ;
- maintenir la température autour de la consigne.

2.4.2 Traitement intelligent

Le système doit intégrer un modèle de contrôle intelligent capable de prédire la commande thermique à appliquer. Ce modèle doit exploiter les variables représentatives de l’état thermique du système.

Les fonctions attendues sont :

- utiliser un modèle de réseau de neurones de type NN-MPC ;
- prendre en entrée la température actuelle, la consigne, l’humidité et l’historique thermique ;
- fournir une commande de chauffage comprise entre 0% et 100% ;
- permettre une inférence rapide compatible avec une exécution embarquée.

2.4.3 Gestion des modes de fonctionnement

Le système doit proposer plusieurs modes de fonctionnement afin de s’adapter aux différents scénarios d’utilisation.

Les modes attendus sont :

- **mode automatique** : la commande est calculée par le modèle NN-MPC ;
- **mode manuel** : l’utilisateur impose directement la puissance de chauffage ;
- **mode programmé** : la consigne est appliquée automatiquement selon une planification temporelle.

2.4.4 Commande à distance

Le système doit permettre à l’utilisateur d’agir sur la régulation à partir de l’interface de supervision.

Les fonctions attendues sont :

- modifier la consigne de température à distance ;
- changer le mode de fonctionnement ;
- envoyer une commande manuelle en pourcentage ;
- activer ou désactiver une planification temporelle.

2.4.5 Supervision et communication IoT

Le système doit assurer la communication entre la partie embarquée et l'interface de supervision. Cette communication permet le suivi en temps réel et l'exploitation des données expérimentales.

Les fonctions attendues sont :

- publier les mesures du système vers la plateforme IoT ;
- afficher les données en temps réel sur un dashboard ;
- visualiser la température, l'humidité, la consigne et la commande ;
- afficher l'état de la communication et du système ;
- permettre l'analyse historique des mesures.

2.4.6 Gestion des données

Le système doit enregistrer les données expérimentales afin de permettre leur exploitation ultérieure.

Les fonctions attendues sont :

- enregistrer les mesures du système ;
- stocker les données sous forme de séries temporelles ;
- permettre l'export ou l'exploitation des données pour l'analyse ;
- visualiser les historiques à travers une interface graphique.

2.4.7 Sécurité de fonctionnement

Le système doit intégrer des mécanismes de sécurité permettant de limiter les risques liés au chauffage et aux erreurs de mesure.

Les fonctions attendues sont :

- détecter les erreurs de lecture du capteur ;
- limiter la commande de chauffage entre 0% et 100% ;
- arrêter ou réduire la commande en cas de comportement anormal ;
- signaler les états critiques à l'utilisateur.

TABLE 2.1 – Synthèse des besoins fonctionnels du système

Code	Besoin fonctionnel	Description
BF1	Acquisition des mesures	Mesurer la température et l'humidité à l'intérieur de l'enceinte thermique.
BF2	Calcul de commande	Calculer une commande thermique adaptée à partir du modèle NN-MPC.
BF3	Génération Slow PWM	Convertir la commande en pourcentage en un signal ON/OFF périodique.
BF4	Pilotage de l'actionneur	Appliquer la commande thermique à l'actionneur de chauffage.
BF5	Gestion des modes	Permettre les modes AUTO, MANUAL et PROGRAMME.
BF6	Commande à distance	Modifier la consigne, le mode et la puissance manuelle depuis l'interface.
BF7	Supervision temps réel	Afficher les mesures, la consigne, la commande et l'état du système.
BF8	Stockage des données	Enregistrer les mesures pour l'analyse historique et l'exploitation expérimentale.
BF9	Diagnostic et alertes	Signaler les défauts de capteur, les erreurs et les états critiques.

2.5 Besoins non fonctionnels

Les besoins non fonctionnels définissent les contraintes de qualité, de performance, de fiabilité et d'ergonomie que le système doit satisfaire.

TABLE 2.2 – Besoins non fonctionnels du système

Critère	Exigence	Justification
Temps réel	Période de contrôle de 1 s	Assurer une mise à jour régulière de la commande thermique.
Précision	Erreur faible autour de la consigne	Garantir une régulation stable et exploitable.
Temps de calcul	Inférence rapide sur Raspberry Pi	Ne pas perturber la boucle de contrôle embarquée.
Robustesse	Fonctionnement stable malgré les perturbations	Maintenir la température proche de la consigne.
Fiabilité	Gestion des erreurs de lecture capteur	Éviter les commandes incorrectes en cas de données invalides.
Sécurité	Limitation de la commande entre 0% et 100%	Protéger le système contre les comportements anormaux.
Communication IoT	Transmission périodique des données	Assurer la supervision et l'analyse à distance.
Ergonomie	Interface claire et simple d'utilisation	Faciliter le pilotage et le suivi expérimental.
Évolutivité	Architecture modulaire	Permettre l'ajout de nouvelles fonctionnalités sans modifier toute la solution.

Ces contraintes sont importantes car le système doit fonctionner dans un environnement embarqué, avec des ressources limitées, tout en assurant une supervision claire et fiable.

2.6 Identification des acteurs

L'identification des acteurs permet de déterminer les entités externes qui interagissent avec le système de régulation thermique intelligent. Dans notre cas, deux acteurs principaux sont retenus : utilisateur humain, et le système embarqué.

L'utilisateur humain représente l'utilisateur principal du système. Il utilise l'interface de supervision pour suivre les mesures, modifier la consigne, choisir le mode de fonctionnement, consulter les historiques et gérer le service embarqué.

Le système embarqué assure l'acquisition des mesures, l'exécution du contrôleur NN-MPC, la génération du signal Slow PWM et l'échange des données avec la couche de supervision IoT.

La plateforme IoT, composée de Node-RED, MQTT, InfluxDB et Grafana, est considé-

rée dans cette modélisation comme une composante interne de la solution, et non comme un acteur externe.

TABLE 2.3 – Acteurs du système

Acteur	Type	Rôle principal
Utilisateur humain	Humain	Supervise le système, modifie la consigne, choisit le mode de fonctionnement, consulte les historiques et gère l'état du service embarqué.
Système embarqué	Matériel / logiciel	Acquiert les mesures, exécute le contrôleur NN-MPC, applique la commande thermique et échange les données avec la supervision IoT.

2.7 Contraintes du système

En plus des besoins fonctionnels et non fonctionnels, le système doit respecter certaines contraintes liées à son environnement d'utilisation.

- **Contraintes matérielles** : le système doit fonctionner sur une carte Raspberry Pi avec des ressources limitées.
- **Contraintes thermiques** : le procédé présente une forte inertie, ce qui nécessite une commande progressive et stable.
- **Contraintes de communication** : les données doivent être transmises correctement entre la Raspberry Pi et la plateforme de supervision.
- **Contraintes de sécurité** : la commande de chauffage doit rester limitée et contrôlée.
- **Contraintes d'exploitation** : l'utilisateur doit pouvoir suivre le système sans manipuler directement le programme embarqué.

Ces contraintes orientent les choix de conception présentés dans les chapitres suivants, notamment l'utilisation d'un modèle NN-MPC léger, d'un signal Slow PWM et d'une architecture IoT modulaire.

2.8 Conclusion

Dans ce chapitre, nous avons présenté l'analyse et la spécification des besoins du système de régulation thermique intelligent.

Nous avons d'abord décrit le fonctionnement général du système et son architecture fonctionnelle. Ensuite, nous avons défini les besoins fonctionnels liés au contrôle thermique,

au traitement intelligent, aux modes de fonctionnement, à la supervision IoT, au stockage des données et à la sécurité. Les besoins non fonctionnels ont également été précisés afin de fixer les contraintes de performance, de fiabilité, de temps réel et d'ergonomie.

Les besoins identifiés dans ce chapitre serviront de base pour le choix du modèle thermique, la conception du contrôleur NN-MPC et la définition de l'architecture de supervision.

Le chapitre suivant sera consacré à la modélisation du système thermique ainsi qu'à la conception du modèle NN-MPC proposé.

Modélisation et Conception du Système

Sommaire

3.1	Introduction.....	20
3.2	Diagrammes et modélisation UML	20
3.2.1	Diagramme de cas d'utilisation global	20
3.2.2	Raffinement des cas d'utilisation.....	22
3.2.3	Diagrammes de séquence	28
3.2.4	Diagramme d'activité de la boucle de contrôle embarquée.....	32
3.2.5	Diagramme de déploiement de l'architecture IoT	34
3.3	Modélisation du système thermique	35
3.3.1	Description du système physique	35
3.3.2	Schéma du système thermique	35
3.3.3	Modélisation mathématique	36
3.3.4	Identification des paramètres	38
3.3.5	Simulation du système	39
3.3.6	Principe de validation du modèle	39
3.4	Génération du dataset.....	39
3.4.1	Utilisation de GEKKO et du contrôleur MPC	39
3.4.2	Stratégie de génération.....	40
3.4.3	Structure des données.....	40
3.5	Clarification méthodologique sur les deux rôles du MPC	41
3.5.1	MPC de référence utilisé pour la génération du dataset	41
3.5.2	MPC simplifié utilisé pour l'évaluation expérimentale.....	42
3.5.3	Justification du choix du NN-MPC	42
3.6	Conception du modèle de réseau de neurones	43
3.6.1	Objectif du modèle	43
3.6.2	Choix du modèle	44

3.6.3	Variables d'entrée et sortie	44
3.6.4	Architecture du réseau	45
3.6.5	Justification de l'architecture	45
3.6.6	Avantages de l'approche NN-MPC	46
3.7	Entraînement du modèle	46
3.7.1	Prétraitement des données	46
3.7.2	Paramètres d'apprentissage	46
3.7.3	Suivi de l'apprentissage	47
3.7.4	Critères d'évaluation du modèle	47
3.7.5	Principe de comparaison entre la commande prédite et la commande MPC	47
3.8	Conversion du modèle	48
3.8.1	Conversion en TensorFlow Lite	48
3.9	Conclusion	48

3.1 Introduction

Dans ce chapitre, nous présentons la modélisation et la conception du système de régulation thermique intelligent.

Nous commençons par établir un modèle mathématique décrivant le comportement thermique du système. Ensuite, un dataset est généré à l'aide d'un contrôleur MPC afin de représenter différentes conditions de fonctionnement.

Par la suite, un modèle de réseau de neurones est conçu et entraîné pour approximer le comportement du contrôleur MPC. Enfin, le modèle est converti en TensorFlow Lite pour permettre son implémentation sur une plateforme embarquée.

3.2 Diagrammes et modélisation UML

Le langage UML permet de représenter de manière claire les interactions entre les acteurs et le système, le déroulement des scénarios de fonctionnement, ainsi que l'organisation logique et matérielle de la solution proposée [13]. Dans ce travail, plusieurs vues complémentaires sont retenues afin de mieux décrire la conception du système : les cas d'utilisation, les diagrammes de séquence, un diagramme d'activité de la boucle de contrôle embarquée et un diagramme de déploiement de l'architecture IoT.

3.2.1 Diagramme de cas d'utilisation global

Description textuelle du diagramme de cas d'utilisation

Le diagramme de cas d'utilisation global présente les principales interactions entre les acteurs externes et le système de régulation thermique intelligent.

Dans cette modélisation, deux acteurs principaux sont retenus : Utilisateur humain, qui représente l'utilisateur responsable de la supervision et du pilotage du système, et le système embarqué Raspberry Pi, qui assure les fonctions locales liées à la mesure, à la réception de la commande et à l'action de chauffage.

Du côté utilisateur humain, le système permet :

- de superviser le système en temps réel ;
- de configurer la régulation thermique ;
- de consulter l'historique et d'analyser les performances.

Du côté du système embarqué Raspberry Pi, le système permet :

- de recevoir la commande de chauffage ;
- d'exécuter l'action de chauffage ;
- de fournir les mesures de température et d'humidité.

Ainsi, ce diagramme fournit une vue globale et claire des fonctionnalités offertes par le système. Il distingue les actions de supervision et de configuration réalisées par utilisateur

humain, ainsi que les fonctions embarquées assurées par la Raspberry Pi.

Diagramme de cas d'utilisation global

La figure suivante présente le diagramme de cas d'utilisation global du système.

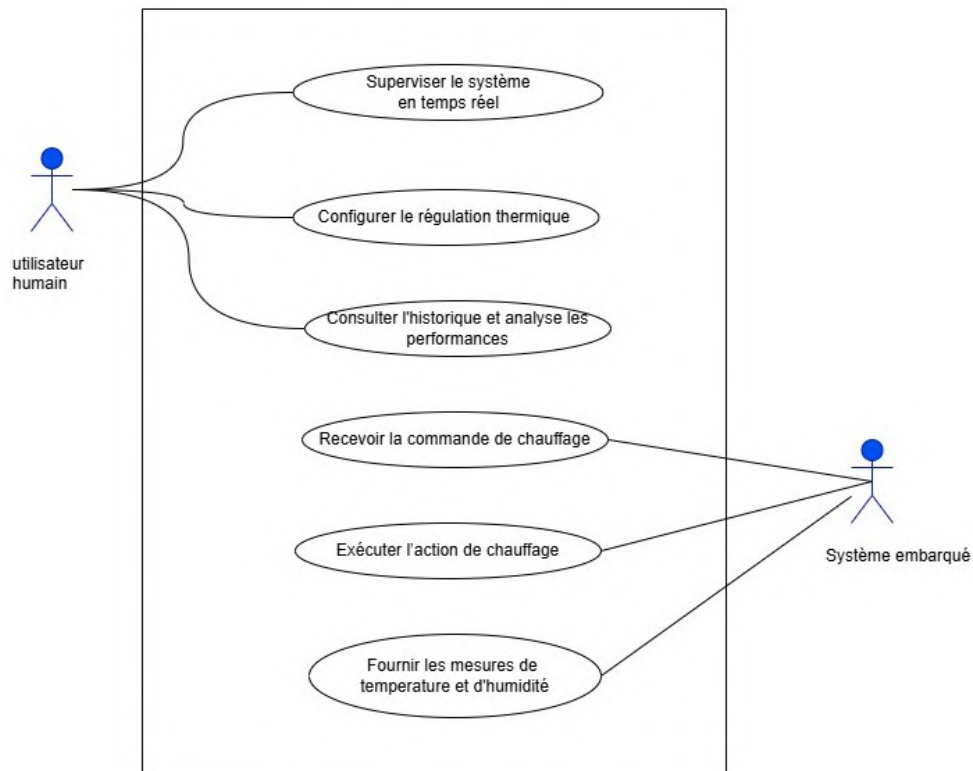


FIGURE 3.1 – Diagramme de cas d'utilisation global du système de régulation thermique intelligent

3.2.2 Raffinement des cas d'utilisation

Cette partie présente le raffinement de trois cas d'utilisation principaux du système. Chaque raffinement permet de détailler les sous-fonctions nécessaires à la réalisation d'une fonctionnalité globale, en distinguant les actions de utilisateur humain et les services fournis par le système.

Raffinement du cas d'utilisation : Superviser le système en temps réel

Le cas d'utilisation « Superviser le système en temps réel » décrit les fonctionnalités permettant à l'utilisateur humain de suivre l'état instantané du procédé thermique à travers le dashboard de supervision.

Élément	Description
Acteurs :	utilisateur humain, système embarqué .
Précondition :	Le système de régulation est actif, la Raspberry Pi publie les mesures et l'état du système, et le dashboard de supervision est accessible.
Cas d'utilisation principal :	Superviser le système en temps réel.
Cas inclus <i>include</i>	- Visualiser la température ; - Visualiser l'humidité ; - Visualiser la consigne actuelle ; - Visualiser la commande de chauffage ; - Visualiser l'état du système ; - Consulter les alertes et le statut ; - Fournir les mesures en temps réel ; - Envoyer l'état du système.
Scénario principal :	1. L'utilisateur humain accède au dashboard de supervision. 2. Le système embarqué fournit les mesures en temps réel. 3. Le dashboard affiche la température, l'humidité, la consigne et la commande de chauffage. 4. Le système embarqué envoie l'état du système. 5. L'utilisateur consulte l'état global, les alertes et le statut de fonctionnement.

Scénarios alternatifs :	1a. Si une mesure n'est pas disponible, la dernière valeur valide peut être affichée temporairement. 2a. Si l'état du système devient critique, une alerte est affichée sur le dashboard. 3a. Si la communication MQTT est interrompue, les données temps réel ne sont plus actualisées jusqu'au rétablissement de la communication.
Postcondition :	L'utilisateur dispose d'une vue instantanée de l'état thermique du procédé et du fonctionnement de la régulation.
Exceptions :	- Perte de communication MQTT ; - Absence de nouvelle mesure ; - Service embarqué arrêté ; - Erreur de capteur ou état critique.

TABLE 3.1 – Description du raffinement relatif à la supervision temps réel

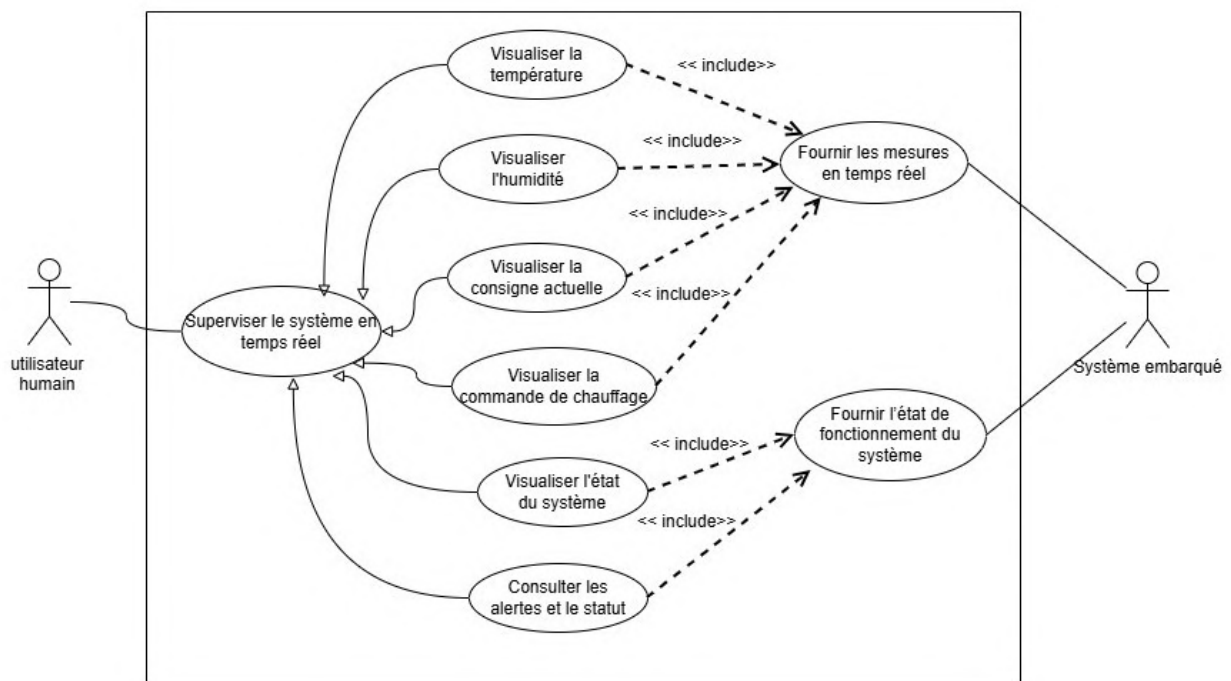


FIGURE 3.2 – Raffinement du cas d'utilisation relatif à la supervision du système en temps réel.

La Fig. 3.2 met en évidence les informations visualisées par l'utilisateur à partir du dashboard. Les relations *include* montrent que l'affichage des grandeurs principales dépend de la disponibilité des mesures et de l'état fournis par le système embarqué.

Raffinement du cas d'utilisation : Configurer la régulation thermique

Le cas d'utilisation « Configurer la régulation thermique » décrit les actions permettant à l'utilisateur humain de paramétrer le mode de fonctionnement du système et de gérer l'état de la régulation.

Élément	Description
Acteur :	Utilisateur humain.
Précondition :	Le dashboard de pilotage est accessible et la communication avec le système embarqué est disponible.
Cas d'utilisation principal :	Configurer la régulation thermique.
Cas inclus <i>include</i>	- Paramétrer le mode de fonctionnement ; - Définir la consigne ; - Régler la puissance manuelle ; - Définir les horaires ; - Définir les consignes programmées ; - Gérer l'état de la régulation ; - Démarrer la régulation ; - Arrêter la régulation ; - Redémarrer la régulation ; - Consulter le statut du système.
Cas étendus <i>extend</i>	- Mode automatique ; - Mode manuel ; - Mode programmé.
Scénario principal :	1. L'utilisateur humain accède à l'interface de configuration. 2. Il choisit le mode de fonctionnement souhaité. 3. En mode automatique, il définit la consigne de température. 4. En mode manuel, il définit la consigne et règle la puissance manuelle. 5. En mode programmé, il définit les horaires et les consignes programmées. 6. Il peut démarrer, arrêter ou redémarrer la régulation. 7. Le système met à jour le statut de fonctionnement.

Scénarios alternatifs :	1a. Si une consigne invalide est saisie, le système refuse la commande ou applique une valeur autorisée. 2a. Si la régulation est arrêtée, la commande de chauffage est désactivée. 3a. Si le service embarqué ne répond pas, le dashboard affiche un statut non disponible. 4a. Si le mode programmé contient des horaires incorrects, la planification n'est pas appliquée.
Postcondition :	Le mode de fonctionnement, la consigne et l'état de la régulation sont mis à jour selon les paramètres définis par l'utilisateur.
Exceptions :	- Perte de communication avec la Raspberry Pi ; - Service de régulation indisponible ; - Paramètres de programmation incorrects.

TABLE 3.2 – Description du raffinement relatif à la configuration de la régulation thermique

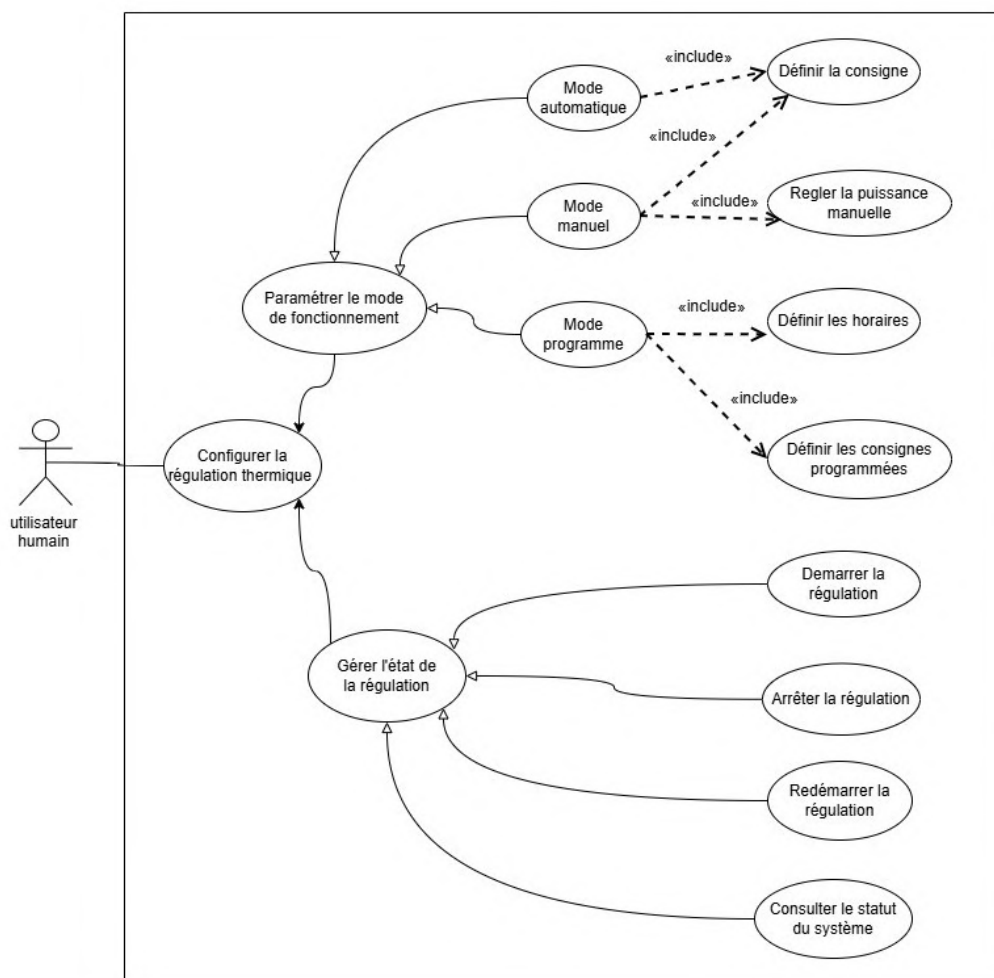


FIGURE 3.3 – Raffinement du cas d'utilisation relatif à la configuration de la régulation thermique.

La Fig. 3.3 détaille les actions de paramétrage disponibles pour l'utilisateur. Les modes automatique, manuel et programmé étendent le paramétrage général du système, tandis que les relations *include* représentent les paramètres nécessaires à chaque mode de fonctionnement.

Raffinement du cas d'utilisation : Consulter l'historique et analyser les performances

Le cas d'utilisation « Consulter l'historique et analyser les performances » décrit les fonctionnalités permettant à l'utilisateur humain d'exploiter les données enregistrées afin d'évaluer le comportement du système thermique.

Élément	Description
Acteur :	Utilisateur humain.

Précondition :	Les données expérimentales sont enregistrées dans la base de données ou disponibles dans les fichiers de journalisation, et l'interface d'analyse est accessible.
Cas d'utilisation principal :	Consulter l'historique et analyser les performances.
Cas inclus <i>include</i>	- Visualiser les courbes historiques ; - Filtrer les données par période ; - Analyser l'évolution de la température ; - Analyser l'évolution de l'humidité ; - Analyser l'évolution de la commande ; - Évaluer la stabilité du système ; - Fournir les données historiques.
Scénario principal :	1. L'utilisateur humain accède à l'interface d'analyse historique. 2. Il choisit la période de données à consulter. 3. Le système fournit les données historiques correspondantes. 4. L'utilisateur visualise les courbes de température, d'humidité et de commande. 5. Il analyse l'évolution des grandeurs mesurées. 6. Il évalue la stabilité globale du système et les performances de régulation.
Scénarios alternatifs :	1a. Si aucune donnée n'est disponible pour la période choisie, le système affiche une absence de données. 2a. Si la base de données est indisponible, l'analyse historique ne peut pas être actualisée. 3a. Si les données sont incomplètes, l'interprétation des performances reste limitée.
Postcondition :	L'utilisateur dispose d'une vue historique du comportement thermique et peut évaluer les performances du système.
Exceptions :	- Absence de données historiques ; - Erreur de connexion à la base de données ; - Données incomplètes ou non exploitables ; - Période de filtrage invalide.

TABLE 3.3 – Description du raffinement relatif à l'analyse historique et aux performances

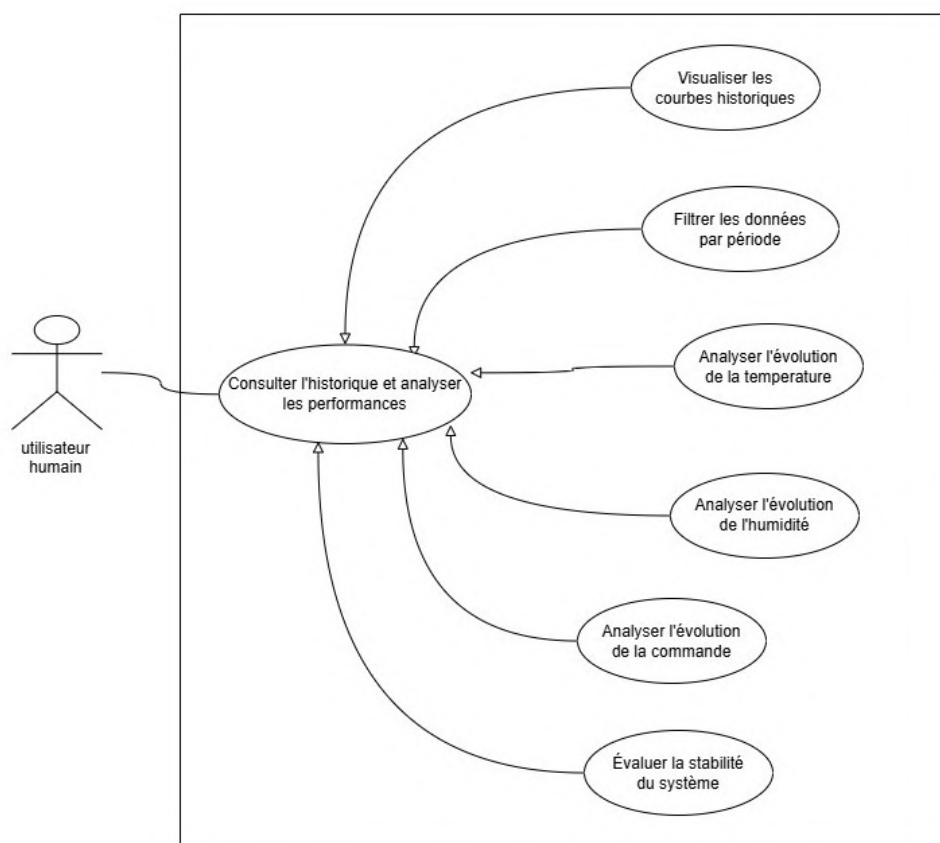


FIGURE 3.4 – Raffinement du cas d'utilisation relatif à la consultation de l'historique et à l'analyse des performances.

La Fig. 3.4 présente les fonctions liées à l'exploitation des données historiques. Les relations *include* montrent que l'analyse des performances repose sur la disponibilité des données enregistrées, qui permettent de visualiser les courbes, de filtrer les essais et d'évaluer la stabilité du système.

3.2.3 Diagrammes de séquence

Les diagrammes de séquence permettent de représenter le fonctionnement dynamique du système et l'ordre chronologique des échanges entre ses différentes composantes. Dans ce travail, deux scénarios principaux sont distingués. Le premier décrit le pilotage du système à partir du dashboard Node-RED, tandis que le second présente le déroulement de la boucle de régulation embarquée exécutée par la Raspberry Pi.

Séquence de pilotage depuis le dashboard Node-RED

Le premier diagramme de séquence présente le pilotage du système thermique à partir du dashboard Node-RED. Ce dashboard constitue l'interface de commande utilisée par l'utilisateur pour saisir les paramètres de régulation, modifier le mode de fonctionnement ou

gérer l'état du service embarqué. Les actions saisies sont d'abord transmises au contrôleur de supervision, qui vérifie les champs et oriente ensuite le traitement selon le type de demande.

Deux scénarios principaux sont distingués. Le premier scénario concerne la configuration de la régulation thermique. Dans ce cas, l'utilisateur peut sélectionner le mode automatique, manuel ou programmé. En mode automatique, la consigne est transmise au programme embarqué de la Raspberry Pi à travers le broker MQTT Mosquitto. La Raspberry Pi met alors à jour le mode et la consigne, puis utilise le contrôleur NN-MPC pour calculer la commande PWM à appliquer au système thermique. En mode manuel, l'utilisateur impose directement la puissance de chauffage, qui est limitée entre 0% et 100% avant d'être appliquée. En mode programmé, le contrôleur de supervision enregistre les paramètres du programme, vérifie périodiquement l'horaire défini, puis publie automatiquement la consigne programmée vers la Raspberry Pi. Cette dernière utilise ensuite le contrôleur NN-MPC pour calculer la commande correspondante.

Le second scénario concerne la gestion du service embarqué. Dans ce cas, l'utilisateur peut demander le démarrage, l'arrêt, le redémarrage ou la consultation de l'état du service `thermal-control.service`. Ces actions sont transmises par le dashboard au contrôleur de supervision, puis exécutées directement sur le service système. Le service retourne ensuite son état, par exemple `active` ou `inactive`, afin que l'interface puisse afficher l'état actualisé.

Après l'application d'une commande de régulation, le programme embarqué publie les données du système, telles que la température, l'humidité, la consigne, la commande et l'état, vers le broker MQTT. Ces données sont ensuite transmises au contrôleur de supervision, qui met à jour l'interface Node-RED afin d'afficher les informations actualisées à l'utilisateur.

Chapitre 3. Modélisation et Conception du Système

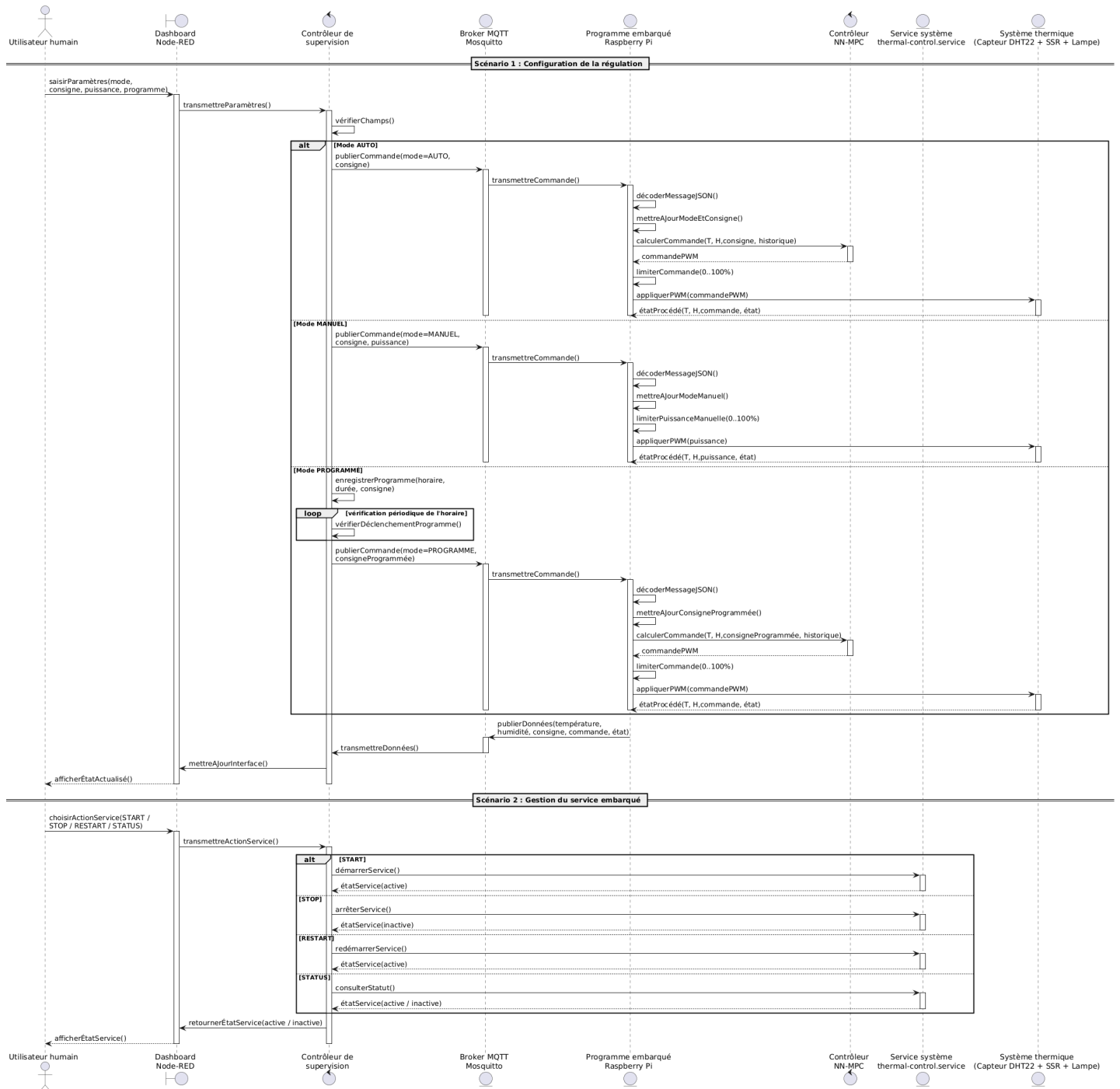


FIGURE 3.5 – Diagramme de séquence du pilotage du système depuis le dashboard Node-RED.

La Fig. 3.5 met en évidence le rôle du dashboard Node-RED comme interface de commande et de supervision. Elle montre que le protocole MQTT assure la communication entre le contrôleur de supervision et le programme embarqué exécuté sur la Raspberry Pi lors de la configuration de la régulation. Elle montre également que la gestion du service embarqué est traitée séparément, à travers des actions système appliquées au service `thermal-control.service`. Cette organisation permet de modifier les paramètres de régulation, de suivre l'état du système et de gérer le service embarqué directement depuis le dashboard, sans accès direct par SSH.

Séquence de régulation embarquée par NN-MPC

Le second diagramme de séquence décrit le fonctionnement interne de la boucle de régulation embarquée exécutée sur la Raspberry Pi. Cette séquence met en évidence les échanges temporels entre les différents éléments du système, à savoir le capteur DHT22, le programme embarqué, le modèle NN-MPC converti en TensorFlow Lite, le relais SSR, la lampe chauffante, le broker MQTT et le dashboard Node-RED.

À chaque cycle de régulation, la Raspberry Pi commence par acquérir les mesures de température et d'humidité à partir du capteur DHT22. Ces mesures sont ensuite vérifiées afin de s'assurer de leur validité avant d'être utilisées par l'algorithme de commande. Lorsque les valeurs mesurées sont valides, le programme embarqué prépare les entrées du modèle NN-MPC, notamment la température, l'humidité, la consigne et les informations historiques nécessaires au calcul de la commande.

Le modèle NN-MPC, exécuté sous TensorFlow Lite, réalise alors l'inférence et retourne une commande de chauffage exprimée en pourcentage. Cette commande est limitée entre 0% et 100% afin de respecter les contraintes physiques de l'actionneur. Elle est ensuite convertie en signal Slow PWM, qui permet de piloter le relais statique SSR. Le relais commande à son tour la lampe chauffante, ce qui modifie progressivement la température à l'intérieur de l'enceinte thermique.

En fonctionnement normal, la Raspberry Pi publie les mesures, la consigne, la commande calculée et l'état du système vers le broker MQTT. Ces données sont ensuite transmises au dashboard Node-RED afin de mettre à jour les indicateurs de supervision en temps réel. En cas de mesure invalide ou d'erreur de lecture du capteur, le système ne poursuit pas le calcul de commande normal. Une alerte est alors publiée via MQTT puis affichée dans l'interface de supervision afin de signaler l'état anormal du système.

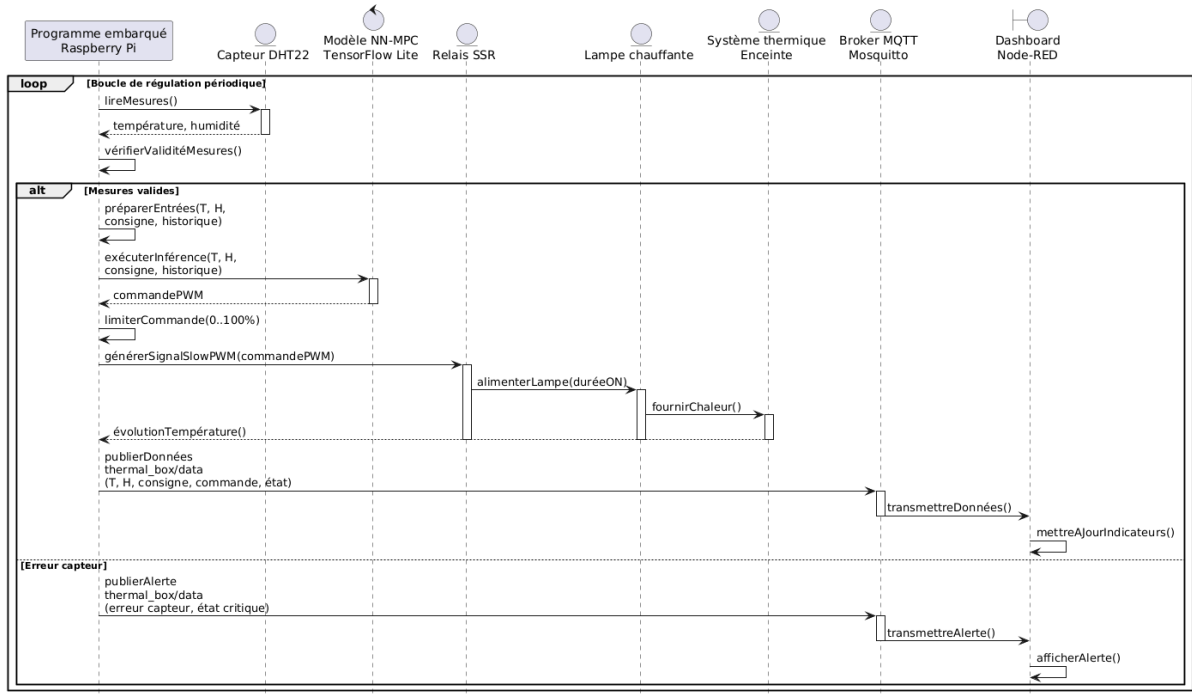


FIGURE 3.6 – Diagramme de séquence de la régulation embarquée par le contrôleur NN-MPC.

La Fig. 3.6 illustre donc le déroulement complet d'un cycle de régulation embarquée. Elle met en relation l'acquisition des mesures, la validation des données, l'exécution du modèle NN-MPC, la génération de la commande Slow PWM, l'actionnement du relais SSR et de la lampe chauffante, ainsi que la publication des données vers la supervision IoT. Ce diagramme complète les cas d'utilisation en décrivant le comportement temporel réel du système lors de son fonctionnement embarqué.

3.2.4 Diagramme d'activité de la boucle de contrôle embarquée

Le diagramme d'activité permet de représenter l'enchaînement logique des opérations exécutées au sein de la boucle de régulation embarquée. Contrairement au diagramme de séquence, qui met l'accent sur les échanges entre composants, ce diagramme décrit ici le flux de traitement interne suivi par la Raspberry Pi à chaque cycle de contrôle.

Dans le système proposé, la boucle commence par l'acquisition des mesures de température et d'humidité à partir du capteur DHT22. Les valeurs obtenues sont ensuite vérifiées avant leur utilisation. Si une nouvelle mesure exploitable est disponible, elle est utilisée pour préparer les entrées du modèle NN-MPC. Dans le cas contraire, le programme conserve temporairement la dernière valeur valide disponible, afin d'assurer la continuité de fonctionnement de la régulation.

Le modèle NN-MPC embarqué est ensuite exécuté pour produire la commande thermique. Cette commande est bornée dans l'intervalle $[0, 100]\%$, puis traduite en signal Slow PWM appliqué au relais SSR. Enfin, les mesures, la consigne, la commande et l'état du

système sont publiés via MQTT pour alimenter la supervision IoT. Le cycle est ensuite répété périodiquement tant que le système reste en fonctionnement.

Diagramme d'activité – Boucle de contrôle embarquée (Raspberry Pi)

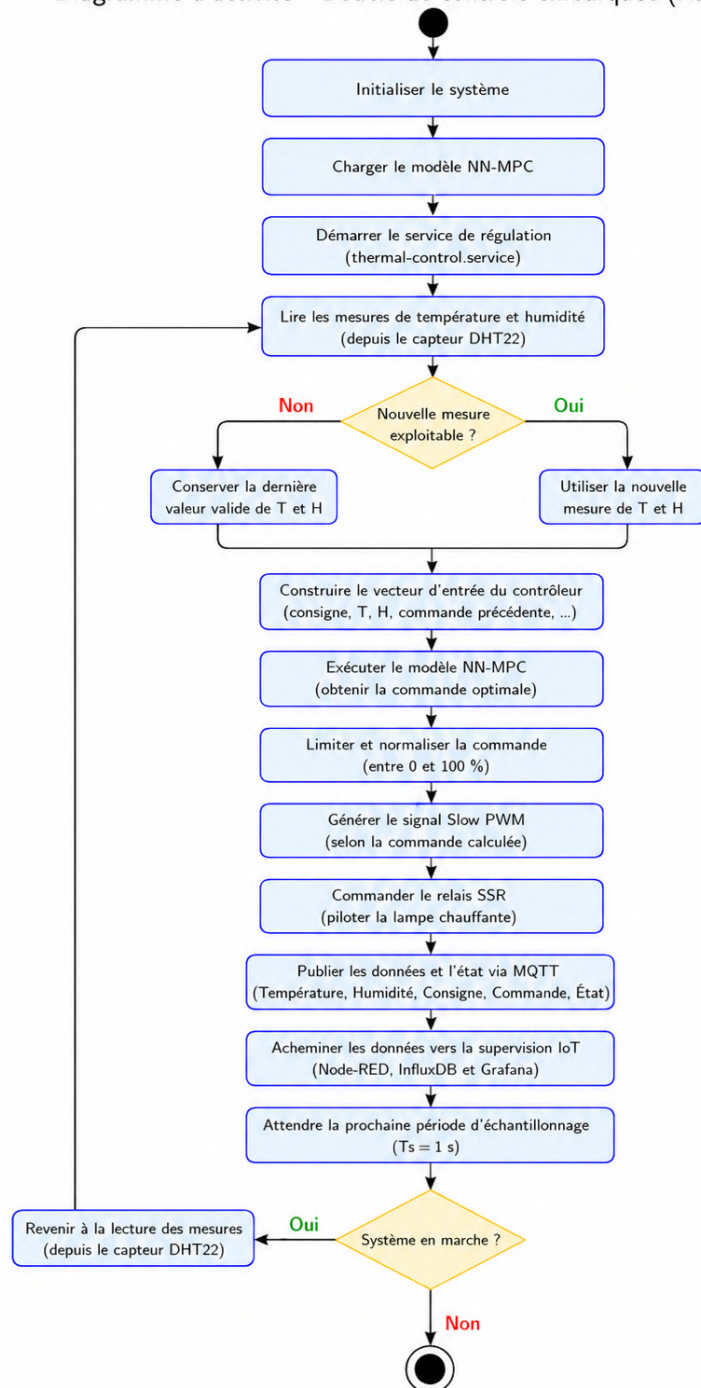


FIGURE 3.7 – Diagramme d'activité de la boucle de contrôle embarquée exécutée sur la Raspberry Pi.

La Fig. 3.7 met en évidence la logique interne d'un cycle complet de régulation. Elle décrit la lecture des mesures, la gestion d'une valeur non exploitable, l'exécution du modèle NN-MPC, la génération du signal Slow PWM, la publication MQTT et la répétition

périodique de la boucle de contrôle.

3.2.5 Diagramme de déploiement de l'architecture IoT

Le diagramme de déploiement complète les représentations précédentes en décrivant l'organisation matérielle et logicielle de la solution réalisée. Il permet d'identifier les principaux nœuds du système, les composants qui y sont exécutés, ainsi que les échanges assurant l'intégration entre la régulation embarquée et la supervision IoT.

Dans l'architecture proposée, la Raspberry Pi 4 constitue l'unité embarquée de contrôle. Elle exécute le service de régulation, le script Python, le modèle NN-MPC, la lecture des mesures issues du capteur DHT22, la génération du signal Slow PWM et la communication MQTT. Elle assure également une journalisation locale des données sous forme de fichiers CSV.

Le broker Mosquitto joue le rôle d'intermédiaire de communication entre la Raspberry Pi et le serveur de supervision. Du côté supervision, Node-RED gère le dashboard de pilotage, les modes AUTO, MANUAL et PROGRAMME, ainsi que les commandes de gestion du service. InfluxDB est utilisé pour le stockage temporel des données, tandis que Grafana permet l'analyse historique intégrée dans l'interface de supervision.

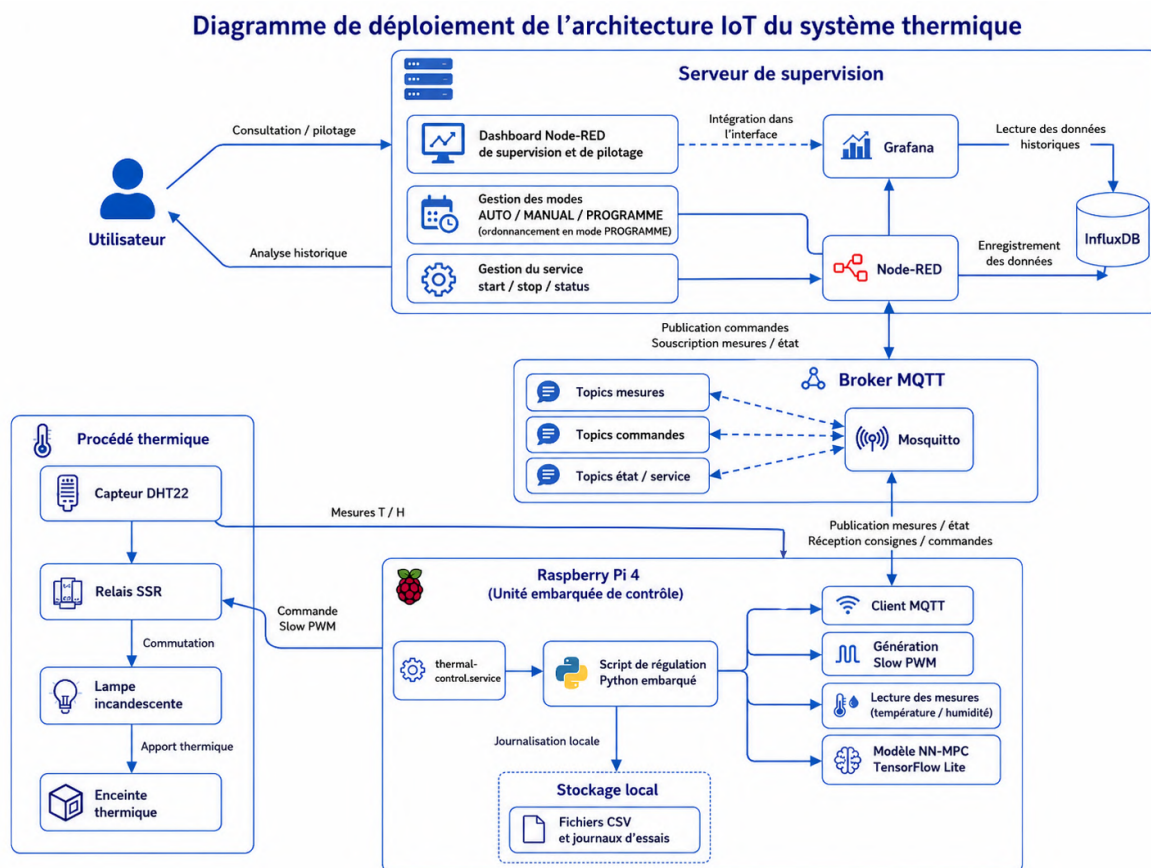


FIGURE 3.8 – Diagramme de déploiement de l'architecture matérielle et logicielle du système de régulation thermique connecté.

La Fig. 3.8 fournit une vue synthétique de la répartition des composants dans l'architecture finale. Elle met en relation le procédé thermique réel, l'unité embarquée Raspberry Pi, le broker MQTT et le serveur de supervision regroupant Node-RED, InfluxDB et Grafana. Ce diagramme permet ainsi de mieux comprendre l'intégration complète entre la régulation NN-MPC, l'infrastructure IoT et l'interface utilisateur.

3.3 Modélisation du système thermique

3.3.1 Description du système physique

Le système étudié est une enceinte thermique chauffée par une lampe via un relais statique (SSR). La température interne évolue sous l'effet d'un apport thermique contrôlé et de pertes vers l'environnement.

Du point de vue physique, le système se caractérise par :

- une forte inertie thermique due à la masse de l'air et des parois ;
- des pertes proportionnelles à l'écart de température avec l'ambiante ;
- une réponse lente, typique des systèmes diffusifs.

3.3.2 Schéma du système thermique

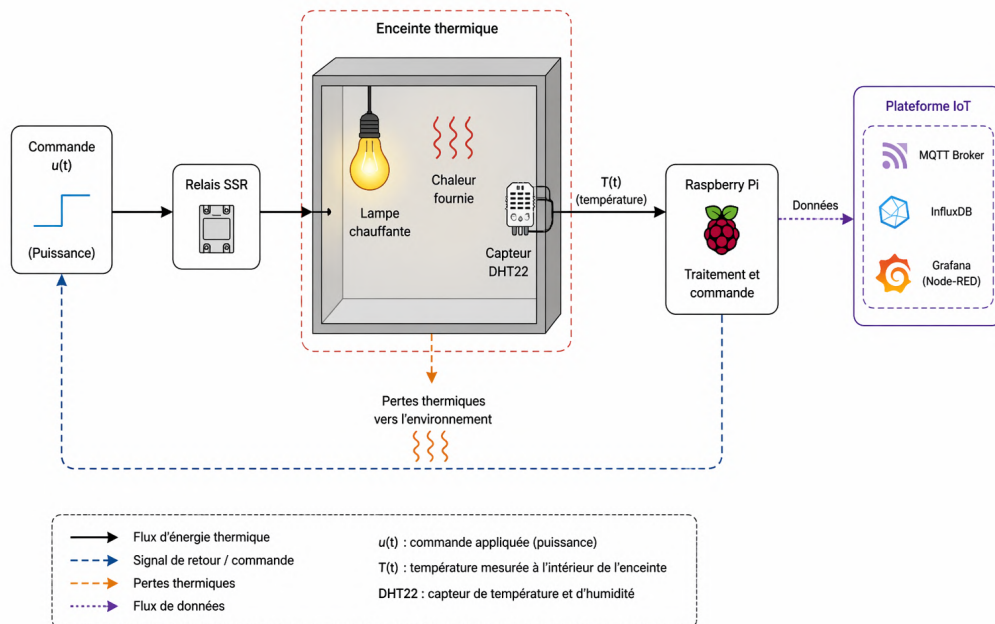


FIGURE 3.9 – Schéma simplifié du système thermique en boucle fermée

La commande $u(t)$ agit sur la lampe, générant un flux thermique entrant. La température mesurée est renvoyée à l'unité de calcul (Raspberry Pi) pour assurer la régulation.

3.3.3 Modélisation mathématique

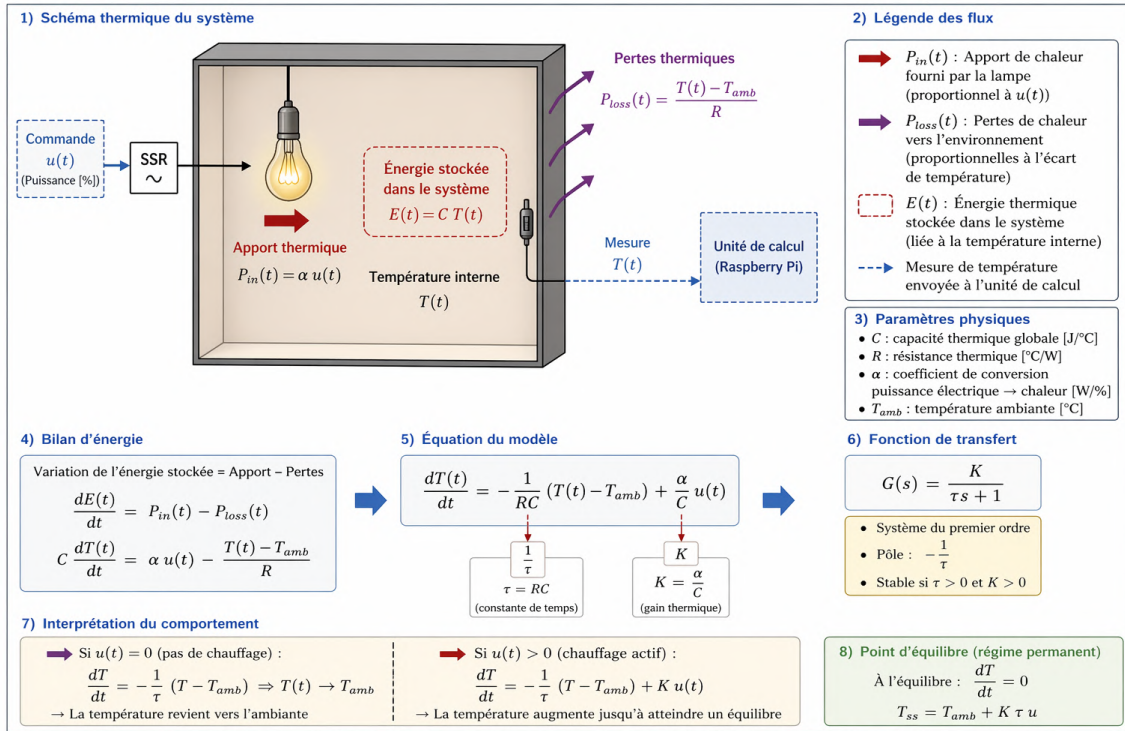


Figure 3.2 : Bilan thermique de l'enceinte – Flux de chaleur, pertes et énergie stockée

FIGURE 3.10 – Représentation du bilan thermique du système mettant en évidence l'apport de chaleur, les pertes vers l'environnement et l'énergie stockée

La figure 3.10 illustre les échanges thermiques au sein du système, notamment l'apport de chaleur fourni par la lampe, les pertes vers l'environnement et l'énergie stockée dans l'enceinte.

Elle montre que l'évolution de la température résulte d'un équilibre entre ces différents flux énergétiques.

La modélisation du système thermique repose sur un bilan d'énergie appliqué à l'enceinte. La variation de l'énergie thermique stockée dans le système est égale à la différence entre la puissance thermique apportée et les pertes vers l'environnement . Ces échanges thermiques peuvent être formalisés à l'aide d'un bilan d'énergie, ce qui conduit à l'équation suivante :

$$C \frac{dT(t)}{dt} = P_{in}(t) - P_{loss}(t) \quad (3.1)$$

où C représente la capacité thermique globale du système. Ce paramètre regroupe l'effet de la masse thermique (air, parois, composants internes) et caractérise la quantité d'énergie nécessaire pour faire varier la température.

Modélisation des flux thermiques La puissance thermique perdue vers l'environnement

est modélisée par une loi de type résistive :

$$P_{\text{loss}}(t) = \frac{T(t) - T_{\text{amb}}}{R}$$

où T_{amb} désigne la température ambiante de l'environnement extérieur à l'enceinte, mesurée à 21.1 °C, et R la résistance thermique équivalente. Ce terme traduit le fait que les pertes augmentent avec l'écart de température entre l'intérieur et l'extérieur.

La puissance thermique fournie par la lampe est supposée proportionnelle à la commande appliquée :

$$P_{\text{in}}(t) = \alpha u(t) \quad (3.2)$$

où α représente l'efficacité de conversion de la puissance électrique en chaleur utile, et $u(t)$ la commande thermique appliquée, exprimée en pourcentage dans l'intervalle $[0, 100]\%$.

Équation du modèle

En remplaçant ces expressions dans le bilan d'énergie, on obtient :

$$C \frac{dT(t)}{dt} = \alpha u(t) - \frac{T(t) - T_{\text{amb}}}{R} \quad (3.3)$$

qui peut être réécrite sous forme canonique :

$$\frac{dT(t)}{dt} = -\frac{1}{\tau}(T(t) - T_{\text{amb}}) + K u(t) \quad (3.4)$$

avec :

$$\tau = RC \quad (\text{constante de temps}) \quad \text{et} \quad K = \frac{\alpha}{C} \quad (\text{gain thermique})$$

Analyse physique du modèle

Cette équation met en évidence deux phénomènes fondamentaux :

- le terme $-\frac{1}{\tau}(T - T_{\text{amb}})$ correspond aux pertes thermiques naturelles du système. Il traduit la tendance du système à revenir vers la température ambiante en absence de chauffage ;
- le terme $K u(t)$ représente l'apport de chaleur généré par la lampe. Plus la commande est élevée, plus la température augmente.

La constante de temps τ caractérise la rapidité de réponse du système :

- τ élevée $\Rightarrow$ système lent (forte inertie thermique) ;
- τ faible $\Rightarrow$ système plus réactif.

Forme centrée

Afin de simplifier l'analyse, on introduit la variable d'état centrée :

$$x(t) = T(t) - T_{amb}$$

Le modèle devient alors :

$$\dot{x}(t) = -\frac{1}{\tau}x(t) + K u(t) \quad (3.5)$$

Cette forme met en évidence un système linéaire du premier ordre, plus adapté à l'analyse et à la conception de lois de commande.

Fonction de transfert

Dans le domaine de Laplace, sous conditions initiales nulles, la relation entrée-sortie s'écrit :

$$G(s) = \frac{X(s)}{U(s)} = \frac{K}{\tau s + 1} \quad (3.6)$$

Cette fonction de transfert correspond à un système du premier ordre stable, dont le pôle est situé en $-\frac{1}{\tau}$.

Conclusion sur le modèle

Le modèle obtenu permet de capturer le comportement global du système thermique avec une précision suffisante tout en restant simple.

Il constitue une base adaptée pour :

- la simulation du système ;
- la conception de contrôleurs (PID, MPC) ;
- la génération de données pour l'apprentissage du modèle intelligent.

Prise en compte de l'humidité. Le modèle thermique adopté dans ce travail est volontairement simplifié et décrit principalement l'évolution de la température de l'enceinte. L'humidité relative n'est donc pas introduite explicitement dans le bilan énergétique.

Toutefois, l'humidité peut influencer indirectement le comportement thermique du milieu interne, puisque les propriétés thermiques de l'air humide diffèrent légèrement de celles de l'air sec. Dans ce contexte, cette grandeur est conservée comme variable environnementale mesurée par le capteur DHT22, afin de fournir au réseau NN-MPC une information complémentaire sur l'état thermo-hygrométrique de l'enceinte.

3.3.4 Identification des paramètres

À partir des données expérimentales, le modèle identifié est :

$$441 \dot{T}(t) + (T(t) - 21.1) = 0.355 u(t) \quad (3.7)$$

où $u(t)$ est exprimé en pourcentage de commande dans l'intervalle $[0, 100]\%$, $K = 0.355\text{ }^{\circ}\text{C}\,\%^{-1}$ et $\tau = 441\text{ s}$.

Ainsi, les paramètres identifiés sont :

- $\tau = 441\text{ s}$, traduisant une dynamique thermique lente ;
- $K = 0.355\text{ }^{\circ}\text{C}\,\%^{-1}$, correspondant au gain thermique du système.

Ces valeurs sont cohérentes avec le comportement observé sur la maquette.

3.3.5 Simulation du système

Le modèle est simulé sous GEKKO avec un contrôleur MPC.

L'objectif est de :

- vérifier la stabilité du système ;
- observer la réponse à des variations de consigne ;
- générer une commande optimale.

3.3.6 Principe de validation du modèle

Afin de vérifier la cohérence du modèle thermique identifié avant son utilisation dans la génération du dataset et la conception des stratégies de commande, une étape de validation expérimentale est prévue.

Cette validation consiste à comparer la réponse simulée du modèle avec la réponse mesurée sur la maquette thermique pour une excitation de référence. Elle permet d'apprécier la capacité du modèle à reproduire la dynamique globale du procédé, notamment son évolution transitoire et son comportement thermique général.

Les résultats graphiques de cette comparaison ainsi que leur interprétation seront présentés dans le chapitre consacré aux résultats et à la discussion.

3.4 Génération du dataset

3.4.1 Utilisation de GEKKO et du contrôleur MPC

Afin de générer des données d'apprentissage de haute qualité, nous avons utilisé l'outil GEKKO pour implémenter un contrôleur de type Model Predictive Control (MPC) [22].

Le choix du MPC comme contrôleur de référence est justifié par sa capacité à anticiper l'évolution future du procédé, à intégrer des contraintes sur la commande et à produire une action optimale sur un horizon de prédiction. Ces propriétés expliquent son utilisation fréquente dans les systèmes industriels et les procédés à dynamique lente [1, 2, 3].

Dans notre approche, le modèle thermique identifié est utilisé comme base de prédiction. À chaque instant, le MPC calcule une séquence de commandes optimales sur un horizon

de prédiction, en minimisant une fonction de coût qui pénalise l'écart à la consigne ainsi que les variations de la commande.

Le problème d'optimisation peut être formulé comme suit :

$$\min_{u(t)} \sum_{k=1}^N \left[(T_k - T_{ref})^2 + \lambda \Delta u_k^2 \right] \quad (3.8)$$

où N est l'horizon de prédiction et λ un coefficient de régularisation.

Dans notre implémentation GEKKO :

- la variable manipulée u est contrainte entre 0% et 100% ;
- la variable contrôlée T suit une consigne imposée ;
- le contrôleur fonctionne en mode receding horizon (IMODE = 6).

Le MPC agit ainsi comme un **contrôleur expert**, fournissant la commande optimale pour chaque situation.

3.4.2 Stratégie de génération

La génération du dataset repose sur une simulation en boucle fermée du système thermique contrôlé par le MPC.

Afin de couvrir un large espace de fonctionnement, plusieurs stratégies ont été adoptées :

- **Variation de la consigne** : la consigne de température est modifiée périodiquement dans l'intervalle [25 °C à 35 °C] ;
- **Variabilité thermo-hygrométrique** : l'humidité $H(t)$ est générée aléatoirement dans l'intervalle 40%–95% afin de représenter plusieurs conditions environnementales possibles de l'enceinte et de conserver une variable contextuelle également mesurée en fonctionnement réel par le capteur DHT22 ;
- **Exploration dynamique** : la simulation est effectuée sur 10 000 pas de temps pour couvrir différents régimes (transitoire et permanent).

Une attention particulière est portée à la stabilité du régime permanent. Lorsque la température est proche de la consigne, une correction est appliquée afin de garantir une commande cohérente avec l'équilibre thermique.

Cette stratégie permet d'obtenir un dataset représentatif, couvrant à la fois les phases de montée, de stabilisation et de perturbation.

3.4.3 Structure des données

Chaque échantillon du dataset est constitué des variables suivantes :

- température actuelle $T(t)$;
- consigne T_{ref} ;
- humidité $H(t)$;
- température précédente $T(t - 1)$;

- température précédente $T(t - 2)$;
- commande optimale $u_{MPC}(t)$.

L'introduction des températures passées permet d'intégrer une information temporelle, essentielle pour capturer la dynamique du système.

Le dataset généré contient 10 000 échantillons et est sauvegardé sous forme de fichier CSV. Il constitue la base d'apprentissage du modèle de réseau de neurones.

TABLE 3.4 – Caractéristiques du dataset généré

Paramètre	Valeur
Nombre d'échantillons	10 000
Température ambiante	21.1 °C
Intervalle de consigne	25 °C à 35 °C
Mode de génération des consignes	Paliers successifs définis aléatoirement
Intervalle d'humidité	40–95%
Commande thermique	0–100%
Période d'échantillonnage	1 s

3.5 Clarification méthodologique sur les deux rôles du MPC

Dans ce projet, le contrôleur MPC est utilisé dans deux contextes différents. Cette distinction est importante car le MPC utilisé pour générer le dataset d'apprentissage n'a pas le même rôle que le MPC utilisé pour la comparaison expérimentale.

3.5.1 MPC de référence utilisé pour la génération du dataset

Dans la phase de génération du dataset, un contrôleur MPC basé sur GEKKO est utilisé hors-ligne comme contrôleur expert. Ce MPC est configuré pour produire des commandes optimales à partir du modèle thermique identifié, sur un grand nombre de conditions de fonctionnement.

L'objectif principal de cette étape est d'obtenir une base d'apprentissage de bonne qualité. Le MPC joue donc le rôle d'un générateur de commandes optimales : pour chaque état thermique du système, il calcule la commande u^* qui permet de suivre la consigne.

Cependant, ce MPC de référence est relativement coûteux en calcul, car il résout un problème d'optimisation à chaque pas de temps. Il est donc utilisé principalement en simulation et hors-ligne pour créer le dataset, et non comme solution finale embarquée.

3.5.2 MPC simplifié utilisé pour l'évaluation expérimentale

Pour la comparaison expérimentale sur la maquette réelle, un contrôleur MPC basé sur GEKKO est également utilisé, mais avec une configuration plus légère et adaptée aux essais sur Raspberry Pi.

Le rôle de ce MPC expérimental est de fournir une courbe de référence permettant de comparer les performances du NN-MPC avec un contrôleur prédictif classique. Il est donc utilisé comme contrôleur de comparaison, au même titre que le PID.

Cette version n'a pas le même objectif que le MPC utilisé pour générer le dataset. Elle sert principalement à produire les résultats expérimentaux nécessaires à la comparaison entre PID, MPC et NN-MPC.

3.5.3 Justification du choix du NN-MPC

Le choix final du NN-MPC est motivé par les contraintes embarquées. Un MPC classique nécessite la résolution d'un problème d'optimisation à chaque itération, ce qui peut devenir coûteux sur une Raspberry Pi, surtout dans une application temps réel.

Le NN-MPC permet de conserver le comportement appris à partir du MPC de référence, tout en remplaçant l'optimisation en ligne par une simple inférence du réseau de neurones. Après conversion en TensorFlow Lite, le modèle devient très léger et rapide, ce qui le rend plus adapté au déploiement embarqué.

Ainsi, le MPC de référence est utilisé pour apprendre une stratégie de commande performante, le MPC expérimental est utilisé pour la comparaison, et le NN-MPC constitue la solution finale proposée pour l'implémentation temps réel sur Raspberry Pi.

TABLE 3.5 – Différence entre les deux utilisations du MPC dans ce projet

Aspect	MPC pour génération du dataset	MPC pour évaluation expérimentale
Objectif	Générer des commandes optimales pour entraîner le NN-MPC	Produire une courbe de comparaison avec PID et NN-MPC
Environnement	Simulation hors-ligne sous GEKKO	Essai réel sur Raspberry Pi et maquette thermique
Niveau de calcul	Plus coûteux, car orienté génération de données optimales	Configuration plus légère, adaptée à l'évaluation expérimentale
Rôle	Contrôleur expert / professeur du réseau de neurones	Contrôleur de référence pour la comparaison
Utilisation finale	Non destiné au déploiement embarqué direct	Utilisé uniquement pour l'évaluation comparative
Limite	Trop coûteux pour être utilisé comme solution temps réel finale sur Raspberry Pi	Moins représentatif du MPC expert utilisé pour générer le dataset
Solution retenue	NN-MPC, car il approxime la loi de commande MPC avec un coût de calcul très faible après apprentissage.	

3.6 Conception du modèle de réseau de neurones

3.6.1 Objectif du modèle

L'objectif est de remplacer le contrôleur MPC, coûteux en calcul lorsqu'il est résolu en ligne, par un modèle de réseau de neurones capable de fournir une commande quasi instantanée.

Le principe retenu consiste à apprendre, à partir des données générées par le MPC de référence, une approximation de sa loi de commande. Cette stratégie s'inscrit dans les approches d'approximation neuronale du MPC, dont l'objectif est de réduire le coût de calcul en ligne tout en conservant un comportement proche de celui du contrôleur prédictif original [4, 5, 7].

Le réseau de neurones est ainsi entraîné à approximer la fonction :

$$f : (T(t), T_{ref}, H(t), T(t-1), T(t-2)) \longrightarrow u(t)$$

où $u(t)$ correspond à la commande optimale fournie par le MPC.

Cette approche est connue sous le nom de **NN-MPC** (Neural Network Model Predictive

Control).

3.6.2 Choix du modèle

Un réseau de neurones de type perceptron multicouche (MLP) a été retenu.

Ce choix est motivé par :

- sa capacité à approximer des fonctions non linéaires complexes (théorème d'approximation universelle) ;
- sa simplicité de mise en œuvre ;
- sa compatibilité avec une exécution embarquée (faible coût de calcul).

Contrairement à des architectures plus complexes (RNN, LSTM), le MLP offre un bon compromis entre précision et rapidité d'inférence dans notre cas.

3.6.3 Variables d'entrée et sortie

Le modèle prend en entrée les variables suivantes :

- température actuelle $T(t)$;
- consigne T_{ref} ;
- humidité $H(t)$;
- température passée $T(t - 1)$;
- température passée $T(t - 2)$.

L'humidité $H(t)$ est intégrée comme variable contextuelle mesurée en temps réel par le capteur DHT22. Même si elle n'est pas explicitement incluse dans le modèle thermique simplifié, elle peut refléter certaines variations de l'environnement interne de l'enceinte et contribuer à enrichir l'information fournie au réseau de neurones lors de la prédiction de la commande. Dans ce travail, cette variable est considérée comme une information contextuelle complémentaire, et non comme une grandeur dont l'influence directe sur la commande optimale est explicitement démontrée par le modèle physique.

Les températures passées $T(t - 1)$ et $T(t - 2)$ sont ajoutées afin d'intégrer implicitement la dynamique du système thermique et de fournir au réseau une information sur l'évolution récente de la température.

La sortie du modèle correspond à la commande thermique $u(t)$ appliquée à la lampe. Dans le modèle physique et dans le dataset de référence, cette commande est exprimée en pourcentage dans l'intervalle $[0, 100]\%$.

Afin de faciliter l'apprentissage du réseau de neurones, la sortie est normalisée dans l'intervalle $[0, 1]$ pendant la phase d'entraînement. Après l'inférence, la valeur prédite par le réseau, notée $\hat{u}_{\text{norm}}(t)$, est reconvertie en pourcentage de commande selon la relation :

$$u_{\%}(t) = 100 \times \hat{u}_{\text{norm}}(t) \quad (3.9)$$

La commande obtenue appartient ainsi à l'intervalle $[0, 100]\%$ avant d'être appliquée à la lampe sous forme de signal Slow PWM.

3.6.4 Architecture du réseau

L'architecture adoptée est composée de plusieurs couches entièrement connectées (Dense).

- Couche d'entrée : 5 neurones ;
- Couche cachée 1 : 32 neurones avec activation ReLU ;
- Couche Dropout (0.2) ;
- Couche cachée 2 : 16 neurones avec activation ReLU ;
- Couche Dropout (0.2) ;
- Couche cachée 3 : 8 neurones avec activation ReLU ;
- Couche de sortie : 1 neurone avec activation Sigmoid.

Les fonctions d'activation ReLU sont utilisées dans les couches cachées pour leur efficacité dans l'apprentissage de relations non linéaires. En sortie, l'activation Sigmoid est retenue afin de produire directement une commande normalisée dans l'intervalle $[0, 1]$.

Ce choix est adapté à la nature physique de la variable à prédire, puisque la commande thermique doit rester comprise entre 0% et 100%. Une activation linéaire aurait pu générer des valeurs en dehors de l'intervalle admissible, nécessitant ensuite une saturation externe dans le programme embarqué. Une ReLU clippée aurait également permis de borner la sortie, mais au prix d'un mécanisme de limitation supplémentaire. L'activation Sigmoid offre donc une solution plus directe, en intégrant naturellement la contrainte de bornage dans l'architecture du réseau.

L'utilisation de Dropout permet de réduire le risque de surapprentissage en désactivant aléatoirement certains neurones durant l'entraînement.

3.6.5 Justification de l'architecture

Le choix de l'architecture repose sur plusieurs considérations :

- une structure progressive ($32 \rightarrow 16 \rightarrow 8$) permet de réduire progressivement la dimension des représentations internes ;
- un nombre limité de couches garantit un bon compromis entre capacité d'apprentissage et coût de calcul ;
- l'ajout de Dropout améliore la généralisation du modèle sur des données non vues ;
- l'activation Sigmoid assure une sortie bornée compatible avec la commande physique du système.

Cette architecture est volontairement compacte afin de permettre une exécution efficace sur Raspberry Pi, tout en conservant une précision élevée.

3.6.6 Avantages de l'approche NN-MPC

L'utilisation d'un réseau de neurones pour approximer le MPC présente plusieurs avantages :

- réduction significative du temps de calcul grâce à une inférence rapide ;
- suppression de la résolution répétée d'un problème d'optimisation en ligne ;
- adaptation aux contraintes des systèmes embarqués à ressources limitées ;
- conservation d'un comportement proche du contrôleur prédictif de référence.

Ces avantages sont cohérents avec les travaux consacrés à l'approximation du MPC par des modèles neuronaux ou explicites, qui visent à transférer hors ligne la complexité du calcul afin d'obtenir une commande plus rapide lors de l'exécution temps réel [6, 7, 8].

Ainsi, le modèle NN-MPC constitue une solution efficace pour une implémentation temps réel sur Raspberry Pi.

3.7 Entraînement du modèle

3.7.1 Prétraitement des données

Avant l'entraînement, les données issues du dataset sont normalisées afin d'améliorer la stabilité et la vitesse de convergence.

Les variables d'entrée sont mises à l'échelle dans l'intervalle $[0, 1]$ à l'aide d'une normalisation min-max :

$$x_{norm} = \frac{x - x_{min}}{x_{max} - x_{min}} \quad (3.10)$$

Cette étape permet d'éviter que certaines variables dominent l'apprentissage en raison de leur échelle.

Les données sont ensuite divisées en trois ensembles :

- **Ensemble d'apprentissage** (70%) ;
- **Ensemble de validation** (15%) ;
- **Ensemble de test** (15%).

3.7.2 Paramètres d'apprentissage

L'entraînement du réseau de neurones est réalisé à l'aide des paramètres suivants :

- Optimiseur : Adam ;
- Fonction de perte : Mean Squared Error (MSE) ;
- Batch size : 32 ;
- Nombre maximal d'époques : 200 ;
- Early stopping : activé avec une patience de 15 ;

- Répartition des données : 70% entraînement, 15% validation et 15% test.

L'optimiseur Adam est choisi pour sa rapidité de convergence et sa robustesse face aux variations de gradient.

La fonction de perte MSE permet de mesurer l'écart entre la commande prédite par le réseau et la commande optimale fournie par le MPC.

3.7.3 Suivi de l'apprentissage

Afin de contrôler le bon déroulement de l'entraînement, les pertes calculées sur les ensembles d'apprentissage et de validation sont suivies au cours des époques.

L'évolution conjointe de ces deux courbes permet d'analyser :

- la convergence progressive du modèle ;
- la stabilité de l'apprentissage ;
- l'apparition éventuelle d'un surapprentissage ;
- l'efficacité du mécanisme d'*early stopping*.

Les courbes obtenues et leur interprétation seront présentées dans le chapitre consacré aux résultats expérimentaux.

3.7.4 Critères d'évaluation du modèle

Après l'entraînement, les performances du réseau NN-MPC sont évaluées sur l'ensemble de test afin de mesurer sa capacité à approximer la commande produite par le contrôleur MPC de référence.

Les indicateurs retenus sont :

- l'erreur quadratique moyenne (**MSE**) ;
- l'erreur absolue moyenne (**MAE**) ;
- le coefficient de détermination (**R²**).

Ces métriques permettent d'apprécier la précision globale de la prédiction, l'écart moyen entre la commande réelle et la commande estimée, ainsi que la capacité du modèle à reproduire la variabilité de la loi de commande MPC.

Les valeurs numériques associées à ces indicateurs seront détaillées dans le chapitre des résultats et discussion.

3.7.5 Principe de comparaison entre la commande prédite et la commande MPC

Une comparaison graphique est également prévue entre :

- la commande optimale générée par le contrôleur MPC de référence ;
- la commande prédite par le réseau de neurones NN-MPC.

Cette comparaison permet de vérifier visuellement la qualité de l'approximation réalisée par le modèle neuronal et d'identifier d'éventuels écarts entre les deux signaux.

Les résultats de cette analyse comparative seront présentés dans le chapitre consacré à l'évaluation expérimentale.

3.8 Conversion du modèle

3.8.1 Conversion en TensorFlow Lite

Après la phase d'entraînement, le modèle de réseau de neurones est converti vers le format TensorFlow Lite afin de permettre son exécution sur un système embarqué.

TensorFlow Lite est une version optimisée de TensorFlow conçue pour les dispositifs à ressources limitées, tels que la Raspberry Pi [21, 14].

La conversion est réalisée à l'aide de l'outil **TFLite Converter**, qui permet de transformer le modèle entraîné en un format léger et efficace.

Objectifs de la conversion :

- réduire la taille du modèle ;
- améliorer le temps d'inférence ;
- permettre une exécution locale sans serveur distant ;
- faciliter l'intégration dans un système embarqué.

Principe de fonctionnement :

Le modèle converti est exécuté sur la Raspberry Pi à l'aide de l'interpréteur TensorFlow Lite.

À chaque itération de la boucle de contrôle, les données d'entrée (température, consigne, humidité et historique) sont injectées dans le modèle, qui génère instantanément la commande thermique.

Cette commande est ensuite appliquée au système via le relais SSR, permettant une régulation en temps réel.

Avantages de TensorFlow Lite :

- faible consommation mémoire ;
- temps d'exécution réduit ;
- compatibilité avec les systèmes embarqués ;
- absence de dépendance à un serveur externe.

3.9 Conclusion

Dans ce chapitre, nous avons présenté la modélisation et la conception du système de régulation thermique intelligent.

Nous avons d’abord décrit les interactions principales du système à travers les diagrammes UML. Ensuite, le comportement thermique de l’enceinte a été modélisé à partir d’un bilan énergétique, ce qui a permis d’obtenir un modèle du premier ordre adapté à la simulation et à la commande.

Ce modèle a ensuite été utilisé pour générer un dataset d’apprentissage à l’aide d’un contrôleur MPC sous GEKKO. Nous avons également présenté la structure du modèle NN-MPC, ses variables d’entrée et de sortie, ainsi que la méthodologie d’entraînement et la conversion vers TensorFlow Lite.

Ainsi, ce chapitre constitue la base théorique et méthodologique nécessaire à l’implémentation pratique du système dans le chapitre suivant.

Implémentation et Validation du Système

Sommaire

4.1	Introduction.....	52
4.2	Architecture matérielle du système.....	52
4.2.1	Unité de traitement embarquée : Raspberry Pi	52
4.2.2	Capteur de température et d’humidité DHT22	53
4.2.3	Relais statique SSR	55
4.2.4	Lampe incandescente	56
4.2.5	Enceinte thermique	57
4.2.6	Montage expérimental	57
4.3	Architecture globale et chaîne fonctionnelle	58
4.3.1	Chaîne fonctionnelle du système	59
4.3.2	Synthèse des choix techniques	59
4.4	Implémentation logicielle embarquée	59
4.4.1	Environnement de développement	60
4.4.2	Déploiement du modèle NN-MPC sur Raspberry Pi	60
4.4.3	Boucle de contrôle embarquée	60
4.4.4	Génération du signal de commande	61
4.5	Communication MQTT et structure des messages	62
4.5.1	Topics MQTT utilisés.....	62
4.5.2	Structure du message de mesure.....	63
4.5.3	Structure du message de commande	63
4.6	Supervision et pilotage avec Node-RED.....	64
4.6.1	Logique interne du flow Node-RED	64
4.6.2	Séparation entre mesures et commandes	67
4.6.3	Développement du dashboard Node-RED	67
4.6.4	Gestion du service système	68

4.6.5	Mode programmé et ordonnanceur temporel	68
4.7	Stockage et analyse historique des données.....	69
4.7.1	Analyse historique avec Grafana	70
4.8	Robustesse et gestion des défauts	71
4.9	Précautions de sécurité.....	72
4.10	Validation fonctionnelle du système	72
4.10.1	Scénario de test	72
4.10.2	Vérification de la communication MQTT.....	73
4.10.3	Vérification du dashboard Node-RED	73
4.10.4	Vérification du mode programmé	73
4.10.5	Vérification de l'intégration Grafana	73
4.11	Conclusion	73

4.1 Introduction

Ce chapitre présente l'implémentation pratique du système de régulation thermique intelligent développé dans ce projet. Après la phase de modélisation et de conception du contrôleur NN-MPC, cette partie décrit la réalisation concrète de la solution sur une plateforme embarquée basée sur Raspberry Pi.

L'objectif principal est de montrer comment les différents éléments matériels et logiciels ont été intégrés afin de former un système complet capable d'assurer l'acquisition des mesures, l'exécution du contrôleur, la génération de la commande thermique, la supervision à distance et l'enregistrement des données expérimentales.

Le système réalisé ne se limite pas à une simple boucle de régulation locale. Il intègre également une couche IoT permettant le monitoring en temps réel, la modification de la consigne, la gestion du service embarqué, le mode programmé et l'analyse historique des données à travers Grafana.

4.2 Architecture matérielle du système

L'architecture matérielle du système repose sur une chaîne fonctionnelle complète composée d'un capteur, d'une unité de traitement embarquée, d'un actionneur de puissance et d'une enceinte thermique.

Le principe général est le suivant : le capteur DHT22 mesure la température et l'humidité à l'intérieur de l'enceinte, la Raspberry Pi exécute le contrôleur NN-MPC, puis la commande calculée est appliquée à la lampe chauffante à travers un relais statique SSR. Cette organisation permet de réaliser une boucle de régulation thermique en conditions réelles.

4.2.1 Unité de traitement embarquée : Raspberry Pi

La carte Raspberry Pi 4 est utilisée comme unité de traitement principale. Elle assure l'acquisition des données, l'exécution du modèle NN-MPC, la communication MQTT et la génération de la commande Slow PWM [14, 15].

Ce choix est justifié par sa compatibilité avec Python et TensorFlow Lite, sa connectivité réseau intégrée et sa capacité à exécuter un modèle de réseau de neurones léger en temps réel.

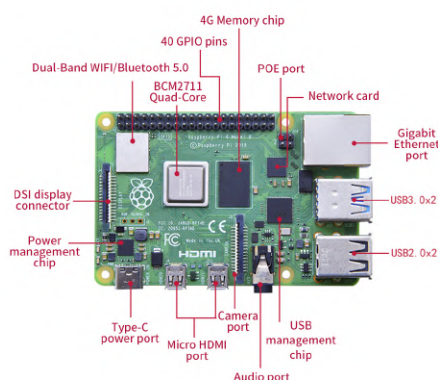


FIGURE 4.1 – Carte Raspberry Pi 4 utilisée comme unité de traitement embarquée.

TABLE 4.1 – Caractéristiques principales de la Raspberry Pi 4

Caractéristique	Valeur
Processeur	Quad-core ARM Cortex-A72 (1.5 GHz)
RAM	4 GB
Stockage	Carte microSD
Connectivité	Wi-Fi, Ethernet, Bluetooth
GPIO	40 broches
Système d'exploitation	Raspberry Pi OS / Linux

TABLE 4.2 – Configuration des broches GPIO utilisées

Composant	Pin GPIO	Fonction
Capteur DHT22	GPIO4	Mesure de la température et de l'humidité
Relais de la lampe	GPIO18	Commande du chauffage par PWM logiciel
Alimentation capteur	5V	Alimentation du capteur DHT22
Masse commune	GND	Référence électrique commune

4.2.2 Capteur de température et d'humidité DHT22

Le capteur DHT22 est utilisé pour mesurer la température et l'humidité à l'intérieur de l'enceinte thermique. Ces grandeurs constituent les informations principales exploitées pour la supervision du procédé et pour l'alimentation du contrôleur embarqué [16].

Le DHT22 a été retenu pour sa simplicité d'intégration avec la Raspberry Pi, son interface numérique et sa précision suffisante pour une maquette thermique expérimentale.



FIGURE 4.2 – Capteur DHT22 utilisé pour la mesure de la température et de l’humidité.

TABLE 4.3 – Caractéristiques principales du capteur DHT22

Caractéristique	Valeur
Plage de température	−40 °C à 80 °C
Précision température	±0,5 °C
Plage d’humidité	0% à 100%
Précision humidité	±2%
Type de signal	Numérique
Temps de réponse	Environ 2 s

Gestion de la cadence de mesure et tolérance de réutilisation : Le capteur DHT22 présente un temps de réponse d’environ 2 s, alors que la boucle de commande embarquée est exécutée avec une période de 1 s. Cette différence implique qu’une nouvelle mesure exploitable n’est pas nécessairement disponible à chaque itération de la boucle. Dans ce cas, le programme conserve temporairement les dernières valeurs valides de température et d’humidité afin de maintenir l’exécution du contrôleur NN-MPC sans interruption.

Dans le fonctionnement nominal du système, cette réutilisation concerne principalement **un cycle intermédiaire**, soit environ 1 s avant l’arrivée d’une nouvelle mesure exploitable. Cette tolérance reste cohérente avec la dynamique lente du procédé thermique étudié.

Afin d’évaluer l’ordre de grandeur de l’écart possible pendant ce délai, on s’appuie sur le modèle thermique identifié précédemment :

$$441 \dot{T}(t) + (T(t) - 21,1) = 0,355 u(t).$$

Dans l’hypothèse conservatrice d’une commande maximale $u(t) = 100\%$ et d’une température proche de l’ambiante, la vitesse de variation thermique maximale estimée est :

$$\dot{T}_{\max} \approx \frac{0,355 \times 100}{441} \approx 0,081 \text{ °C/s}.$$

Ainsi, si la dernière mesure valide est réutilisée pendant un cycle supplémentaire de

1 s, l'évolution non observée de la température reste de l'ordre de :

$$\Delta T_{1\text{ cycle}} \leq 0,08 \text{ }^{\circ}\text{C}$$

Cette variation reste faible devant la précision nominale du DHT22, donnée à $\pm 0,5 \text{ }^{\circ}\text{C}$. L'impact attendu sur la commande demeure donc limité dans le cas nominal d'un unique cycle de réutilisation.

Limite de tolérance retenue : Dans ce travail, la conservation d'une ancienne mesure est considérée comme acceptable lorsqu'elle reste limitée à **un seul cycle intermédiaire de 1 s**. Cette situation correspond au fonctionnement attendu compte tenu du temps de réponse du DHT22. Au-delà de ce délai, l'absence de nouvelle mesure exploitable devient non nominale : la valeur conservée peut s'éloigner progressivement de la température réelle et rendre la commande moins représentative de l'état instantané du procédé.

Ainsi, cette stratégie permet de préserver la continuité de la boucle de régulation sans bloquer le contrôleur à chaque cycle, tout en précisant clairement la limite acceptable de réutilisation des mesures.

TABLE 4.4 – Connexion du capteur DHT22

Broche DHT22	Connexion	Fonction
VCC	5V	Alimentation
DATA	GPIO4	Transmission des données
GND	GND	Masse commune

4.2.3 Relais statique SSR

Le relais statique SSR assure l'interface entre la Raspberry Pi et la lampe chauffante. Il permet de commander la puissance appliquée à la charge tout en isolant la partie commande basse tension de la partie puissance [17].

Ce composant est adapté à ce projet car il permet une commutation silencieuse, fiable et sans pièces mécaniques. Il est commandé par un signal Slow PWM généré par la Raspberry Pi.



FIGURE 4.3 – Relais statique utilisé pour la commande de la lampe chauffante.

TABLE 4.5 – Caractéristiques principales du relais SSR

Caractéristique	Valeur
Type	Relais statique
Tension de commande	3–32 V DC
Tension de sortie	24–380 V AC
Courant maximal	25 A selon le modèle
Type de commutation	Zero-crossing
Isolation	Optocoupleur intégré

TABLE 4.6 – Connexion du relais SSR

Broche SSR	Connexion	Fonction
Entrée +	GPIO18	Signal de commande PWM
Entrée -	GND	Référence électrique
Sortie AC	Lampe chauffante	Commutation de puissance

4.2.4 Lampe incandescente

La lampe incandescente est utilisée comme source de chaleur. Elle joue le rôle d'actionneur thermique du système. Sa puissance est modulée indirectement à travers le relais SSR, selon la commande calculée par le contrôleur NN-MPC.

Ce choix est motivé par la simplicité de mise en œuvre, le faible coût et la capacité de la lampe à produire rapidement de la chaleur dans une enceinte fermée.



FIGURE 4.4 – Lampe incandescente utilisée comme élément chauffant.

TABLE 4.7 – Caractéristiques principales de la lampe incandescente

Caractéristique	Valeur
Type	Lampe incandescente
Rôle	Source de chaleur
Tension d'alimentation	220–240 V AC
Puissance nominale	75 W
Commande	Via relais SSR
Fonction dans le système	Chauffage de l'enceinte thermique

4.2.5 Enceinte thermique

L'enceinte thermique constitue le procédé à contrôler. Elle est réalisée à partir d'une boîte en bois de dimensions approximatives 25 cm × 15 cm × 15 cm. Cette enceinte permet de créer un environnement fermé dans lequel la température peut être mesurée, régulée et analysée.

Elle contient la lampe incandescente, le capteur DHT22 et le relais SSR. Une face transparente permet de fermer l'enceinte tout en facilitant l'observation du comportement thermique.

L'utilisation du bois permet de limiter les échanges thermiques directs avec l'environnement extérieur et de reproduire un système thermique à forte inertie.

4.2.6 Montage expérimental

La Fig. 4.5 présente le montage expérimental réel du système de régulation thermique.



FIGURE 4.5 – Montage expérimental réel du système de régulation thermique.

Ce montage permet de valider le fonctionnement de la boucle de contrôle en conditions réelles, avec les contraintes physiques liées au capteur, à l'actionneur et à l'inertie thermique de l'enceinte.

4.3 Architecture globale et chaîne fonctionnelle

Le système développé est organisé autour de deux couches complémentaires : une couche de contrôle locale et une couche IoT de supervision.

La couche locale assure le fonctionnement temps réel du système. Elle est exécutée sur la Raspberry Pi et regroupe l'acquisition des mesures, l'exécution du modèle NN-MPC et la génération de la commande thermique.

La couche IoT assure la communication, le monitoring, le stockage et l'analyse des données expérimentales. Elle repose sur MQTT, Node-RED, InfluxDB et Grafana.

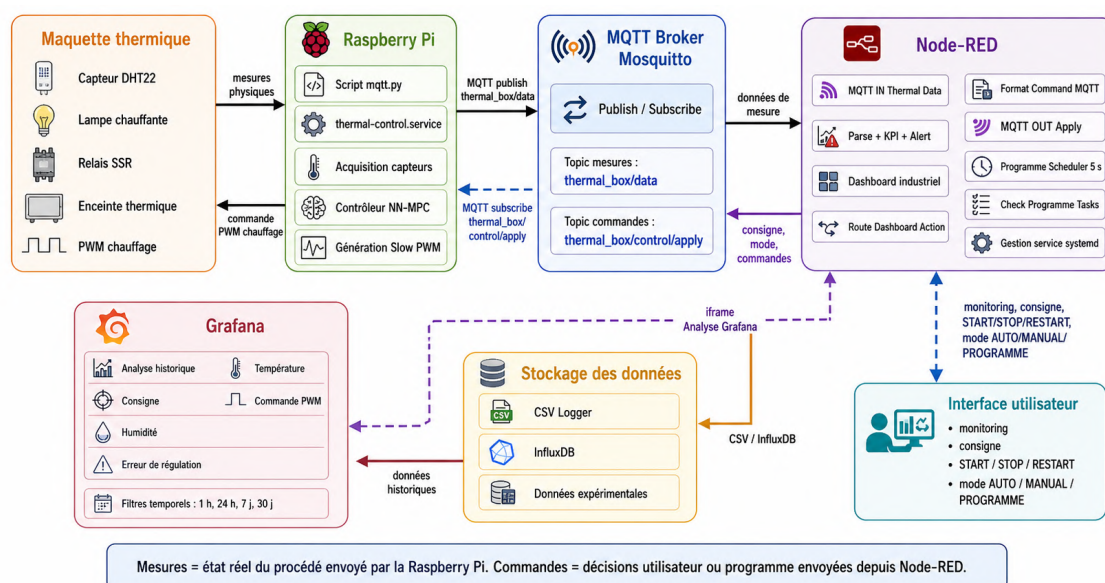


FIGURE 4.6 – Architecture globale du système de supervision et de contrôle thermique.

Cette séparation entre contrôle local et supervision distante constitue un choix impor-

tant. Elle permet de garantir que la régulation thermique reste fonctionnelle même en cas de problème au niveau de l'interface de supervision.

4.3.1 Chaîne fonctionnelle du système

Le fonctionnement global du système peut être résumé par la chaîne suivante :

Mesure → Traitement → Décision → Commande → Supervision

Le capteur DHT22 fournit les mesures physiques. La Raspberry Pi traite ces données et exécute le modèle NN-MPC afin de calculer la commande thermique. Cette commande est convertie en signal Slow PWM et appliquée à la lampe chauffante via le relais SSR. En parallèle, les données sont publiées via MQTT vers Node-RED pour la supervision, le stockage et l'analyse historique.

4.3.2 Synthèse des choix techniques

TABLE 4.8 – Synthèse des choix techniques du système

Élément	Choix retenu	Justification
Unité de traitement	Raspberry Pi 4	Exécution locale du modèle NN-MPC et connectivité réseau intégrée.
Capteur	DHT22	Mesure simultanée température/humidité avec intégration simple.
Actionneur	Lampe incandescente + SSR	Source thermique simple et commande fiable par Slow PWM.
Communication	MQTT / Mosquitto	Protocole léger adapté à la supervision IoT.
Supervision	Node-RED Dashboard	Interface temps réel, contrôle utilisateur et gestion du service.
Stockage	CSV + InfluxDB	Exploitation locale des essais et analyse temporelle historique.
Analyse	Grafana via iframe	Visualisation avancée intégrée dans l'interface Node-RED.

4.4 Implémentation logicielle embarquée

L'implémentation logicielle embarquée repose sur un programme Python exécuté localement sur la Raspberry Pi. Ce programme assure le fonctionnement autonome de la boucle de régulation thermique.

Il réalise l'acquisition des mesures, l'exécution du modèle NN-MPC, la génération du Slow PWM, la publication MQTT et la réception des consignes envoyées par Node-RED.

4.4.1 Environnement de développement

Le développement du modèle de contrôle a été réalisé sous **Google Colab**, qui offre un environnement Python adapté au prototypage rapide, à la simulation et à l'entraînement de modèles de réseaux de neurones[18].

Le langage **Python** est utilisé pour l'ensemble du développement en raison de sa simplicité et de sa compatibilité avec les bibliothèques scientifiques. Les bibliothèques **NumPy**, **Pandas** et **Matplotlib** sont utilisées pour le traitement des données et la visualisation des résultats.

La bibliothèque **GEKKO** est utilisée pour la simulation du contrôleur MPC et la génération des commandes optimales servant de référence pour l'apprentissage du modèle NN-MPC. Ensuite, **TensorFlow/Keras** est utilisé pour la conception et l'entraînement du réseau de neurones [19, 20, 21].

Après l'entraînement, le modèle NN-MPC est converti en format **TensorFlow Lite** afin d'être déployé efficacement sur la Raspberry Pi.

4.4.2 Déploiement du modèle NN-MPC sur Raspberry Pi

Après l'entraînement, le modèle NN-MPC est exporté vers un format léger compatible avec les plateformes embarquées. TensorFlow Lite permet de réduire la taille du modèle et d'améliorer le temps d'inférence.

Le modèle embarqué reçoit comme entrées la température mesurée, la consigne, l'humidité et les valeurs précédentes de température. Il fournit en sortie une commande thermique sous forme de pourcentage.

4.4.3 Boucle de contrôle embarquée

La boucle de contrôle est exécutée périodiquement sur la Raspberry Pi. Chaque itération comprend les étapes suivantes :

- acquisition de la température et de l'humidité ;
- prétraitement et normalisation des données ;
- exécution du modèle NN-MPC ;
- calcul de la commande thermique ;
- génération du signal Slow PWM ;
- publication des données via MQTT.

Dans l'implémentation embarquée, l'acquisition capteur est associée à une vérification de validité des mesures. Si une nouvelle mesure du DHT22 est exploitable, elle est utilisée

directement. En cas de lecture non valide ou d'absence de nouvelle valeur exploitable, le programme conserve temporairement les dernières mesures valides avant d'exécuter l'inférence du modèle NN-MPC.

Comme précisé dans la section consacrée au capteur DHT22, cette réutilisation est considérée comme acceptable lorsqu'elle reste limitée à **un seul cycle intermédiaire de 1 s**. Cette tolérance permet de préserver la continuité de la boucle de régulation sans bloquer le contrôleur, tout en tenant compte du temps de réponse du capteur.

À l'issue de l'inférence, la sortie normalisée du modèle TensorFlow Lite est reconvertie en pourcentage de commande, conformément à la relation définie précédemment, avant la génération du signal Slow PWM.

Cette organisation assure un fonctionnement périodique et cohérent de la commande embarquée, depuis l'acquisition des mesures jusqu'à l'application de la puissance de chauffage et à la publication des données vers la supervision IoT.

4.4.4 Génération du signal de commande

La commande calculée par le modèle est appliquée sous forme d'un signal Slow PWM. La commande en pourcentage est directement utilisée comme rapport cyclique du signal.

Pour une période fixe $T = 1\text{ s}$, une commande de 25% signifie que la lampe est activée pendant 0.25 s et désactivée pendant 0.75 s. De la même manière, une commande de 50% correspond à 0.5 s ON et 0.5 s OFF.

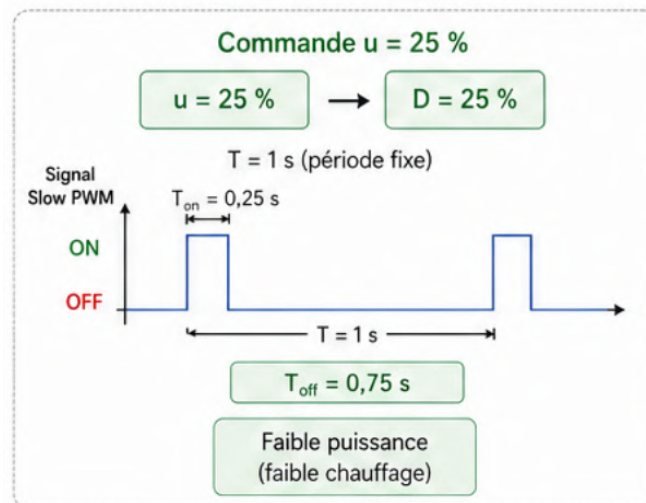


FIGURE 4.7 – Conversion de la commande en pourcentage en signal Slow PWM.

Le Slow PWM est adapté aux systèmes thermiques à forte inertie, car il permet de moduler la puissance moyenne appliquée à la lampe tout en limitant les commutations rapides du relais SSR.

4.5 Communication MQTT et structure des messages

Le protocole MQTT est utilisé pour assurer l'échange de données entre la Raspberry Pi et Node-RED. La Raspberry Pi publie les mesures du système, tandis que Node-RED envoie les commandes de régulation et de supervision [23].

MQTT a été choisi pour sa légèreté, sa simplicité d'intégration et son efficacité dans les applications IoT.

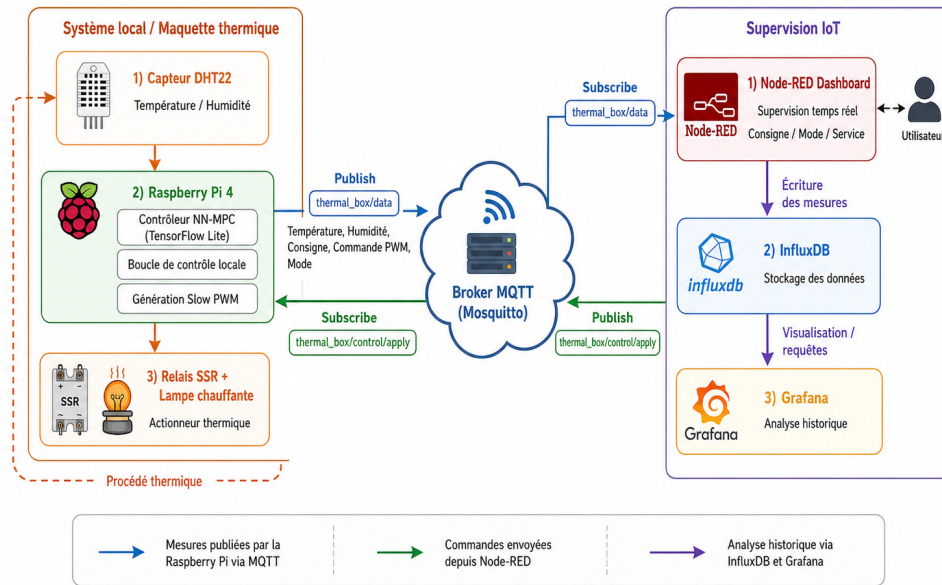


FIGURE 4.8 – Architecture MQTT reliant la Raspberry Pi, le broker Mosquitto et les outils de supervision.

4.5.1 Topics MQTT utilisés

Deux topics principaux sont utilisés afin de séparer clairement les mesures et les commandes[24].

TABLE 4.9 – Topics MQTT utilisés dans le système

Topic MQTT	Sens	Rôle
<code>thermal_box/data</code>	Raspberry Pi → Node-RED	Envoi des mesures expérimentales : température, humidité, commande PWM et état du système.
<code>thermal_box/control/apply</code>	Node-RED → Raspberry Pi	Envoi de la consigne, du mode de fonctionnement et des commandes utilisateur.

Cette organisation permet d'éviter les conflits entre l'état réel du procédé et les décisions utilisateur. Les mesures décrivent ce qui se passe réellement dans la maquette, tandis que les commandes représentent les actions demandées depuis l'interface Node-RED.

4.5.2 Structure du message de mesure

Les messages publiés par la Raspberry Pi sont structurés au format JSON afin de faciliter leur traitement dans Node-RED.

TABLE 4.10 – Structure du message de mesure publié par la Raspberry Pi

Champ	Type	Description
temperature	float	Température mesurée dans l'enceinte
humidity	float	Humidité mesurée par le capteur DHT22
command	float	Commande PWM calculée
pwm_on	float	Durée ON du signal Slow PWM
pwm_off	float	Durée OFF du signal Slow PWM
controller	string	Contrôleur actif
sensor	string	État du capteur
alert	string	État d'alerte du système
timestamp	string	Date et heure d'émission

4.5.3 Structure du message de commande

Les commandes envoyées depuis Node-RED vers la Raspberry Pi sont également envoyées au format JSON.

TABLE 4.11 – Structure du message de commande envoyé par Node-RED

Champ	Type	Description
mode	string	Mode AUTO, MANUAL ou PROGRAMME
setpoint	float	Consigne de température
manual_power	float	Commande imposée en mode manuel
action	string	Action demandée par l'utilisateur
program_enabled	boolean	Activation du mode programmé
tasks	array	Liste des tâches programmées
source	string	Origine de la commande

4.6 Supervision et pilotage avec Node-RED

Node-RED constitue la couche centrale de supervision et de pilotage du système. Il reçoit les mesures publiées par la Raspberry Pi, les traite à travers des fonctions dédiées, puis les affiche dans le dashboard.

Il permet également d'envoyer des commandes vers la Raspberry Pi, de gérer le service système, d'activer le mode programmé et de rediriger les données vers les outils de stockage et d'analyse.

4.6.1 Logique interne du flow Node-RED

Le flow Node-RED est organisé comme une chaîne de traitement. Les données reçues via MQTT sont d'abord décodées, puis normalisées sous un format unique exploitable par le dashboard. Ensuite, les variables principales sont extraites pour alimenter les cartes KPI, les alertes, le stockage CSV et l'envoi vers InfluxDB.

Les actions utilisateur suivent une chaîne séparée. Les commandes issues du dashboard sont routées selon leur type : commandes de régulation, commandes système ou commandes liées au mode programmé. Cette séparation améliore la lisibilité du flow et limite les risques de conflit entre les mesures reçues et les commandes envoyées.

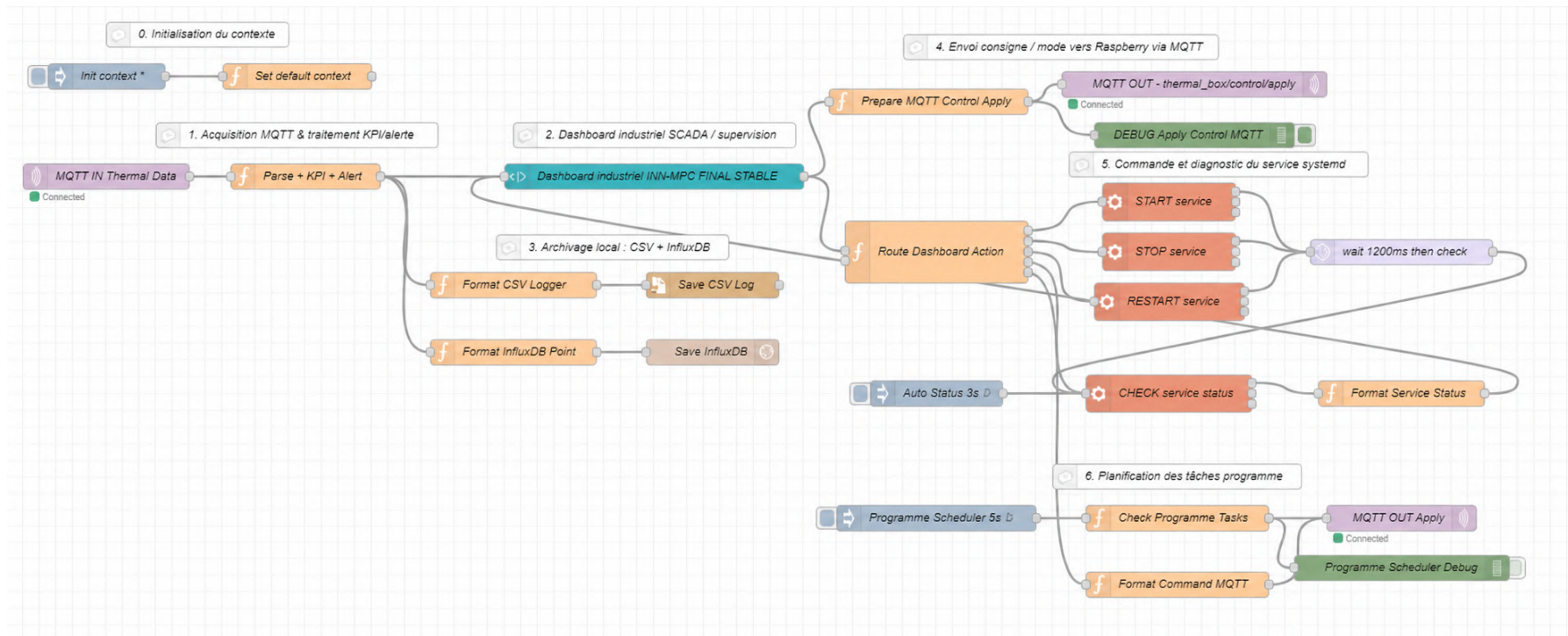


FIGURE 4.9 – Flow Node-RED développé pour la supervision, le pilotage et la gestion du service.

Le flow Node-RED contient plusieurs nodes élémentaires. Afin de rendre l'analyse plus claire, ces nodes sont regroupés dans le tableau suivant sous forme de blocs fonctionnels. Chaque bloc représente une partie logique du traitement, indépendamment du nombre exact de nodes utilisés pour son implémentation.

TABLE 4.12 – Organisation logique du flow Node-RED

Bloc fonctionnel	Rôle
Initialisation du contexte	Définition des valeurs par défaut utilisées par le dashboard, telles que la consigne, le mode de fonctionnement et l'état initial du système.
MQTT IN Thermal Data	Réception des mesures publiées par la Raspberry Pi sur le topic <code>thermal_box/data</code> .
Parse + KPI + Alert	Décodage du message JSON, extraction des grandeurs utiles, calcul de l'erreur de régulation et génération de l'état d'alerte.
Dashboard Industriel	Affichage temps réel des variables principales et interaction avec l'utilisateur à travers les modes AUTO, MANUAL et PROGRAMME.
CSV Logger	Formatage et enregistrement local des données expérimentales dans un fichier CSV pour l'exploitation sous Python.
Format InfluxDB	Préparation des données temporelles avant leur insertion dans InfluxDB pour l'analyse historique avec Grafana.
Route Dashboard Action	Routage des actions utilisateur selon leur nature : commande de régulation, commande système ou programmation temporelle.
Format Command MQTT	Construction du payload de commande contenant la consigne, le mode et la puissance manuelle avant l'envoi vers la Raspberry Pi.
MQTT OUT Apply	Envoi des commandes vers la Raspberry Pi via le topic <code>thermal_box/control/apply</code> .
Service Control	Exécution des commandes START , STOP et RESTART du service <code>thermal-control.service</code> .
Service Status	Lecture de l'état réel du service à l'aide de <code>systemctl is-active</code> et affichage du résultat dans le dashboard.
Programme Scheduler	Vérification périodique des tâches programmées et application automatique de la consigne lorsque l'intervalle temporel devient actif.

4.6.2 Séparation entre mesures et commandes

Une séparation stricte est appliquée entre les mesures provenant de la Raspberry Pi et les commandes définies par l'utilisateur. Les données reçues sur le topic `thermal_box/data` représentent l'état réel du procédé, tandis que les consignes et modes de fonctionnement sont stockés côté Node-RED puis envoyés via `thermal_box/control/apply`.

Ce choix évite qu'une ancienne consigne renvoyée par la Raspberry Pi écrase la consigne choisie dans le dashboard. La consigne affichée dans l'interface provient donc uniquement du contexte Node-RED. Cette organisation améliore la cohérence de l'affichage et la fiabilité de l'interaction utilisateur.

4.6.3 Développement du dashboard Node-RED

Le dashboard Node-RED constitue l'interface homme-machine du système. Il permet de visualiser les grandeurs principales en temps réel et d'envoyer les commandes vers la Raspberry Pi.

L'interface est organisée autour de plusieurs zones : monitoring temps réel, configuration du système, alertes, gestion du service, mode programmé et analyse Grafana.

Trois modes de fonctionnement sont intégrés :

- **Mode automatique** : l'utilisateur définit uniquement la consigne, tandis que la commande PWM est calculée automatiquement par le contrôleur NN-MPC ;
- **Mode manuel** : l'utilisateur peut imposer directement une commande PWM pour les tests et diagnostics ;
- **Mode programmé** : l'utilisateur définit des tâches temporelles contenant une heure de début, une durée et une consigne.

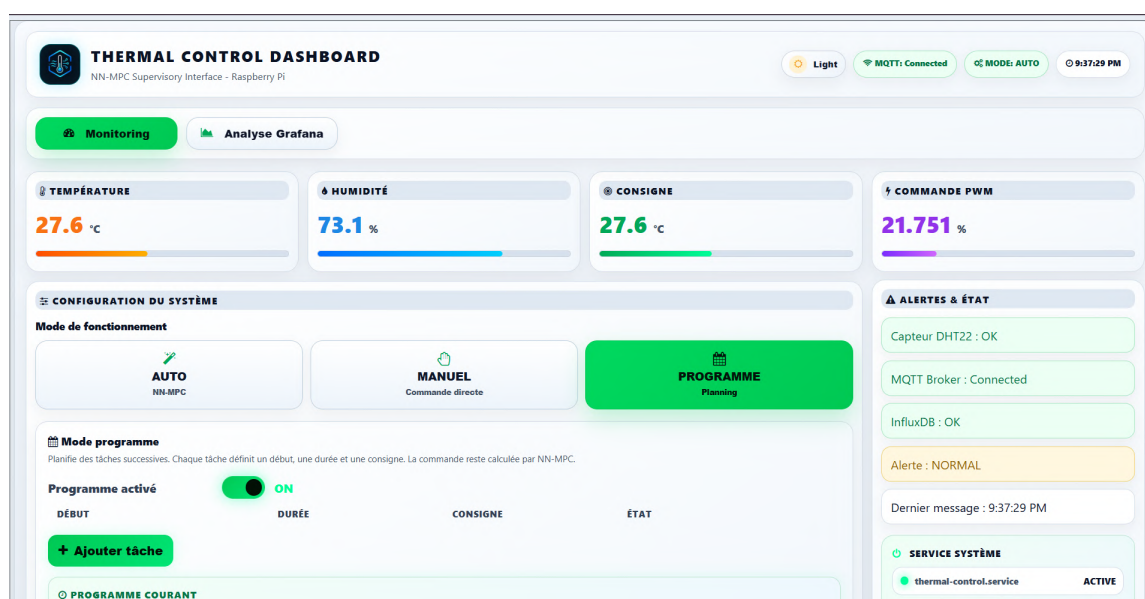


FIGURE 4.10 – Dashboard Node-RED principal développé pour la supervision temps réel et le pilotage du système thermique.

Cette interface facilite l'exploitation expérimentale de la maquette car elle regroupe dans un seul environnement le monitoring, le pilotage, la gestion du service et l'accès aux courbes historiques.

4.6.4 Gestion du service système

Le script Python embarqué est exécuté sous forme d'un service systemd nommé `thermal-control.service` [28]. Cette solution permet de démarrer, arrêter, redémarrer et vérifier l'état du programme de contrôle directement depuis le dashboard Node-RED.



FIGURE 4.11 – Module de gestion du service `thermal-control.service` intégré dans le dashboard Node-RED.

TABLE 4.13 – Commandes système intégrées dans le dashboard Node-RED

Action	Commande exécutée
Démarrage du service	<code>systemctl start thermal-control.service</code>
Arrêt du service	<code>systemctl stop thermal-control.service</code>
Redémarrage du service	<code>systemctl restart thermal-control.service</code>
Vérification de l'état	<code>systemctl is-active thermal-control.service</code>

L'état affiché dans l'interface provient directement du système d'exploitation, ce qui permet d'obtenir une information fiable sur l'exécution réelle du programme.

4.6.5 Mode programmé et ordonnanceur temporel

Le mode programmé permet de planifier automatiquement l'application des consignes. L'utilisateur définit des tâches contenant une heure de début, une durée et une consigne.

Ces tâches sont stockées dans le contexte Node-RED. Un ordonnanceur exécuté toutes les cinq secondes vérifie l'heure actuelle et active la tâche correspondante lorsque son intervalle temporel est atteint.

CONFIGURATION DU SYSTÈME

Mode de fonctionnement

AUTO
NN-MPC

MANUEL
Commande directe

PROGRAMME
Planning

Mode programme
Planifie des tâches successives. Chaque tâche définit un début, une durée et une consigne. La commande reste calculée par NN-MPC.

Programme activé ☒ **ON**

DÉBUT	DURÉE	CONSIGNE	ÉTAT
09:39 PM	1 min	27 °C	PRÊTE - durée 01:00

+ Ajouter tâche

PROGRAMME COURANT

Programme prêt

En mode PROGRAMME, l'interface modifie uniquement la consigne selon les tâches planifiées. La commande PWM reste calculée automatiquement par le contrôleur NN-MPC.

APPLIQUER envoie la consigne et les tâches programme au flow Node-RED.

APPLIQUER

FIGURE 4.12 – Interface du mode programmé permettant de définir l'heure de début, la durée et la consigne.

Il est important de noter que le mode programmé ne remplace pas le contrôleur NN-MPC. Il modifie uniquement la consigne de température. La commande PWM reste calculée automatiquement par le contrôleur embarqué.

4.7 Stockage et analyse historique des données

Les données expérimentales sont stockées afin de permettre l'analyse, la génération des figures de résultats et le suivi historique du comportement thermique.

Deux formes de stockage sont utilisées : un fichier CSV pour l'exploitation locale des données et une base InfluxDB pour la visualisation temporelle avec Grafana.

TABLE 4.14 – Grandeurs enregistrées pendant les essais

Grandeur	Description
Temps	Date et heure de la mesure
Température	Température interne de l'enceinte thermique
Humidité	Humidité mesurée par le capteur DHT22
Consigne	Température de référence définie par l'utilisateur
Commande PWM	Commande calculée par le contrôleur NN-MPC
Erreur	Différence entre la consigne et la température mesurée
Alerte	État du système : normal, à surveiller ou critique

Le stockage CSV permet d'exploiter les données sous Python, tandis qu'InfluxDB permet une visualisation temporelle plus avancée avec Grafana.

4.7.1 Analyse historique avec Grafana

Grafana est utilisé pour compléter le monitoring temps réel par une analyse historique des données. Les courbes Grafana permettent de suivre l'évolution de la température, de la consigne, de la commande PWM, de l'humidité et de l'erreur de régulation.

Le dashboard Grafana est intégré dans l'interface Node-RED à travers une iframe. Cette intégration permet de consulter les courbes expérimentales directement depuis la même interface de supervision, sans ouvrir une page externe.

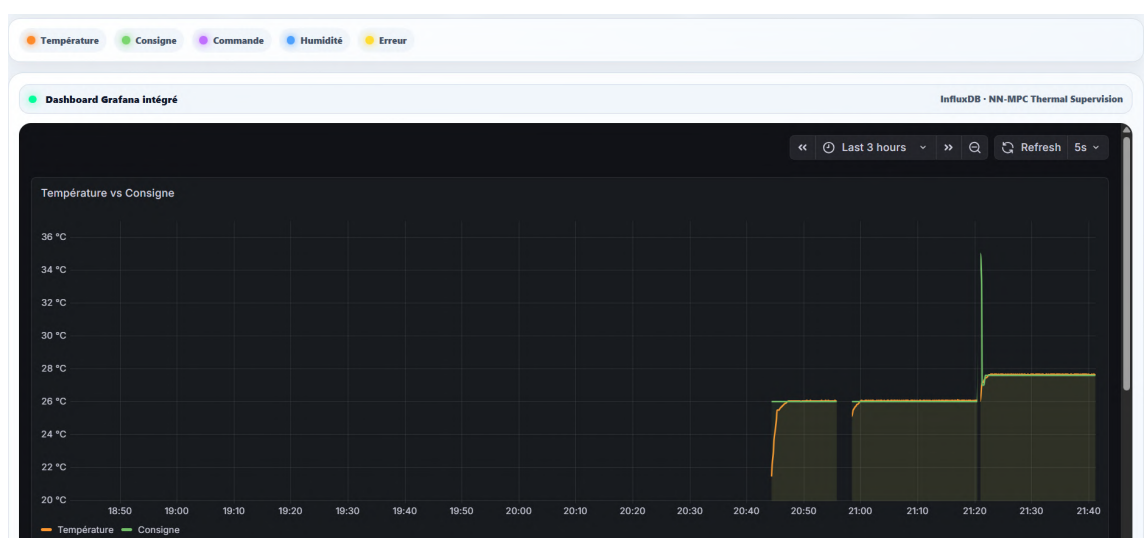


FIGURE 4.13 – Dashboard Grafana intégré dans Node-RED via iframe pour l'analyse historique des données expérimentales.

Cette partie est importante pour la validation expérimentale, car elle permet d'observer le comportement dynamique du système sur différentes fenêtres temporelles.

4.8 Robustesse et gestion des défauts

La conception du système prend en compte plusieurs situations pouvant perturber le fonctionnement expérimental : perte temporaire de communication MQTT, arrêt du service embarqué, absence de données capteur ou incohérence d’affichage.

Pour limiter ces problèmes, plusieurs mécanismes sont mis en place :

- l’état du service `thermal-control.service` est vérifié directement à partir du système d’exploitation ;
- le dernier payload complet reçu est conservé afin d’éviter la remise à zéro des KPI lors des messages incomplets ;
- les alertes sont calculées à partir de l’écart entre la consigne et la température mesurée ;
- la boucle de contrôle locale reste fonctionnelle même en cas de problème au niveau du dashboard.

La Fig. 4.14 illustre volontairement un cas critique utilisé pour vérifier le fonctionnement du module d’alertes.

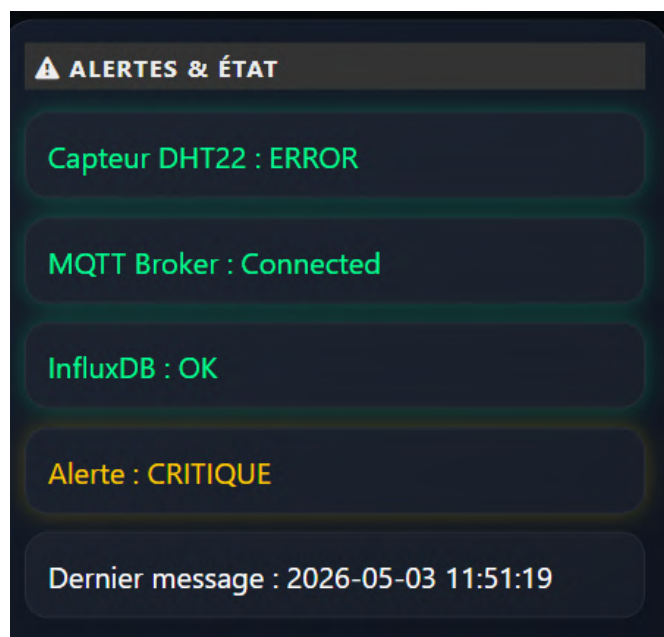


FIGURE 4.14 – Exemple d’état critique détecté par le module d’alertes et d’état du dashboard.

TABLE 4.15 – Mécanismes de robustesse intégrés

Problème traité	Solution mise en place
Ancienne consigne renvoyée par Raspberry Pi	Consigne stockée uniquement dans le contexte Node-RED
Message service incomplet	Fusion avec le dernier payload complet
Service inactif ou arrêté	Contrôle via <code>systemctl</code> depuis le dashboard
Mode programmé non déclenché	Scheduler périodique toutes les 5 secondes
Perte d’affichage Grafana	Monitoring temps réel maintenu dans Node-RED

Cette approche améliore la fiabilité du système et rend l’exploitation expérimentale plus sûre.

4.9 Précautions de sécurité

La lampe incandescente étant alimentée par le secteur, une attention particulière est accordée à la séparation entre la partie commande basse tension et la partie puissance.

Le relais SSR assure l’isolation entre la Raspberry Pi et la charge AC. Les connexions de puissance sont réalisées de manière à éviter tout contact direct avec les conducteurs sous tension.

Cette précaution est essentielle pour garantir la sécurité de l’expérimentation et protéger la plateforme embarquée.

4.10 Validation fonctionnelle du système

La validation fonctionnelle consiste à vérifier que chaque partie du système fonctionne correctement avant l’analyse détaillée des performances du contrôleur.

Les tests réalisés portent sur la communication MQTT, l’affichage temps réel dans Node-RED, l’envoi des consignes vers la Raspberry Pi, la gestion du service `systemd`, le mode programmé et l’intégration Grafana.

4.10.1 Scénario de test

Afin d’évaluer le fonctionnement global du système, des essais expérimentaux sont réalisés sur la maquette thermique en conditions réelles.

Le protocole consiste à appliquer une consigne de température, puis à observer l’évolution de la température, de la commande PWM et de l’état du système jusqu’à l’atteinte du régime permanent.

4.10.2 Vérification de la communication MQTT

La première vérification consiste à s'assurer que la Raspberry Pi publie correctement les données sur `thermal_box/data`. Les messages reçus doivent contenir les grandeurs nécessaires au monitoring.

La deuxième vérification consiste à envoyer une commande depuis Node-RED vers la Raspberry Pi via `thermal_box/control/apply`. La consigne modifiée depuis le dashboard doit être reçue et prise en compte par le script embarqué.

4.10.3 Vérification du dashboard Node-RED

Le dashboard est validé en vérifiant l'affichage en temps réel des mesures, la modification de la consigne, le changement de mode et l'actualisation de l'état du service.

Une attention particulière est portée à la cohérence de la consigne. Celle-ci est stockée dans le contexte Node-RED afin d'éviter les conflits avec des valeurs anciennes envoyées par la Raspberry Pi.

4.10.4 Vérification du mode programmé

Le mode programmé est vérifié en définissant une tâche avec une heure de début, une durée et une consigne. L'ordonnanceur Node-RED vérifie périodiquement l'heure actuelle et envoie automatiquement la consigne lorsque la tâche devient active.

Cette vérification confirme que le système peut appliquer des scénarios de fonctionnement sans intervention manuelle.

4.10.5 Vérification de l'intégration Grafana

L'intégration Grafana est validée en observant les courbes expérimentales sur différentes fenêtres temporelles. Les données affichées permettent de vérifier le suivi de la température, de la consigne, de la commande PWM et de l'humidité.

Cette étape confirme que les données enregistrées sont exploitables pour l'analyse des performances présentée dans le chapitre suivant.

4.11 Conclusion

Dans ce chapitre, nous avons présenté l'implémentation réelle du système de régulation thermique intelligent.

Nous avons détaillé l'architecture matérielle, la boucle de contrôle embarquée, le déploiement du modèle NN-MPC, la génération du signal Slow PWM ainsi que la couche IoT développée autour de MQTT, Node-RED, InfluxDB et Grafana.

La supervision Node-RED, la gestion du service `thermal-control.service`, le mode programmé et l'intégration Grafana montrent que le système développé dépasse une simple commande locale et constitue une plateforme complète de régulation, supervision et analyse.

La validation fonctionnelle réalisée confirme que les différentes parties du système communiquent correctement et que l'architecture proposée est prête pour l'évaluation expérimentale détaillée des performances du contrôleur NN-MPC.

Résultats et Discussion

Sommaire

- 5.1 Introduction..... 76
- 5.2 Génération du dataset d'apprentissage 76
- 5.3 Apprentissage du modèle NN-MPC 78
 - 5.3.1 Architecture du réseau de neurones 78
 - 5.3.2 Paramètres d'apprentissage 78
 - 5.3.3 Courbes d'apprentissage 79
 - 5.3.4 Prédiction de la commande MPC..... 80
 - 5.3.5 Taille du modèle et performance d'inférence..... 81
- 5.4 Validation du modèle thermique 81
- 5.5 Comparaison expérimentale des contrôleurs 83
 - 5.5.1 Comparaison des réponses en température 83
 - 5.5.2 Comparaison des commandes appliquées 84
 - 5.5.3 Synthèse quantitative des performances 84
- 5.6 Analyse de robustesse du contrôleur NN-MPC 85
 - 5.6.1 Suivi de consignes multiples 85
 - 5.6.2 Rejet de perturbations 86
- 5.7 Analyse énergétique 87
- 5.8 Résultats fonctionnels de la supervision IoT 89
 - 5.8.1 Dashboard de supervision temps réel 89
 - 5.8.2 Validation des modes de fonctionnement 90
 - 5.8.3 Gestion du service embarqué 92
 - 5.8.4 Analyse historique intégrée avec Grafana..... 93
 - 5.8.5 Diagnostic automatique et résumé expérimental 93
 - 5.8.6 Synthèse de la validation fonctionnelle 94
- 5.9 Discussion générale..... 94
- 5.10 Conclusion 95

5.1 Introduction

Ce chapitre présente les résultats obtenus lors du développement, de l'apprentissage et de la validation expérimentale du système de régulation thermique intelligent.

L'objectif principal est d'évaluer la capacité du contrôleur NN-MPC à approximer le comportement d'un contrôleur MPC de référence, puis à assurer une régulation thermique rapide, précise et stable sur une maquette réelle.

Les résultats sont présentés selon une progression logique : génération du dataset, apprentissage du modèle NN-MPC, validation du modèle thermique, comparaison expérimentale des contrôleurs PID, MPC et NN-MPC, analyse de robustesse, analyse énergétique et validation fonctionnelle de la supervision IoT.

5.2 Génération du dataset d'apprentissage

Le dataset utilisé pour l'apprentissage du modèle NN-MPC est généré à partir d'un contrôleur MPC simulé avec la bibliothèque **GEKKO**. Cette étape permet de produire une base de données représentative du comportement thermique du système pour plusieurs consignes de température.

Le modèle thermique identifié est utilisé sous la forme :

$$441\dot{y}(t) + (y(t) - 21.1) = 0.355u(t)$$

où $y(t)$ représente la température interne de l'enceinte thermique et $u(t)$ la commande de chauffage exprimée en pourcentage.

Dans cette phase, le MPC est utilisé comme un contrôleur de référence hors-ligne. Il permet de calculer des commandes optimales pour plusieurs conditions de fonctionnement. Ces commandes sont ensuite utilisées comme sorties cibles pour entraîner le réseau de neurones.

Les consignes utilisées pour la génération du dataset sont choisies aléatoirement dans la plage 25 °C à 35 °C. Cette variation permet d'exposer le modèle à plusieurs niveaux de fonctionnement thermique et d'améliorer sa capacité de généralisation.

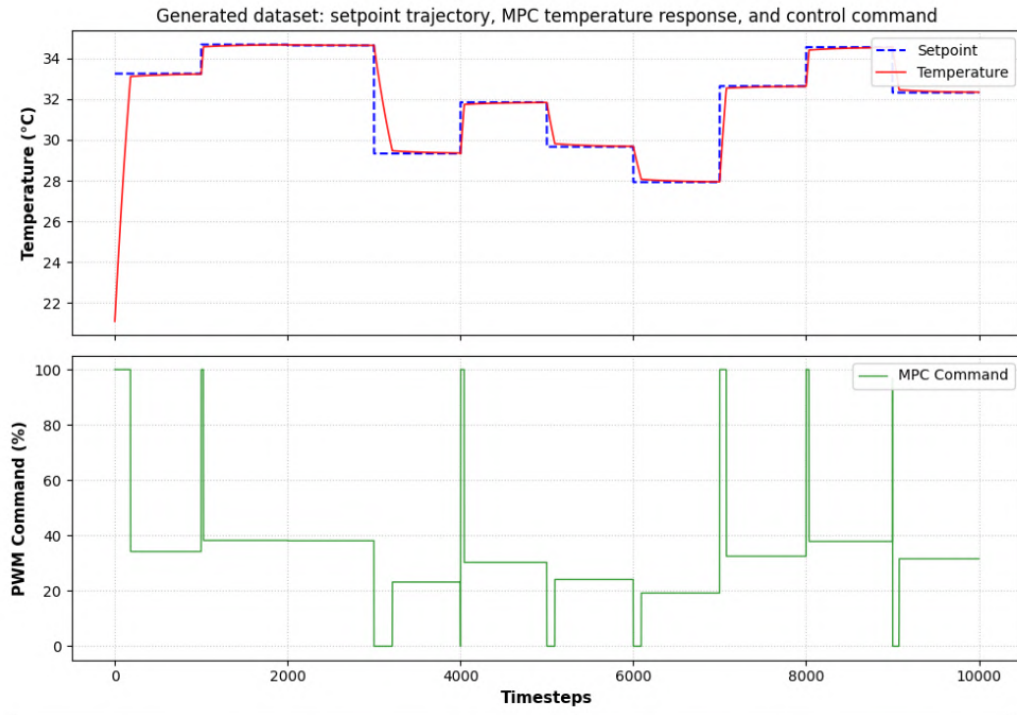


FIGURE 5.1 – Dataset généré : trajectoire de consigne, réponse thermique simulée et commande MPC optimale sur 10 000 pas de temps.

La Fig. 5.1 montre la trajectoire de consigne, la réponse thermique simulée et la commande MPC optimale. La diversité des consignes permet au modèle NN-MPC d'apprendre une loi de commande généralisable sur toute la plage de fonctionnement étudiée.

TABLE 5.1 – Caractéristiques du dataset d'apprentissage

Propriété	Valeur	Détails
Nombre total d'échantillons	10 000	Pas de simulation utilisés pour générer le dataset
Variables d'entrée	5	$y(t)$, y_r , h , $y(t-1)$, $y(t-2)$
Variable de sortie	1	Commande optimale $u^* \in [0, 100]\%$
Plage de consigne	25 °C à 35 °C	Consignes générées aléatoirement dans l'intervalle de fonctionnement
Période d'échantillonnage	1 s	Boucle de contrôle Raspberry Pi
Répartition des données	70% / 15% / 15%	Entraînement / validation / test

Cette base d'apprentissage constitue une étape essentielle du projet. Elle permet de transférer la stratégie de commande MPC vers un modèle de réseau de neurones plus léger et compatible avec une exécution embarquée sur Raspberry Pi.

5.3 Apprentissage du modèle NN-MPC

Le modèle NN-MPC est entraîné afin d'approximer la loi de commande fournie par le contrôleur MPC de référence. Le réseau de neurones reçoit comme entrées l'état thermique du système et la consigne, puis prédit directement la commande de chauffage à appliquer.

Cette approche permet de remplacer la résolution en ligne d'un problème d'optimisation MPC par une inférence rapide du réseau de neurones.

5.3.1 Architecture du réseau de neurones

L'architecture retenue est de type perceptron multicouche. Elle est volontairement légère afin de garantir une exécution rapide sur Raspberry Pi.

TABLE 5.2 – Architecture du réseau de neurones NN-MPC

Couche	Type	Activation	Neurones	Paramètres
Input	Dense	–	5	–
Hidden 1	Dense	ReLU	32	192
Dropout 1	Dropout	0.2	–	0
Hidden 2	Dense	ReLU	16	528
Dropout 2	Dropout	0.2	–	0
Hidden 3	Dense	ReLU	8	136
Output	Dense	Sigmoid	1	9
Total des paramètres entraînables				865

L'architecture comporte trois couches cachées de 32, 16 et 8 neurones. Les fonctions d'activation ReLU permettent d'apprendre les non-linéarités de la loi de commande. Les couches Dropout sont utilisées pour limiter le surapprentissage.

La couche de sortie utilise une activation sigmoïde afin de produire une commande normalisée, ensuite convertie en pourcentage de puissance. Le modèle contient seulement 865 paramètres entraînables, ce qui le rend adapté à une exécution embarquée.

5.3.2 Paramètres d'apprentissage

L'entraînement est réalisé avec la fonction de perte MSE, qui pénalise les écarts entre la commande MPC de référence et la commande prédite par le réseau de neurones.

TABLE 5.3 – Hyperparamètres d'apprentissage du modèle NN-MPC

Hyperparamètre	Valeur	Description
Optimiseur	Adam	Méthode adaptative basée sur le gradient
Taux d'apprentissage	0.001	Taux d'apprentissage initial
Fonction de perte	MSE	Erreur quadratique moyenne
Batch size	32	Apprentissage par mini-batch
Nombre maximal d'époques	200	Limite maximale de l'entraînement
Early stopping	Oui, patience = 15	Arrêt automatique pour éviter le surapprentissage
Méthode de validation	Ensemble de validation séparé	Suivi de la généralisation pendant l'entraînement
Train / Val / Test split	70% / 15% / 15%	Partitionnement du dataset

L'utilisation de l'early stopping permet d'arrêter automatiquement l'apprentissage lorsque l'erreur de validation ne s'améliore plus. Cela permet de conserver un modèle performant sans surapprentissage significatif.

5.3.3 Courbes d'apprentissage

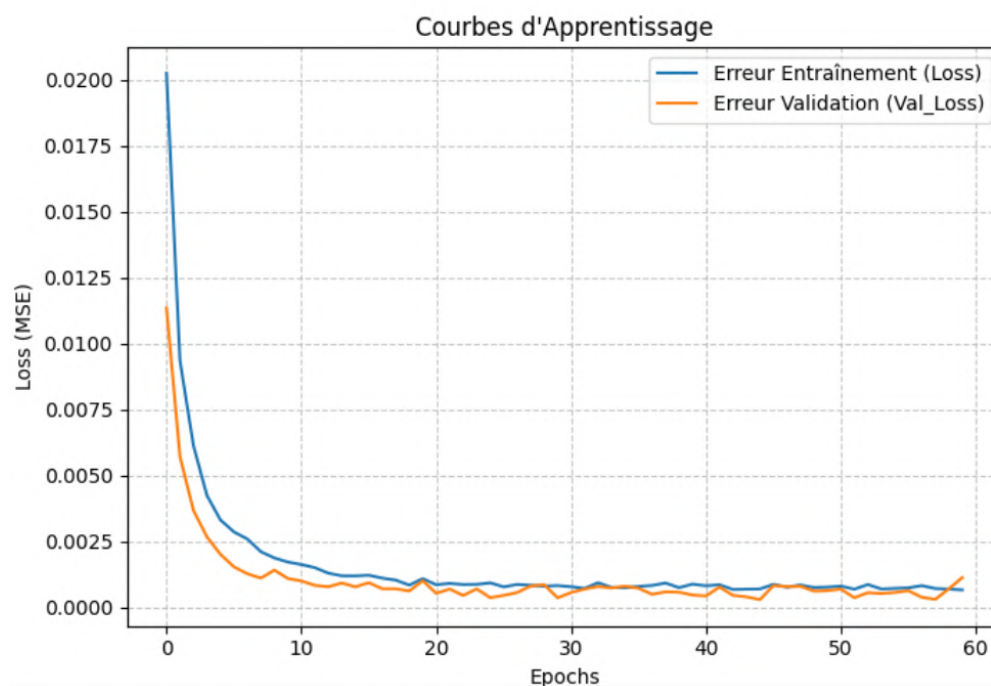


FIGURE 5.2 – Évolution des pertes d'apprentissage et de validation du modèle NN-MPC.

La Fig. 5.2 montre l'évolution de la fonction de perte au cours de l'entraînement. On observe une diminution rapide de l'erreur pendant les premières époques, suivie d'une phase de stabilisation.

La proximité entre les courbes d'apprentissage et de validation montre que le modèle présente une bonne capacité de généralisation. L'absence d'écart important entre les deux courbes indique que le surapprentissage reste limité.

5.3.4 Prédiction de la commande MPC

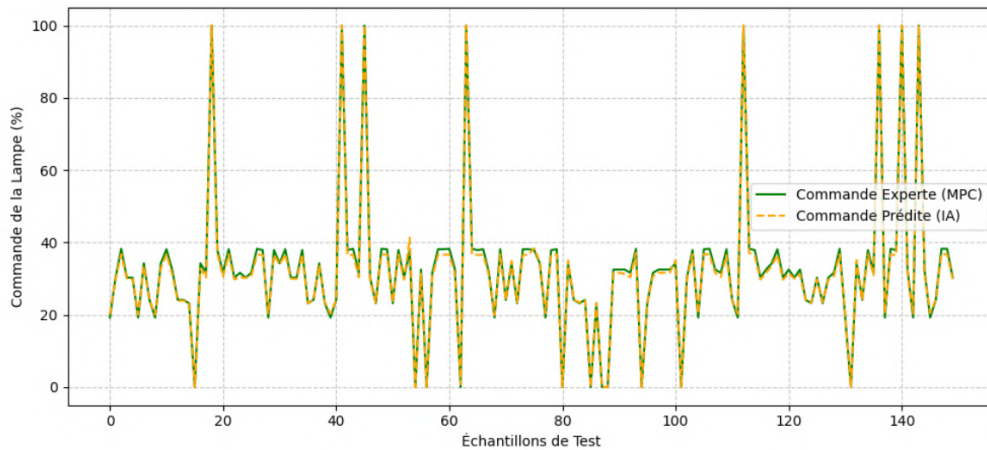


FIGURE 5.3 – Comparaison entre la commande MPC de référence et la commande prédite par le modèle NN-MPC.

La Fig. 5.3 compare les commandes MPC de référence avec les commandes prédites par le réseau de neurones sur l'ensemble de test.

La forte superposition entre les deux courbes confirme que le NN-MPC parvient à approximer correctement la loi de commande MPC. Ce résultat justifie l'utilisation du modèle entraîné comme alternative légère au MPC classique pour une exécution embarquée.

TABLE 5.4 – Performances de prédiction du modèle NN-MPC

Métrique	Valeur
Coefficient de détermination R^2	98.06%

Le score $R^2 = 98.06\%$ montre que le modèle explique correctement la majorité de la variance de la commande MPC de référence. La qualité de prédiction obtenue confirme l'efficacité de l'apprentissage supervisé.

5.3.5 Taille du modèle et performance d'inférence

Après l'entraînement, le modèle est converti au format TensorFlow Lite afin de réduire sa taille et d'améliorer son temps d'inférence sur Raspberry Pi.

TABLE 5.5 – Taille du modèle et performances d'inférence

Métrique	Keras (.h5)	TFLite (.tflite)
Taille du modèle	48.96 KB	5.78 KB
Temps d'inférence sur Raspberry Pi 4	~ 0.20 ms	≈ 0.015 ms
Score R^2	98.06%	98.06%
Utilisation RAM	$\sim 10\text{--}20$ MB	$\sim 2\text{--}5$ MB
Utilisation CPU sur Raspberry Pi 4	–	$\sim 1\text{--}3\%$

Le tableau 5.5 montre l'intérêt de la conversion vers TensorFlow Lite. La taille du modèle passe de 48.96 KB à 5.78 KB, tandis que le temps d'inférence diminue d'environ 0.20 ms à 0.015 ms.

Cette réduction est importante pour une application embarquée, car elle permet au contrôleur NN-MPC de fonctionner en temps réel avec une charge mémoire et CPU très faible. Le score R^2 reste identique après conversion, ce qui confirme que le passage vers TensorFlow Lite ne dégrade pas la qualité de prédiction.

5.4 Validation du modèle thermique

Avant de comparer les contrôleurs, il est nécessaire de valider le modèle thermique identifié. Cette validation consiste à comparer la réponse simulée du modèle avec la réponse mesurée expérimentalement sur la maquette, pour un échelon de commande fixé à $u_0 = 60\%$.

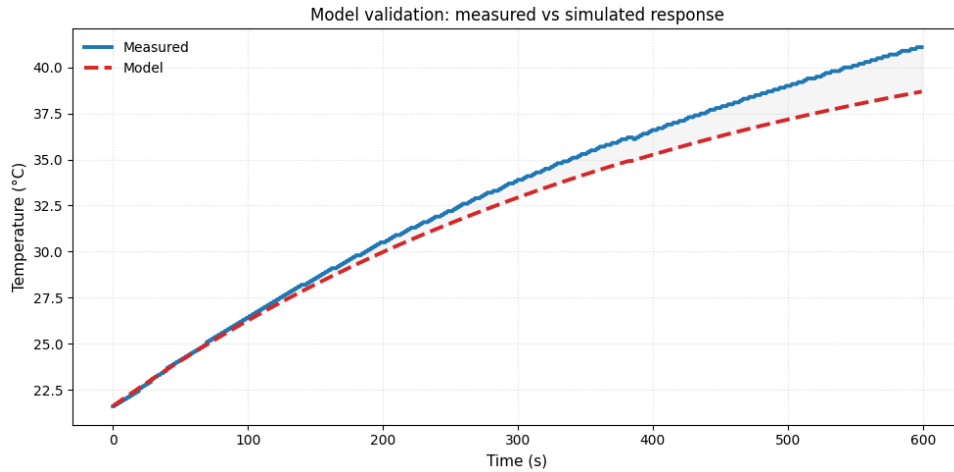


FIGURE 5.4 – Validation du modèle thermique : comparaison entre la réponse mesurée et la réponse simulée pour $u_0 = 60\%$.

Afin de compléter l’analyse graphique, la concordance entre la réponse mesurée et la réponse simulée est également quantifiée à l’aide de deux indicateurs numériques : le coefficient de détermination R^2 et la racine de l’erreur quadratique moyenne RMSE. Ces métriques sont calculées sur l’intervalle temporel représenté dans la Fig. 5.4, soit de 0 à 600 s.

TABLE 5.6 – Indicateurs quantitatifs de validation du modèle thermique

Métrique	Valeur
Coefficient de détermination R^2	95,22%
RMSE (°C)	1,22

La Fig. 5.4 montre que la réponse simulée suit correctement la tendance générale de la réponse expérimentale. Cette observation visuelle est confirmée par le coefficient de détermination $R^2 = 95,22\%$, qui indique que le modèle reproduit de manière satisfaisante l’évolution globale de la température mesurée.

Le RMSE de 1,22 °C met en évidence un écart mesurable entre la réponse simulée et la réponse expérimentale, principalement visible lorsque la température augmente sur un horizon plus long. Ce niveau d’erreur reste cohérent avec le caractère expérimental de la maquette et avec le choix d’un modèle thermique simplifié du premier ordre. Il peut notamment s’expliquer par les pertes thermiques réelles de l’enceinte, les approximations du modèle identifié ainsi que les incertitudes de mesure liées au capteur DHT22.

Ainsi, le modèle identifié reste suffisamment représentatif pour être utilisé dans la génération du dataset d’apprentissage et dans l’évaluation des stratégies de commande, tout en présentant les limites attendues d’un modèle thermique simplifié.

5.5 Comparaison expérimentale des contrôleurs

Conformément à la distinction méthodologique présentée à la section 3.5, le MPC utilisé dans cette comparaison correspond au contrôleur prédictif simplifié dédié à l'évaluation expérimentale.

Les performances des trois contrôleurs PID, MPC et NN-MPC sont ensuite évaluées sur la même maquette thermique. La consigne est fixée à 30 °C, avec une température initiale proche de 21.1 °C.

L'objectif de cette comparaison est d'évaluer la rapidité, le dépassement, la précision en régime permanent, la stabilité et l'énergie utile consommée par chaque stratégie de commande.

5.5.1 Comparaison des réponses en température

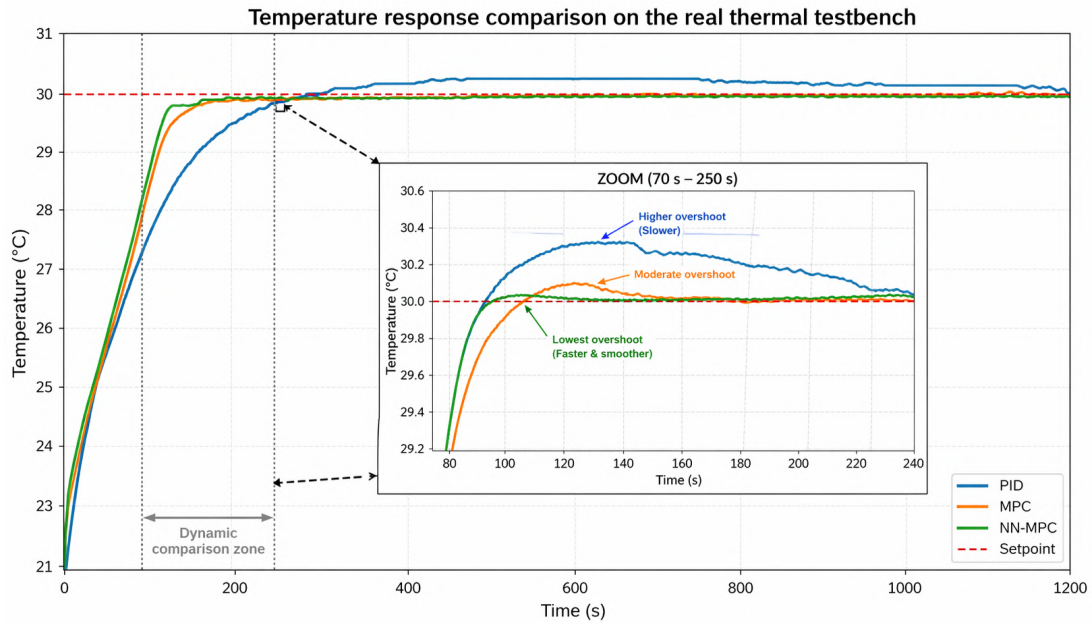


FIGURE 5.5 – Comparaison des réponses en température des contrôleurs PID, MPC et NN-MPC pour une consigne de 30 °C.

La Fig. 5.5 montre que le contrôleur NN-MPC atteint la consigne plus rapidement que le PID et légèrement plus rapidement que le MPC. Le dépassement reste faible, ce qui indique une bonne maîtrise de la dynamique thermique.

Le PID présente une réponse plus lente, en raison de la forte inertie du système thermique. Le MPC améliore la rapidité grâce à la prédiction du comportement futur, tandis que le NN-MPC reproduit cette stratégie avec une exécution plus légère.

5.5.2 Comparaison des commandes appliquées

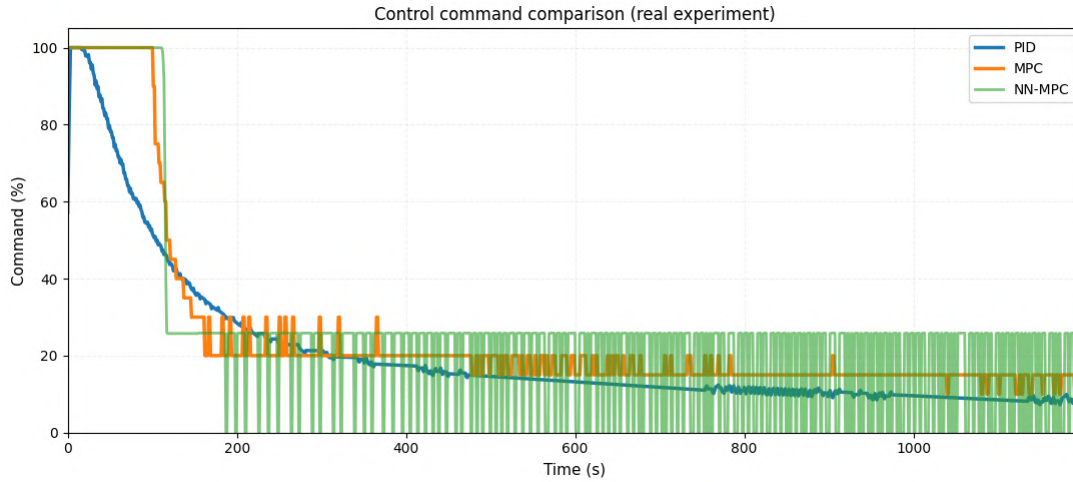


FIGURE 5.6 – Comparaison des commandes PWM appliquées par les contrôleurs PID, MPC et NN-MPC.

La Fig. 5.6 présente l'évolution des commandes appliquées par les trois contrôleurs. La commande NN-MPC est appliquée sous forme de Slow PWM, ce qui explique les commutations observées sur le signal de commande.

Malgré cette forme de commande, la réponse en température reste stable et précise. Cela montre que le Slow PWM est adapté au procédé thermique, car l'inertie du système filtre naturellement les commutations de la lampe.

5.5.3 Synthèse quantitative des performances

TABLE 5.7 – Comparaison expérimentale des performances pour une consigne de 30 °C

Métrique	PID	MPC	NN-MPC	Meilleur
Dépassement (%)	1.33	0.67	0.33	NN-MPC
Temps de montée (s)	148.86	96.98	92.57	NN-MPC
Temps de stabilisation (s)	189.20	117.03	110.37	NN-MPC
Erreur statique (°C)	0.276	0.123	0.045	NN-MPC
MAE (°C)	0.791	0.468	0.419	NN-MPC
Écart-type en régime permanent (°C)	0.043	0.042	0.050	MPC
Indice énergétique (%)	100.0	98.7	97.3	NN-MPC

Le tableau 5.7 confirme quantitativement les observations graphiques. Le NN-MPC présente le plus faible dépassement, le temps de montée le plus court, le temps de stabilisation le plus réduit et l'erreur statique la plus faible.

Le MPC présente une stabilité légèrement meilleure en régime permanent, avec un écart-type de 0.042°C contre 0.050°C pour le NN-MPC. Cependant, le NN-MPC offre un meilleur compromis global, car il améliore la rapidité, la précision et l'énergie utile tout en conservant une exécution beaucoup plus légère sur Raspberry Pi.

Le dépassement de 0.33% observé pour le NN-MPC reste néanmoins très faible et inférieur au seuil de 1% fixé dans les objectifs. Dans le contexte d'un procédé thermique lent et inertiel, cette valeur demeure acceptable, car elle traduit une réponse rapide tout en conservant une précision statique élevée et un comportement globalement stable.

5.6 Analyse de robustesse du contrôleur NN-MPC

Afin d'évaluer la capacité de généralisation du contrôleur NN-MPC, deux tests complémentaires sont réalisés : le suivi de consignes multiples et le rejet de perturbations.

5.6.1 Suivi de consignes multiples

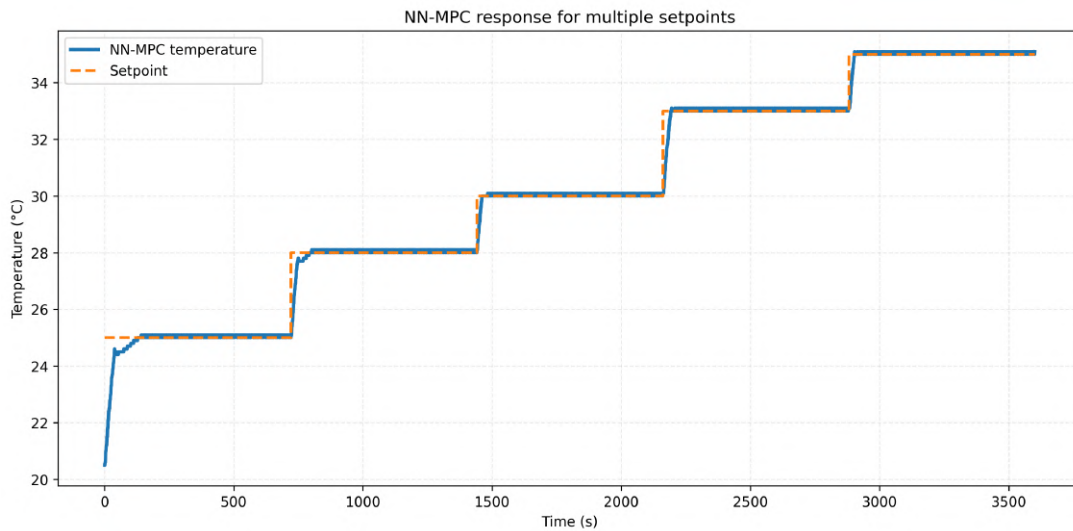


FIGURE 5.7 – Réponse du contrôleur NN-MPC pour cinq consignes successives : 25°C , 28°C , 30°C , 33°C et 35°C .

La Fig. 5.7 montre la réponse du contrôleur NN-MPC pour cinq consignes successives. La température suit correctement chaque changement de consigne avec un dépassement très faible et un régime permanent stable.

Ce résultat confirme que le modèle ne fonctionne pas uniquement pour une seule consigne, mais qu'il conserve de bonnes performances sur toute la plage 25°C à 35°C .

TABLE 5.8 – Performances du NN-MPC pour plusieurs consignes

Consigne y_r (°C)	OS (%)	t_r (s)	t_s (s)	e_{ss} (°C)
25	0.400	38.110	53.910	0.067
28	0.357	25.830	23.570	0.060
30	0.333	16.790	14.540	0.059
33	0.303	28.080	23.570	0.059
35	0.286	19.060	14.540	0.059
Moyenne	0.336	25.574	26.026	0.061

Le tableau 5.8 confirme la capacité du NN-MPC à suivre plusieurs consignes de température. Le dépassement moyen reste très faible, avec une valeur moyenne de 0.336%, et l'erreur statique moyenne est limitée à 0.061 °C.

Ces résultats montrent que le modèle conserve une bonne capacité de généralisation sur la plage de fonctionnement étudiée.

5.6.2 Rejet de perturbations

Afin d'évaluer la robustesse des contrôleurs, une perturbation thermique externe a été appliquée expérimentalement en ouvrant temporairement la fermeture de l'enceinte thermique. Cette action provoque un échange d'air avec l'environnement extérieur et entraîne une baisse de la température interne.

La perturbation est appliquée à l'instant $t = 300$ s pendant une durée de 20 s, soit jusqu'à $t = 320$ s. Elle conduit à une déviation thermique maximale de 0.60 °C pour le PID, de 0.40 °C pour le MPC et de 0.30 °C pour le NN-MPC.

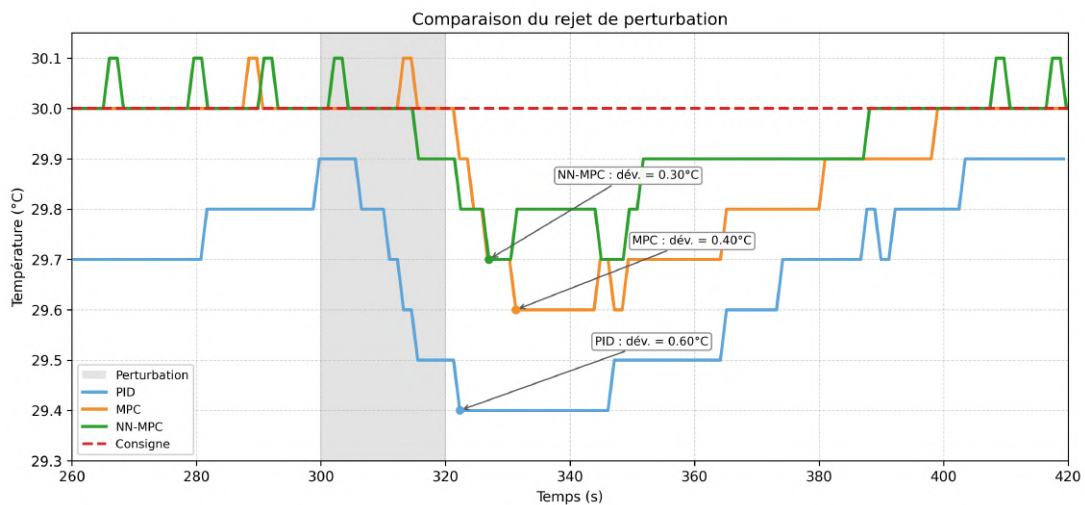


FIGURE 5.8 – Réponse du système face à une perturbation externe pour une consigne de 30 °C.

La Fig. 5.8 met en évidence la réponse des trois contrôleurs face à l'ouverture temporaire de l'enceinte. Le PID présente la chute de température la plus marquée, atteignant 29.40 °C. Le MPC limite davantage cette baisse, avec une température minimale de 29.60 °C, tandis que le NN-MPC conserve la meilleure robustesse, avec un minimum de 29.70 °C.

Dans cette analyse, le temps de récupération est défini comme le délai nécessaire, après la fin de la perturbation à $t = 320$ s, pour que la température revienne dans la bande de précision comprise entre 29.9 °C et 30.1 °C, correspondant à une tolérance de ± 0.1 °C autour de la consigne.

TABLE 5.9 – Performances de rejet de perturbation pour une consigne de 30 °C

Métrique	PID	MPC	NN-MPC
Déviati on maximale (°C)	0.60	0.40	0.30
Température minimale observée (°C)	29.40	29.60	29.70
Temps de récupération après perturbation	≈ 83.48 s	≈ 60.98 s	≈ 31.79 s

Le tableau 5.9 confirme que le NN-MPC présente la meilleure capacité de rejet de perturbation. Il limite la déviation maximale à 0.30 °C, contre 0.40 °C pour le MPC et 0.60 °C pour le PID.

Le NN-MPC retrouve également la bande de précision autour de la consigne en environ 31.79 s, alors que le MPC nécessite environ 60.98 s et le PID environ 83.48 s. Ces résultats montrent que le contrôleur NN-MPC atténue plus efficacement l'effet de la perturbation externe et rétablit plus rapidement une régulation précise.

5.7 Analyse énergétique

L'analyse énergétique vise à comparer l'effort de commande nécessaire pour conduire le système thermique jusqu'au régime stabilisé. Dans ce travail, il ne s'agit pas d'une mesure directe de l'énergie électrique dissipée par la lampe en joules, mais d'un **indice énergétique relatif** construit à partir de la commande de chauffage appliquée par chaque contrôleur.

Pour un contrôleur i , l'effort de commande utile est évalué jusqu'au temps de stabilisation $t_{s,i}$ selon la relation continue suivante :

$$E_i = \int_0^{t_{s,i}} u_i(t) dt \quad (5.1)$$

où $u_i(t)$ représente la commande de chauffage normalisée dans l'intervalle $[0, 1]$, et $t_{s,i}$ désigne le temps de stabilisation du contrôleur considéré.

Dans le traitement numérique des résultats expérimentaux, la commande initialement exprimée en pourcentage est d'abord normalisée par :

$$u_i(k) = \frac{u_{\%,i}(k)}{100} \quad (5.2)$$

L'effort de commande utile est ensuite calculé sous forme discrète par :

$$E_i \approx \sum_{k=0}^{N_{s,i}} u_i(k) \Delta t_k \quad (5.3)$$

où Δt_k représente l'intervalle de temps entre deux échantillons successifs, et $N_{s,i}$ l'indice correspondant au temps de stabilisation.

Dans l'analyse réalisée, le temps de stabilisation est détecté lorsque la température reste dans une bande de tolérance de $\pm 0.5^\circ\text{C}$ autour de la consigne pendant une durée minimale de 30 s. Ce critère correspond à celui appliqué dans le traitement des fichiers expérimentaux.

Afin de faciliter la comparaison entre les trois contrôleurs, l'indice énergétique est normalisé par rapport au PID :

$$I_{E,i} = \frac{E_i}{E_{\text{PID}}} \times 100 \quad (5.4)$$

Le PID est choisi comme référence de normalisation, car il constitue la stratégie de commande classique utilisée comme base de comparaison dans l'étude expérimentale. Ainsi, les indices du MPC et du NN-MPC expriment directement la réduction relative de l'effort de commande par rapport à cette référence. Un indice inférieur à 100% indique donc une sollicitation plus faible de l'actionneur thermique jusqu'au temps de stabilisation.

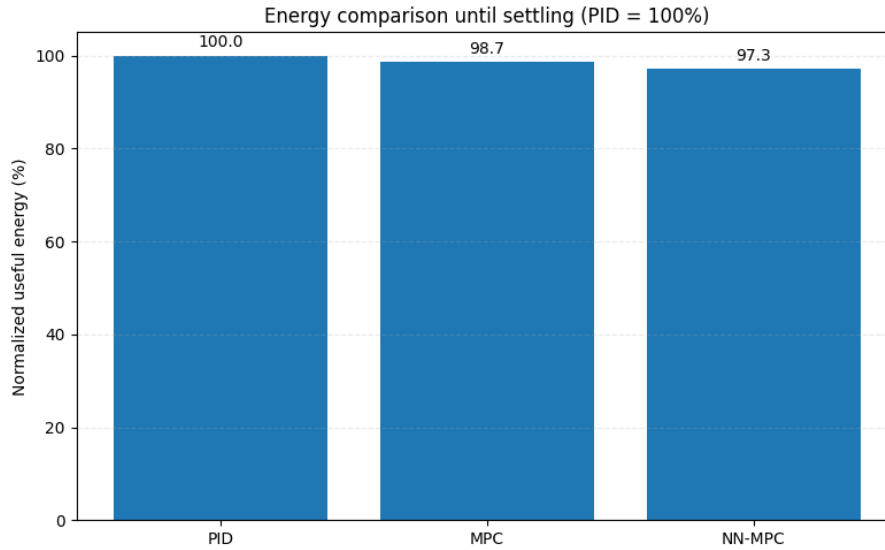


FIGURE 5.9 – Comparaison de l'indice énergétique utile jusqu'au temps de stabilisation.

La Fig. 5.9 montre que le NN-MPC présente l'indice énergétique utile le plus faible, avec 97.3%, contre 98.7% pour le MPC et 100% pour le PID pris comme référence.

Ces résultats signifient que, selon le critère défini par les équations 5.1 à 5.4, le NN-MPC atteint le régime stabilisé avec un effort de commande légèrement inférieur à celui des autres contrôleurs. Plus précisément, il réduit l'indice énergétique relatif de 2.7% par rapport au PID, tandis que le MPC présente une réduction de 1.3%.

Cette tendance s'explique par une commande plus anticipative et mieux adaptée à la dynamique thermique de la maquette. Le NN-MPC limite ainsi la sollicitation cumulée de l'actionneur tout en conservant de bonnes performances de rapidité et de précision.

5.8 Résultats fonctionnels de la supervision IoT

En complément des performances de régulation, la supervision IoT développée autour de Node-RED, MQTT, InfluxDB et Grafana a été validée fonctionnellement.

L'objectif de cette section est de montrer que le système ne se limite pas à l'exécution du contrôleur NN-MPC, mais qu'il constitue une plateforme complète permettant le suivi temps réel, le pilotage à distance, l'analyse historique, le diagnostic automatique et la gestion du service embarqué.

5.8.1 Dashboard de supervision temps réel

Le dashboard Node-RED représente l'interface principale entre l'utilisateur et le système thermique. Il regroupe les grandeurs essentielles du procédé : température mesurée, humidité, consigne, commande PWM, état MQTT, état de la base InfluxDB, état du service et dernier message reçu.

Cette interface permet à l'utilisateur de suivre le comportement du système sans accéder directement au terminal de la Raspberry Pi. Elle facilite également les essais expérimentaux, car les paramètres de régulation et les états du système sont centralisés dans une même interface.

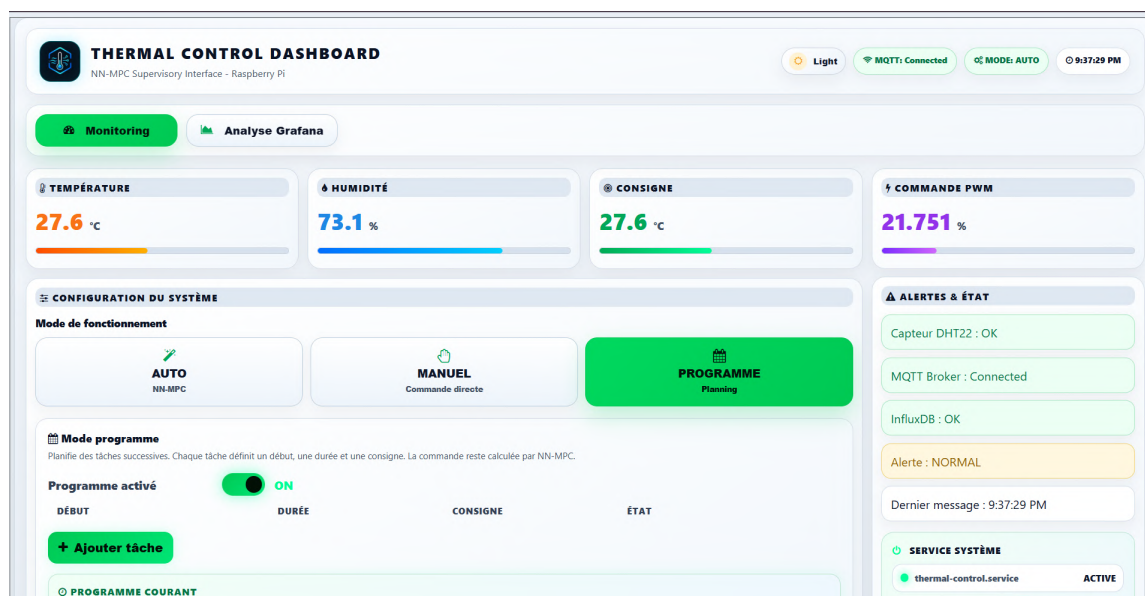


FIGURE 5.10 – Dashboard Node-RED principal développé pour la supervision temps réel du système thermique NN-MPC.

La Fig. 5.10 montre l'interface principale de supervision. Les cartes de monitoring permettent de visualiser immédiatement l'état thermique du système et la commande calculée par le contrôleur. Les indicateurs d'état permettent de vérifier la connectivité MQTT, la disponibilité d'InfluxDB, l'état du capteur et l'activité du service embarqué.

5.8.2 Validation des modes de fonctionnement

Le dashboard intègre trois modes de fonctionnement : **AUTO**, **MANUEL** et **PROGRAMME**. Ces modes permettent d'adapter l'utilisation du système selon l'objectif de l'essai.

En mode **AUTO**, l'utilisateur définit uniquement la consigne de température. La commande PWM reste calculée automatiquement par le contrôleur NN-MPC. Ce mode correspond au fonctionnement normal du système.

The screenshot shows the 'CONFIGURATION DU SYSTÈME' interface. At the top, there are three tabs: 'AUTO NN-MPC' (highlighted in green), 'MANUEL Commande directe', and 'PROGRAMME Planning'. Below the tabs, the 'Mode automatique' section is active. It contains a text input for 'Consigne température' with the value '27'. To its right, a 'Commande PWM' section shows 'Calculée par NN-MPC'. Below these, a slider for 'Réglage rapide : 27 °C' is shown. A note states: 'En AUTO, aucune commande manuelle n'est envoyée. La commande PWM reste générée par le contrôleur NN-MPC.' At the bottom, a green button labeled 'APPLIQUER' is visible, with a note above it: 'APPLIQUER envoie la consigne et les tâches programme au flow Node-RED.'

FIGURE 5.11 – Mode AUTO : modification de la consigne avec commande PWM calculée automatiquement par le contrôleur NN-MPC.

En mode **MANUEL**, l'utilisateur peut imposer directement une puissance de chauffage. Ce mode est utile pour les essais, la calibration et le diagnostic de la maquette.

The screenshot shows the 'CONFIGURATION DU SYSTÈME' interface in 'Mode manuel'. The 'MANUEL Commande directe' tab is highlighted in green. The 'Mode manuel' section is active, with a note: 'Ce mode sert aux essais, calibration et diagnostic. L'opérateur peut imposer une commande PWM.' It features two input fields: 'Consigne température' with the value '27' and 'Commande PWM (%)' with the value '30'. Below these, there are two sliders: 'Consigne : 27 °C' and 'Commande manuelle : 30 %'. A note at the bottom states: 'En MANUEL, la commande peut agir directement sur la maquette. Utilisez-le seulement pendant les tests contrôlés.' A green 'APPLIQUER' button is at the bottom, with a note above it: 'APPLIQUER envoie la consigne et les tâches programme au flow Node-RED.'

FIGURE 5.12 – Mode MANUEL : imposition directe de la commande PWM pour les essais et les diagnostics.

En mode **PROGRAMME**, l'utilisateur définit des tâches temporelles comprenant une heure de début, une durée et une consigne. Le système applique automatiquement les consignes planifiées à travers l'ordonnanceur Node-RED.

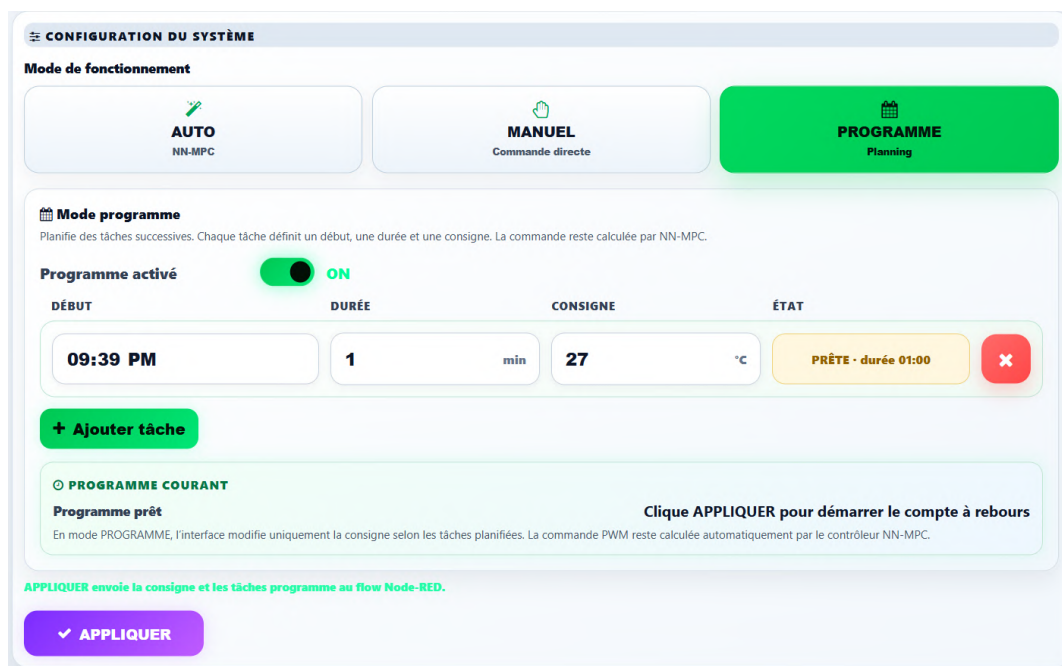


FIGURE 5.13 – Mode PROGRAMME : planification temporelle des consignes appliquées automatiquement par Node-RED.

Les Fig. 5.11, 5.12 et 5.13 confirment que les trois modes sont intégrés dans l'interface de supervision. Cette organisation rend le système plus flexible, car il peut être utilisé à la fois pour la régulation automatique, les tests manuels et la planification temporelle.

5.8.3 Gestion du service embarqué

Le dashboard permet également de gérer le service système `thermal-control.service`, qui exécute le programme Python de contrôle sur la Raspberry Pi.

L'utilisateur peut démarrer, arrêter, redémarrer ou vérifier l'état du service directement depuis l'interface. Cette fonctionnalité améliore l'ergonomie du système, car elle évite l'utilisation répétée du terminal pendant les essais expérimentaux.



FIGURE 5.14 – Module de gestion du service `thermal-control.service` intégré dans le dashboard.

La Fig. 5.14 montre le module de gestion du service. L'état affiché provient de la com-

mande `systemctl is-active`, ce qui permet de vérifier l'exécution réelle du programme embarqué.

5.8.4 Analyse historique intégrée avec Grafana

L'analyse historique est assurée par Grafana, intégré directement dans Node-RED à travers une iframe. Les données mesurées sont stockées dans InfluxDB, puis affichées dans Grafana sous forme de courbes temporelles.

Cette intégration permet d'observer l'évolution de la température, de la consigne, de la commande PWM, de l'humidité et de l'erreur de régulation sur différentes fenêtres temporelles.

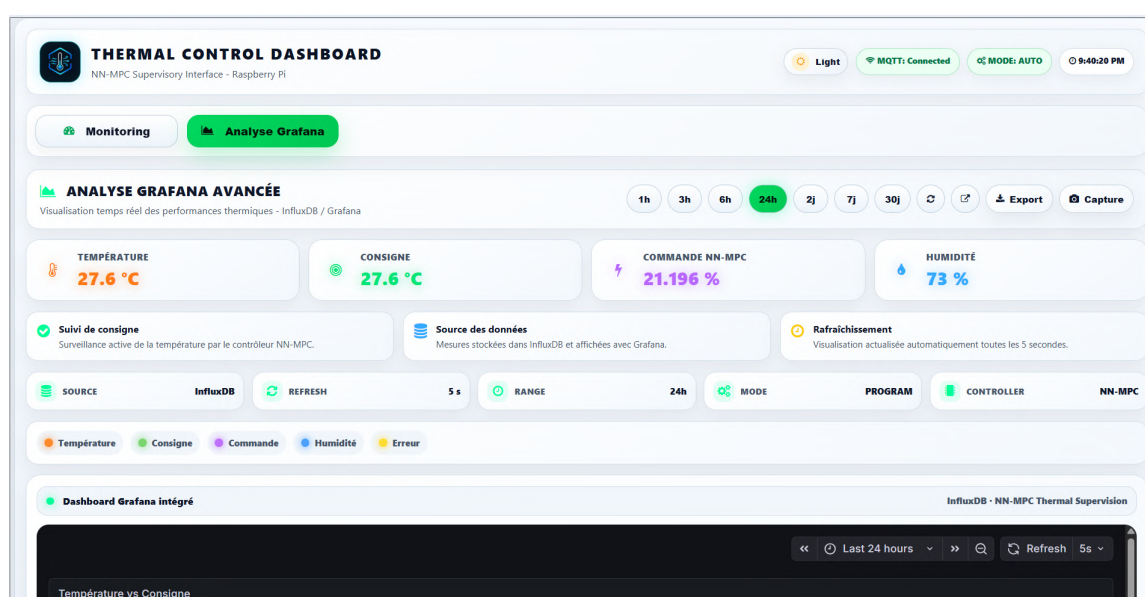


FIGURE 5.15 – Page d'analyse Grafana intégrée dans Node-RED avec filtres temporels et visualisation des données expérimentales.

La Fig. 5.15 montre la page d'analyse historique. Les filtres temporels permettent de sélectionner des périodes comme 1 h, 3 h, 24 h, 7 jours ou 30 jours. Cette fonctionnalité facilite l'exploitation des données expérimentales et l'analyse des performances du système.

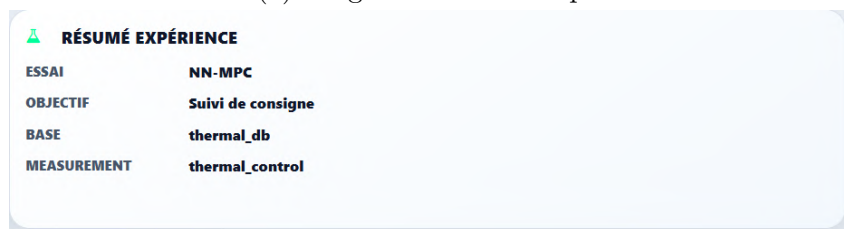
5.8.5 Diagnostic automatique et résumé expérimental

La supervision intègre aussi des modules de diagnostic automatique. Ces modules permettent de vérifier l'état global du système, la synchronisation des données, l'état de la commande, la source de stockage et le contrôleur actif.

Un résumé expérimental est également affiché afin de rappeler le type d'essai, l'objectif, la base de données utilisée, la mesure InfluxDB et le contrôleur en cours d'utilisation.



(a) Diagnostic automatique



(b) Résumé expérimental

FIGURE 5.16 – Modules de diagnostic automatique et de résumé expérimental intégrés dans la supervision IoT.

La Fig. 5.16 présente les modules de diagnostic et de résumé. Ces éléments renforcent la lisibilité de l'interface et permettent d'identifier rapidement l'état du système pendant les essais.

5.8.6 Synthèse de la validation fonctionnelle

Les résultats fonctionnels obtenus montrent que la couche IoT développée permet :

- le monitoring temps réel de la température, de l'humidité, de la consigne et de la commande PWM ;
- le pilotage du système à travers les modes AUTO, MANUAL et PROGRAMME ;
- la gestion du service embarqué `thermal-control.service` ;
- l'analyse historique des données via Grafana et InfluxDB ;
- le diagnostic automatique de l'état du système ;
- la centralisation des informations expérimentales dans une interface unique.

Ces résultats confirment que la supervision IoT apporte une valeur importante au projet. Elle transforme la maquette thermique en une plateforme complète de régulation, de suivi, d'analyse et de diagnostic.

5.9 Discussion générale

Les résultats obtenus confirment la pertinence de l'approche NN-MPC pour la régulation thermique embarquée. Le modèle entraîné approxime efficacement la loi de commande MPC, avec un coefficient de détermination R^2 de 98,06%. Cette performance montre que le réseau de neurones est capable de reproduire le comportement d'un contrôleur prédictif de référence.

La conversion vers TensorFlow Lite renforce l'intérêt de cette approche pour l'embarqué. La taille du modèle passe de 48,96 KB à 5,78 KB, tandis que le temps d'inférence sur Raspberry Pi 4 atteint environ 0,015 ms. **Ce résultat respecte très largement l'objectif de latence fixé à moins de 100 ms.** Le contrôleur NN-MPC peut ainsi fonctionner en temps réel avec une charge mémoire et CPU très faible.

Sur le plan expérimental, le NN-MPC présente les meilleures performances globales pour une consigne de 30 °C. Il obtient le plus faible dépassement, le temps de montée le plus court, le temps de stabilisation le plus réduit et la plus faible erreur statique. L'erreur statique mesurée pour le NN-MPC est de 0,045 °C. **Ce résultat respecte l'objectif de précision fixé à $\pm 0,1$ °C.** De même, le dépassement observé est limité à 0,33%. **Ce résultat respecte l'objectif de dépassement thermique inférieur à 1%.** L'indice énergétique utile est également le plus faible, avec 97,3% par rapport au PID pris comme référence.

Les essais de robustesse confirment la capacité de généralisation du contrôleur. Sur la plage 25 °C à 35 °C, l'erreur statique moyenne reste limitée à 0,061 °C. Lors du test de perturbation, le NN-MPC limite la déviation maximale à 0,30 °C et conserve une erreur statique après perturbation de 0,036 °C.

Ces résultats montrent que le NN-MPC constitue une alternative pertinente au MPC classique dans un contexte embarqué. Il conserve les avantages d'une commande prédictive tout en remplaçant l'optimisation en ligne par une inférence rapide du réseau de neurones.

Les performances obtenues s'inscrivent également dans une tendance plus large d'utilisation des approches prédictives et de l'apprentissage pour améliorer la régulation des systèmes thermiques à forte inertie. La littérature montre en effet que ces stratégies peuvent contribuer à améliorer le suivi de consigne, la robustesse et l'efficacité énergétique dans les applications thermiques et énergétiques [9, 10, 11].

Enfin, la supervision IoT développée autour de Node-RED, InfluxDB et Grafana apporte une valeur pratique importante au système. Elle permet le suivi temps réel, le pilotage des modes de fonctionnement, la gestion du service embarqué, l'analyse historique et le diagnostic automatique.

Ainsi, le projet propose une plateforme complète intégrant régulation intelligente, déploiement embarqué, supervision IoT, stockage des données et analyse des performances.

5.10 Conclusion

Ce chapitre a présenté les principaux résultats obtenus lors de l'apprentissage, de l'évaluation expérimentale et de la validation fonctionnelle du système.

Les résultats montrent que le modèle NN-MPC approxime efficacement la commande MPC de référence, avec un score R^2 de 98.06%, tout en restant léger après conversion vers TensorFlow Lite. Cette caractéristique confirme son adaptation à une exécution embarquée

sur Raspberry Pi.

La comparaison expérimentale avec les contrôleurs PID et MPC montre que le NN-MPC offre le meilleur compromis entre rapidité, précision, faible dépassement et énergie utile. Les tests de suivi multi-consignes et de rejet de perturbation confirment également sa robustesse.

Enfin, la supervision IoT basée sur Node-RED, InfluxDB et Grafana a permis de valider le suivi temps réel, le pilotage des modes de fonctionnement, l'analyse historique et le diagnostic automatique.

Ainsi, l'approche NN-MPC embarquée avec supervision IoT constitue une solution efficace et adaptée à la commande intelligente d'un système thermique réel.

Conclusion générale

Ce projet de fin d'études a porté sur la conception et la validation d'un système complet de régulation thermique intelligente, associant commande avancée, intelligence artificielle embarquée et supervision IoT.

Le travail réalisé s'est construit progressivement autour de cinq étapes complémentaires. Le premier chapitre a permis de définir le cadre général du projet, de formuler la problématique de commande thermique embarquée et de positionner l'approche proposée par rapport aux solutions existantes. Le deuxième chapitre a traduit cette problématique en besoins fonctionnels et non fonctionnels, en précisant les acteurs, les contraintes de fonctionnement et les exigences de supervision. Le troisième chapitre a présenté la modélisation thermique de la maquette, la génération d'un dataset à partir d'un contrôleur MPC de référence, ainsi que la conception méthodologique du modèle NN-MPC. Le quatrième chapitre a décrit l'implémentation matérielle et logicielle du système, depuis la Raspberry Pi et les composants de commande jusqu'à l'architecture MQTT, Node-RED, InfluxDB et Grafana. Enfin, le cinquième chapitre a permis de valider expérimentalement les performances du contrôleur proposé et le bon fonctionnement de la plateforme de supervision.

L'une des contributions principales de ce travail réside dans le choix d'une approche intermédiaire entre les méthodes classiques et les stratégies prédictives complexes. Comme discuté dans l'étude de l'existant, le régulateur PID présente l'avantage d'une mise en œuvre simple, mais ses performances deviennent limitées pour un procédé thermique lent et soumis à des contraintes de précision. À l'inverse, le MPC offre une meilleure capacité d'anticipation et de gestion du comportement dynamique, mais son coût de calcul reste pénalisant pour une exécution embarquée en temps réel.

Dans ce contexte, l'approche retenue consiste à apprendre la loi de commande d'un MPC de référence à l'aide d'un réseau de neurones léger, puis à déployer ce modèle sur Raspberry Pi sous forme TensorFlow Lite. Cette démarche permet de transférer hors ligne la complexité du calcul d'optimisation, afin de conserver en ligne une commande rapide et compatible avec les contraintes d'un système embarqué. Ainsi, le projet ne se limite pas à comparer plusieurs contrôleurs : il propose une architecture de commande cohérente, pensée dès l'origine pour concilier performance de régulation, faisabilité embarquée et exploitabilité expérimentale.

Une seconde contribution importante concerne l'intégration d'une chaîne IoT complète autour de la boucle de contrôle. La solution développée ne se réduit pas à un algorithme isolé : elle inclut la communication MQTT, la gestion des modes de fonctionnement, le pilotage du service embarqué, le stockage structuré des données et leur visualisation historique. Cette architecture donne au système une dimension applicative plus large,

proche d'une plateforme de supervision industrielle simplifiée, capable d'être utilisée pour l'observation, le diagnostic et l'analyse des essais.

Les validations expérimentales ont confirmé l'intérêt de cette démarche. Le modèle NN-MPC a montré sa capacité à approximer efficacement la commande prédictive de référence, tandis que son déploiement embarqué a permis une exécution légère et rapide. Les essais comparatifs, les tests multi-consignes, l'analyse du rejet de perturbation et l'évaluation énergétique ont montré que l'approche proposée constitue une alternative pertinente pour la régulation thermique intelligente sur plateforme à ressources limitées.

Au-delà des résultats obtenus, ce travail ouvre plusieurs perspectives d'amélioration, qui peuvent être hiérarchisées selon leur horizon de mise en œuvre.

Perspectives à court terme : Dans le prolongement direct de ce projet, une première évolution consisterait à enrichir la base d'apprentissage à partir de campagnes expérimentales plus variées, incluant des perturbations thermiques contrôlées, des variations de température ambiante et des régimes de fonctionnement plus longs. Cela permettrait d'améliorer la généralisation du modèle et de mieux couvrir les situations rencontrées en conditions réelles.

Une seconde amélioration immédiate concernerait le renforcement de la sûreté de fonctionnement. Il serait pertinent d'ajouter des contraintes explicites de sécurité, telles que des limites thermiques renforcées, une détection plus avancée des anomalies capteur/actionneur et des stratégies de repli automatique en cas de comportement incohérent du modèle ou de perte de communication.

Perspectives à moyen terme : À un horizon plus avancé, une première piste porterait sur l'adaptation en ligne du contrôleur. Le modèle actuel est appris hors ligne à partir d'un dataset fixe ; l'intégration de mécanismes de ré-entraînement périodique ou d'apprentissage incrémental permettrait d'ajuster progressivement la commande en fonction du vieillissement des composants, des dérives du procédé ou des changements de configuration thermique.

Une deuxième perspective de recherche pourrait analyser plus précisément l'influence réelle de l'humidité sur la dynamique thermique de l'enceinte, à travers un modèle thermo-hygrométrique plus complet ou une étude comparative avec et sans cette variable d'entrée.

Enfin, l'approche pourrait être étendue vers des systèmes thermiques de plus grande échelle ou vers des environnements industriels plus représentatifs. L'évaluation du NN-MPC sur d'autres procédés à forte inertie, l'étude de techniques de compression supplémentaires du modèle, ainsi que le renforcement de la cybersécurité de la supervision distante constitueraient des prolongements naturels de ce travail.

En définitive, ce projet démontre l'intérêt d'une démarche intégrée associant modélisation physique, approximation neuronale d'une commande prédictive, déploiement embarqué et supervision IoT. Il fournit une base solide pour le développement de solutions de régulation intelligentes, légères et connectées, adaptées aux systèmes thermiques réels.

Bibliographie et références

- [1] S. J. Qin and T. A. Badgwell, *A survey of industrial model predictive control technology*, Control Engineering Practice, vol. 11, no. 7, pp. 733–764, 2003.
- [2] D. Q. Mayne, J. B. Rawlings, C. V. Rao and P. O. M. Scokaert, *Constrained model predictive control : Stability and optimality*, Automatica, vol. 36, no. 6, pp. 789–814, 2000.
- [3] J. B. Rawlings, D. Q. Mayne and M. M. Diehl, *Model Predictive Control : Theory, Computation, and Design*, 2nd edition, Nob Hill Publishing, 2017.
- [4] S. Piché, B. Sayyar-Rodsari, D. Johnson and M. Gerules, *Neural Network Based Model Predictive Control*, Advances in Neural Information Processing Systems, 1999.
- [5] G. Lightbody and G. W. Irwin, *Nonlinear control structures based on embedded neural system models*, IEEE Transactions on Neural Networks, vol. 8, no. 3, pp. 553–567, 1997.
- [6] A. Bemporad, M. Morari, V. Dua and E. N. Pistikopoulos, *The explicit linear quadratic regulator for constrained systems*, Automatica, vol. 38, no. 1, pp. 3–20, 2002.
- [7] E. T. Maddalena, C. G. da S. Moraes, G. Waltrich and C. N. Jones, *A neural network architecture to learn explicit MPC controllers from data*, IFAC-PapersOnLine, vol. 53, no. 2, pp. 11362–11367, 2020.
- [8] M. Ławryńczuk, *Neural Networks in Model Predictive Control*, in Nonlinear Predictive Control Using Wiener Models, Springer, 2009.
- [9] M. Killian and M. Kozek, *Ten questions concerning model predictive control for energy efficient buildings*, Building and Environment, vol. 105, pp. 403–412, 2016.
- [10] Z. Afroz, G. M. Shafiullah, T. Urmees and G. Higgins, *Modeling techniques used in building HVAC control systems : A review*, Renewable and Sustainable Energy Reviews, vol. 83, pp. 64–84, 2018.
- [11] J. Drgoňa, J. Arroyo, I. C. Figueroa, D. Blum, K. Arendt, D. Kim, E. P. Ollé, J. Oravec, M. Wetter, D. L. Vrabie and L. Helsén, *All you need to know about model predictive control for buildings*, Annual Reviews in Control, vol. 50, pp. 190–232, 2020.
- [12] Informatique pour tous, *Le cycle en V*. Disponible sur : https://fr.wikipedia.org/wiki/Cycle_en_V (consulté le 17 février 2026)
- [13] Object Management Group, *Unified Modeling Language Specification*. Disponible sur : <https://www.omg.org/spec/UML/> (consulté le 22 février 2026)
- [14] Raspberry Pi Foundation, *Raspberry Pi Documentation*. Disponible sur : <https://www.raspberrypi.com/documentation/> (consulté le 8 février 2026)

- [15] Raspberry Pi Foundation, *Raspberry Pi 4 Model B Specifications*. Disponible sur : <https://www.raspberrypi.com/products/raspberry-pi-4-model-b/specifications/> (consulté le 10 février 2026)
- [16] Aosong Electronics, *DHT22 / AM2302 Digital Temperature and Humidity Sensor Datasheet*. Disponible sur : <https://cdn.sparkfun.com/assets/f/7/d/9/c/DHT22.pdf> (consulté le 9 février 2026)
- [17] Fotek, *Solid State Relay SSR Datasheet*. Disponible sur : <https://www.fotek.com.tw/en/product/solid/SSR-1.htm> (consulté le 18 février 2026)
- [18] Python Software Foundation, *Python Documentation*. Disponible sur : <https://docs.python.org/3/> (consulté le 17 février 2026)
- [19] TensorFlow, *TensorFlow Documentation*. Disponible sur : <https://www.tensorflow.org/> (consulté le 17 février 2026)
- [20] Keras, *Keras Documentation*. Disponible sur : <https://keras.io/> (consulté le 15 février 2026)
- [21] TensorFlow, *TensorFlow Lite Guide*. Disponible sur : <https://www.tensorflow.org/lite> (consulté le 21 février 2026)
- [22] GEKKO, *GEKKO Optimization Suite Documentation*. Disponible sur : <https://gekko.readthedocs.io/> (consulté le 20 février 2026)
- [23] MQTT.org, *MQTT Protocol Documentation*. Disponible sur : <https://mqtt.org/> (consulté le 22 février 2026)
- [24] Eclipse Mosquitto, *Eclipse Mosquitto MQTT Broker Documentation*. Disponible sur : <https://mosquitto.org/documentation/> (consulté le 23 février 2026)
- [25] Node-RED, *Node-RED Documentation*. Disponible sur : <https://nodered.org/docs/> (consulté le 26 février 2026)
- [26] InfluxData, *InfluxDB Documentation*. Disponible sur : <https://docs.influxdata.com/> (consulté le 26 février 2026)
- [27] Grafana Labs, *Grafana Documentation*. Disponible sur : <https://grafana.com/docs/> (consulté le 28 février 2026)
- [28] Freedesktop.org, *systemd Documentation*. Disponible sur : <https://www.freedesktop.org/wiki/Software/systemd/> (consulté le 28 février 2026)